Evaluación 2 — Auditoría de Seguridad de Software

STRIDE · SAST · DAST · SCA · Hardening

Integrantes:

Mauricio Alcantara Mata

Valeria Hernandez Cortes

Ramírez Ramírez Elian Eduardo

Luz Celeste López Rivera

Grupo:

IDGS17

Docente:

Gerardo Ramírez Villarreal

Materia:

Seguridad en el Desarrollo de Aplicaciones

INFORME DE AUDITORÍA DE SEGURIDAD

ÍNDICE

Contenido

Resumen ejecutivo 1

SA.1. Dominio Conceptual Aplicado 3

Static Application Security Testing (SAST) 4

Definición de SAST 4

Objetivos del análisis SAST 4

Aplicación de SAST en InterLockerUp 5

Funcionamiento interno del Static Application Security Testing (SAST) ... 5

Pattern Matching 7

Taint Analysis 8

Control Flow Analysis (Análisis del Flujo de Control) 9

Figura 1.2. Ejemplo conceptual del análisis del flujo de control 11

Relación entre Pattern Matching, Taint Analysis y Control Flow Analysis 11

Ventajas del Static Application Security Testing (SAST) 12

Aplicación práctica en InterLockerUp 15

Desventajas y limitaciones del Static Application Security Testing (SAST) 15

Limitaciones observadas en InterLockerUp 18

¿Por qué SAST no es suficiente? 18

Comparación entre SAST, DAST, IAST y SCA 19

¿Qué analiza cada metodología? 20

¿Cuándo debe utilizarse cada una? 21

Dynamic Application Security Testing (DAST) 23

Definición de DAST 24

Objetivos del análisis DAST 24

Aplicación de DAST en InterLockerUp 25

Funcionamiento interno del Dynamic Application Security Testing (DAST) 26
Crawling (Descubrimiento de Recursos) 27
Passive Scanning (Análisis Pasivo) 28
Active Scanning (Análisis Activo) 29
Objetivos del Active Scanning 30
Principales vulnerabilidades detectadas mediante Active Scanning 31
¿Cómo funciona internamente el Active Scanning? 33
Aplicación en InterLockerUp 34
Fuzzing (Pruebas mediante entradas aleatorias) 35
Objetivos del Fuzzing 36
¿Cómo funciona internamente el Fuzzing? 36
Tipos de entradas utilizadas durante el Fuzzing 37
Aplicación del Fuzzing en InterLockerUp 38
Ventajas del Fuzzing 39
Limitaciones del Fuzzing 39
Ventajas del Dynamic Application Security Testing (DAST) 39
Aplicación práctica en InterLockerUp 41
Desventajas y limitaciones del Dynamic Application Security Testing (DAST) 42
Comparación entre SAST y DAST 43
Interactive Application Security Testing (IAST) 44
Objetivos del análisis IAST 45
¿Cómo funciona IAST? 46
Diferencias entre IAST y DAST 47
Tabla 1.7. Comparación entre DAST e IAST 48
Aplicación en InterLockerUp 48
Ventajas del IAST 49
Desventajas del IAST 49
Software Composition Analysis (SCA) 49

Definición de Software Composition Analysis (SCA) 50
Objetivos del SCA 50
¿Cómo funciona el SCA? 51
Aplicación del SCA en InterLockerUp 53
Ventajas del SCA 54
Desventajas del SCA 54
Modelo de amenazas STRIDE 55
Definición de STRIDE 56
Spoofing — Suplantación de identidad 56
Tampering — Alteración de información 58
Information Disclosure — Divulgación de información 60
Denial of Service — Denegación del servicio 61
Elevation of Privilege — Elevación de privilegios 62
Resumen de amenazas STRIDE aplicadas a InterLockerUp 63
Importancia de STRIDE en InterLockerUp 64
Shift Left Security 64
Definición de Shift Left Security 65
Objetivos de Shift Left Security 66
Beneficios de Shift Left Security 66
Aplicación de Shift Left Security en InterLockerUp 68
Relación entre Shift Left Security y las metodologías de análisis 68
DevSecOps y el pipeline de desarrollo seguro 69
Definición de DevSecOps 70
¿Qué es un pipeline DevSecOps? 70
Etapas del pipeline DevSecOps 70
Aplicación del pipeline DevSecOps en InterLockerUp 73
Beneficios del pipeline DevSecOps 74
Falsos positivos y falsos negativos en las herramientas de análisis de seguridad 75
Definición de falso positivo 75

Características de un falso positivo 75
Definición de falso negativo 75
Características de un falso negativo 76
Comparación entre falso positivo y falso negativo 76
Ejemplos aplicados a InterLockerUp 77
Estrategias para reducir falsos positivos y falsos negativos 78
Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP) y su aplicación al desarrollo de software 79
¿Qué es la LFPDPPP? 79
¿Qué son los datos personales? 79
Aviso de privacidad 80
Derechos ARCO 82
Datos personales tratados por InterLockerUp 83
Relación entre la LFPDPPP y el desarrollo seguro 83
SA.2 — Modelado de amenazas STRIDE aplicado al caso de estudio 84
Introducción 85
Descripción del sistema 85
Diagrama de Flujo de Datos (DFD) 86
Diagrama de Flujo de Datos (DFD) Nivel 1 87
Límites de confianza (Trust Boundaries) 88
Tabla STRIDE 90
Análisis de las amenazas críticas 92
Amenaza crítica 1 – Robo de credenciales (Spoofing) 92
Amenaza crítica 2 – Manipulación de la API REST (Tampering) 92
Amenaza crítica 3 – Denegación de servicio sobre la API (Denial of Service) 93
Análisis 93
Controles de mitigación 93
Análisis 94
SA.3 — SAST aplicado al caso de estudio (SonarQube + Semgrep) 95

Resultados del análisis con SonarQube 96
Hallazgos de seguridad 97
Hallazgo 1. Exposición de información del framework 97
Hallazgo 2. Riesgo de saturación por límite de carga 98
Cobertura de pruebas 98
Resumen y Desglose de Cobertura de Pruebas Unitarias 98
Análisis con Semgrep 99
¿Qué es Semgrep y para qué sirve? 99
Proceso de Configuración y Ejecución del Análisis 99
Resultados del Análisis con Semgrep 100
SA.4 — DAST aplicado al caso de estudio (OWASP ZAP) 101
¿Qué es OWASP ZAP y para qué sirve? 102
Instalación 102
Escaneo DAST 102
Reporte ZAP y Clasificación de Riesgos 103
Análisis de Alertas 105
1. Alerta High 105
2. Cabecera de Seguridad HTTP Ausente Identificada 106
SA.5 — SCA: análisis de dependencias del caso de estudio 106
Introducción y Metodología 107
Herramienta SCA Ejecutada 107
Reporte generado: Diagnóstico de Vulnerabilidades 107
Documentación Detallada de Dependencias Vulnerables 108
Proceso de Actualización y Justificación Técnica de Remediación 111
Conclusión 112
SA.6 — Configuración segura y hardening del caso de estudio 112
Introducción 113
Cabeceras HTTP de seguridad revisadas e implementadas 113
Gestión de secretos revisada 115

Logging de eventos de seguridad básico implementado 115
Configuración de defaults seguros verificada 116
Checklist de defensa en profundidad documentado 117
Conclusión 118
Tabla de trazabilidad básica 118

Resumen ejecutivo

El presente informe detalla los resultados y las acciones correctivas derivadas de la auditoría integral de seguridad aplicada al proyecto **InterLockerUp**. Esta plataforma, diseñada para la gestión de casilleros inteligentes, opera mediante una arquitectura que integra un frontend en React, un backend en Node.js, una base de datos relacional en PostgreSQL y componentes de hardware IoT (ESP32) para el control físico de las cerraduras.

El objetivo principal de esta evaluación es garantizar la confidencialidad, integridad y disponibilidad del sistema y de los datos de los estudiantes, alineando el desarrollo con prácticas modernas de ciberseguridad. Para lograrlo, la auditoría se estructuró bajo un enfoque de defensa en múltiples capas, aplicando las siguientes metodologías:

- **Modelado de Amenazas (STRIDE):** Se mapeó la arquitectura del sistema para prever y mitigar vectores de ataque antes de su explotación, enfocándose en riesgos críticos como la suplantación de identidad (Spoofing), la alteración de datos (Tampering) y la denegación de servicio (DoS).
- **Análisis Estático (SAST):** Mediante el uso de herramientas como SonarQube y Semgrep, se escaneó el código fuente. Esto permitió corregir fallos estructurales tempranos, como la exposición de información del framework (CWE-200) y la falta de límites en la carga de archivos (CWE-400).
- **Análisis de Componentes de Terceros (SCA):** Se utilizó `npm audit` para evaluar la cadena de suministro de software. Se detectaron 14 vulnerabilidades en librerías externas (como `multer`, `axios` y `nodemailer`), las cuales fueron actualizadas a versiones seguras para prevenir ataques de denegación de servicio e inyecciones.
- **Análisis Dinámico (DAST):** A través de OWASP ZAP, se simuló el comportamiento de un atacante sobre el sistema en ejecución. Esto reveló vulnerabilidades como riesgos de ruta transversal (Path Traversal) y la ausencia de cabeceras de seguridad, procediendo a su inmediata remediación.
- **Hardening y Defensa en Profundidad:** Se endureció la infraestructura implementando cabeceras de seguridad HTTP (mediante `helmet`), aislamiento de credenciales y un estricto Control de Acceso Basado

en Roles (RBAC). La base de datos fue reestructurada bajo el modelo de "Confianza Cero" (Zero Trust), segregando las tablas operativas y otorgando privilegios mínimos a alumnos, administradores y dispositivos IoT.

Finalmente, la auditoría validó el cumplimiento del proyecto con la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP), asegurando la correcta manipulación y resguardo de la información institucional. Como conclusión, las implementaciones aplicadas redujeron significativamente la superficie de ataque, dotando a InterLockerUp de un ecosistema digital resiliente y listo para su operación continua.

SA.1. Dominio Conceptual Aplicado

Static Application Security Testing (SAST)

Definición de SAST

El **Static Application Security Testing (SAST)** es una técnica de evaluación de seguridad que consiste en analizar el código fuente, código compilado o archivos binarios de una aplicación **sin necesidad de ejecutarla**. Su finalidad es identificar vulnerabilidades de seguridad, errores de programación y prácticas de desarrollo inseguras antes de que el software sea desplegado.

El análisis estático examina la estructura interna del programa para determinar si existen instrucciones, funciones o flujos de ejecución que puedan derivar en comportamientos inseguros. Debido a que trabaja directamente sobre el código, permite localizar la línea exacta donde se encuentra la vulnerabilidad, facilitando considerablemente las tareas de corrección por parte del equipo de desarrollo.

Las herramientas SAST utilizan motores especializados capaces de recorrer miles de líneas de código en pocos minutos, aplicando reglas de seguridad previamente definidas, modelos de flujo de datos y algoritmos de análisis sintáctico. Como resultado, generan reportes que clasifican los hallazgos según su severidad, describen el riesgo asociado y, en muchos casos, proponen acciones de remediación.

Otra característica importante es que SAST puede integrarse fácilmente en procesos de Integración Continua y Entrega Continua (CI/CD), permitiendo que cada modificación realizada por los desarrolladores sea analizada automáticamente antes de incorporarse al repositorio principal. De esta forma, la seguridad se convierte en una actividad permanente dentro del proceso de desarrollo y no en una revisión aislada al finalizar el proyecto.

Objetivos del análisis SAST

La implementación de análisis estático dentro de un proyecto de software persigue diversos objetivos relacionados con la mejora continua de la seguridad del sistema. Entre los más relevantes se encuentran:

- Detectar vulnerabilidades durante las primeras etapas del desarrollo.
- Identificar prácticas inseguras de programación.
- Reducir el costo asociado a la corrección de errores de seguridad.

- Proporcionar retroalimentación inmediata a los desarrolladores.
- Facilitar la integración de controles de seguridad dentro del pipeline DevSecOps.
- Cumplir con estándares y buenas prácticas de desarrollo seguro.
- Disminuir la probabilidad de que vulnerabilidades críticas lleguen a producción.

Estos objetivos convierten a SAST en una herramienta fundamental dentro de cualquier estrategia moderna de desarrollo seguro.

Aplicación de SAST en InterLockerUp

En el proyecto **InterLockerUp**, el análisis estático tiene una función estratégica debido a la naturaleza del sistema y a la información que procesa. La plataforma está compuesta por un backend desarrollado en Node.js, aplicaciones frontend para estudiantes y administradores, una base de datos PostgreSQL y un componente IoT basado en ESP32 encargado del control físico de los casilleros inteligentes.

Durante el desarrollo del proyecto se utilizó **SonarQube** como herramienta de análisis estático. Este proceso permitió identificar y corregir diversos problemas relacionados con la seguridad y la calidad del código, entre ellos la exposición de información mediante la cabecera X-Powered-By, el uso de funciones criptográficas inadecuadas (Math.random()), bloques catch vacíos y la ausencia de límites en la carga de archivos mediante multer. Tras aplicar las correcciones correspondientes, el análisis final reportó la eliminación de vulnerabilidades, *bugs*, *code smells* y *security hotspots*, mejorando significativamente la postura de seguridad del backend.

En este contexto, la utilización de SAST permitió fortalecer la seguridad del proyecto antes de su despliegue, reduciendo el riesgo de explotación de vulnerabilidades que podrían afectar tanto la información de los usuarios como el funcionamiento del sistema de control de acceso físico.

Funcionamiento interno del Static Application Security Testing (SAST)

El funcionamiento de una herramienta **Static Application Security Testing (SAST)** se basa en el análisis estructural del código fuente sin necesidad de ejecutar la aplicación. A diferencia de otras metodologías de evaluación que observan el comportamiento del software durante su ejecución, el análisis estático examina directamente los archivos que conforman el proyecto, construyendo una representación lógica del programa para

identificar posibles vulnerabilidades antes de que el sistema entre en operación.

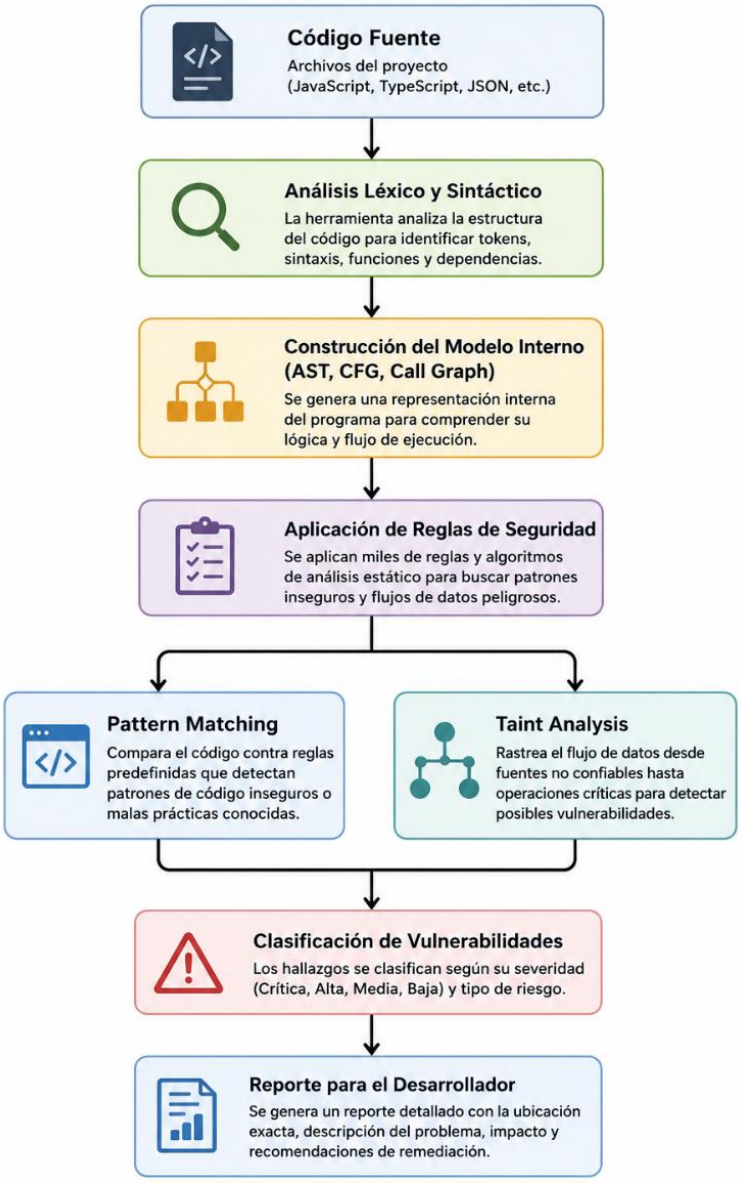
El proceso comienza cuando la herramienta analiza todos los archivos del proyecto, independientemente del lenguaje de programación utilizado. Durante esta etapa se realiza un análisis léxico y sintáctico del código con el propósito de comprender su estructura, identificar funciones, clases, variables, objetos y dependencias entre módulos. Posteriormente, la información obtenida se transforma en una representación interna que permite reconstruir el flujo lógico de ejecución del programa.

Una vez generado este modelo interno, la herramienta aplica diferentes algoritmos especializados para localizar patrones inseguros, seguir el recorrido de los datos dentro de la aplicación y detectar posibles escenarios donde una entrada controlada por un usuario pueda afectar componentes críticos del sistema. Entre las técnicas más utilizadas destacan el **Pattern Matching**, el **Taint Analysis** y el **Control Flow Analysis**, las cuales trabajan de forma complementaria para ofrecer una evaluación integral del código.

El resultado del análisis consiste en un conjunto de hallazgos clasificados por nivel de severidad, indicando la ubicación exacta del problema, una descripción de la vulnerabilidad detectada, el riesgo asociado y, en la mayoría de los casos, recomendaciones para su corrección. Gracias a ello, los desarrolladores pueden solucionar los problemas antes de que la aplicación sea desplegada en un entorno de producción.

La Figura 1.1 muestra de manera conceptual el flujo general seguido por una herramienta SAST durante el proceso de análisis.

Figura 1.1. Flujo general del análisis SAST



Pattern Matching

Una de las técnicas fundamentales utilizadas por las herramientas SAST es el **Pattern Matching** o reconocimiento de patrones. Esta técnica consiste en comparar el código fuente de la aplicación contra un amplio conjunto de reglas previamente definidas que describen construcciones conocidas por representar prácticas inseguras de programación.

Cada regla especifica una secuencia de instrucciones, funciones o estructuras de código que históricamente han estado asociadas con vulnerabilidades de seguridad. Cuando el motor de análisis encuentra una coincidencia entre el código analizado y alguna de estas reglas, genera un hallazgo indicando el posible riesgo identificado.

Por ejemplo, una regla puede detectar el uso de funciones criptográficas consideradas inseguras, contraseñas escritas directamente en el código (*hardcoded secrets*), consultas SQL construidas mediante concatenación de cadenas, desactivación de mecanismos de autenticación o configuraciones que exponen información sensible del servidor.

La principal ventaja del Pattern Matching es su rapidez, ya que permite analizar grandes cantidades de código en poco tiempo con un bajo consumo de recursos. Además, resulta especialmente eficaz para localizar vulnerabilidades ampliamente documentadas y errores frecuentes de programación. Sin embargo, al depender de reglas predefinidas, puede generar falsos positivos cuando identifica una coincidencia sintáctica que, en el contexto específico de la aplicación, no representa realmente una vulnerabilidad.

En el proyecto **InterLockerUp**, esta técnica permitió identificar diversas prácticas que afectaban la seguridad y la calidad del código. Entre ellas se encontraron el uso de funciones criptográficas inadecuadas para la generación de valores aleatorios, la presencia de bloques catch sin tratamiento adecuado de excepciones, la exposición de la cabecera X-Powered-By y configuraciones inseguras relacionadas con la carga de archivos. Estas observaciones fueron detectadas mediante SonarQube y posteriormente corregidas durante el proceso de desarrollo.

Taint Analysis

El **Taint Analysis** constituye una de las técnicas más avanzadas empleadas por las herramientas SAST. Su objetivo es rastrear el recorrido que siguen los datos provenientes de fuentes no confiables hasta los componentes críticos de la aplicación, permitiendo determinar si existe la posibilidad de que información controlada por un atacante alcance operaciones sensibles sin haber sido validada o sanitizada adecuadamente.

Durante este análisis, el motor SAST identifica inicialmente todas aquellas entradas cuyos valores pueden ser manipulados por usuarios externos. Estas

entradas reciben la denominación de **datos contaminados** (*tainted data*). Posteriormente, la herramienta sigue el flujo de dichos datos a través de variables, funciones, objetos y llamadas entre módulos, verificando si en algún punto se aplican mecanismos de validación, filtrado o saneamiento.

Si los datos alcanzan operaciones críticas —como consultas a la base de datos, generación de comandos del sistema operativo, acceso a archivos o construcción de respuestas HTML— sin haber sido correctamente procesados, la herramienta reporta una posible vulnerabilidad.

Esta técnica resulta especialmente útil para detectar ataques como:

- SQL Injection.
- Cross-Site Scripting (XSS).
- Command Injection.
- Path Traversal.
- LDAP Injection.
- XML Injection.

A diferencia del Pattern Matching, el Taint Analysis no se limita a buscar fragmentos específicos de código, sino que analiza el comportamiento lógico del programa, reconstruyendo el flujo completo de los datos dentro de la aplicación. Por esta razón, suele proporcionar resultados más precisos en vulnerabilidades relacionadas con el procesamiento de entradas del usuario.

En **InterLockerUp**, esta técnica resulta especialmente relevante debido a que la plataforma recibe información desde múltiples orígenes, entre ellos formularios web utilizados por estudiantes y administradores, solicitudes HTTP enviadas por dispositivos ESP32, códigos QR dinámicos y peticiones dirigidas a la API REST. Cada uno de estos puntos de entrada representa un posible origen de datos no confiables, por lo que el seguimiento del flujo de información mediante Taint Analysis contribuye a garantizar que las validaciones implementadas impidan la explotación de vulnerabilidades como inyecciones o accesos no autorizados. La arquitectura y el flujo de comunicación del sistema, descritos en la documentación técnica, evidencian la existencia de estas múltiples fuentes de entrada y la necesidad de protegerlas mediante controles de validación adecuados.

Control Flow Analysis (Análisis del Flujo de Control)

Además del **Pattern Matching** y el **Taint Analysis**, las herramientas modernas de **Static Application Security Testing (SAST)** incorporan técnicas de **Control Flow Analysis (CFA)** o **Análisis del Flujo de Control**, cuyo propósito consiste en estudiar las diferentes rutas de ejecución que puede seguir un programa durante su funcionamiento. Mientras que el Pattern Matching identifica patrones inseguros y el Taint Analysis rastrea el recorrido de los datos provenientes de fuentes no confiables, el Control Flow Analysis analiza la lógica de ejecución del software para determinar cómo interactúan sus diferentes componentes y detectar posibles escenarios que puedan derivar en vulnerabilidades.

El análisis comienza con la construcción del **Control Flow Graph (CFG)** o Grafo de Flujo de Control. En este modelo, cada nodo representa un bloque de instrucciones del programa y las aristas indican las posibles transiciones entre dichos bloques durante la ejecución. Gracias a esta representación, la herramienta puede recorrer todas las rutas posibles del software, incluso aquellas que rara vez son ejecutadas durante las pruebas tradicionales.

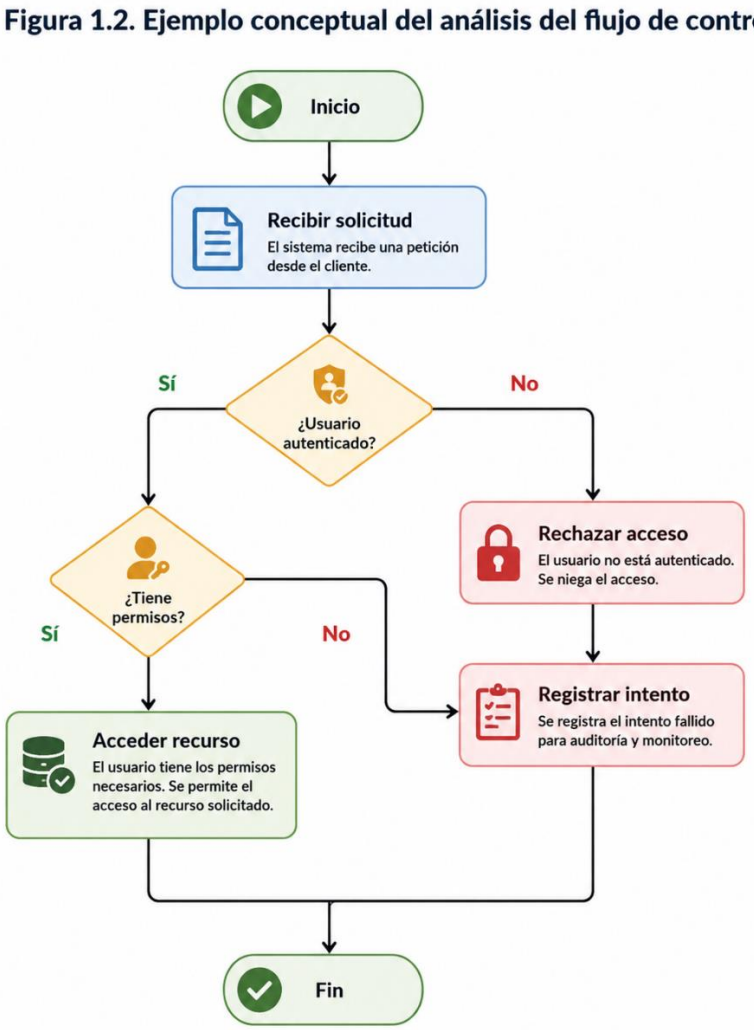
El principal objetivo de esta técnica es identificar comportamientos inseguros derivados de la lógica del programa, tales como rutas de ejecución que omiten validaciones de seguridad, condiciones que permiten el acceso a funciones restringidas, manejo incorrecto de excepciones, ciclos infinitos o estados inconsistentes provocados por determinadas combinaciones de entradas. En otras palabras, el análisis no se limita a revisar líneas individuales de código, sino que estudia cómo interactúan entre sí las diferentes instrucciones y decisiones implementadas por el desarrollador.

Una ventaja importante del Control Flow Analysis es que permite detectar vulnerabilidades que difícilmente serían identificadas mediante simples reglas de coincidencia sintáctica. Por ejemplo, una validación de autenticación puede encontrarse correctamente implementada en una función específica; sin embargo, otra ruta del programa podría acceder al mismo recurso sin ejecutar dicho proceso de validación. Este tipo de inconsistencias suelen descubrirse únicamente cuando se analiza el flujo completo de ejecución.

En el contexto de **InterLockerUp**, esta técnica resulta especialmente relevante debido a que la plataforma implementa múltiples procesos que

dependen del estado del sistema, como la autenticación mediante JSON Web Tokens (JWT), la generación y validación de códigos QR dinámicos, la asignación de lockers, la administración de solicitudes y la interacción con dispositivos ESP32. Todos estos procesos involucran diferentes rutas de ejecución que deben respetar controles de autorización y validación antes de permitir el acceso a recursos críticos. El análisis del flujo de control ayuda a verificar que dichas rutas se ejecuten de forma consistente y que ningún camino alternativo permita omitir las restricciones de seguridad definidas en la aplicación. La arquitectura del sistema y la interacción entre los distintos módulos descritos en la documentación técnica evidencian la necesidad de mantener un flujo de ejecución controlado para garantizar la integridad del proceso de gestión de lockers inteligentes.

Figura 1.2. Ejemplo conceptual del análisis del flujo de control



Relación entre Pattern Matching, Taint Analysis y Control Flow Analysis

Aunque las tres técnicas forman parte del análisis estático, cada una aborda el código desde una perspectiva distinta y complementaria. La combinación de estas metodologías permite a las herramientas SAST ofrecer una evaluación mucho más precisa del estado de seguridad de una aplicación.

Técnica	Objetivo principal	Tipo de análisis	Ejemplo en InterLockerUp
Pattern Matching	Detectar patrones inseguros conocidos	Comparación con reglas predefinidas	Identificación de configuraciones inseguras, uso de funciones criptográficas débiles o malas prácticas de programación detectadas por SonarQube.
Taint Analysis	Rastrear el flujo de datos provenientes del usuario	Flujo de datos	Verificar que los datos recibidos desde formularios web, API REST o dispositivos ESP32 sean validados antes de interactuar con PostgreSQL o generar respuestas al usuario.
Control Flow Analysis	Analizar las rutas de ejecución del programa	Flujo de control	Comprobar que ningún proceso permita omitir la autenticación, la autorización o la validación durante la gestión de lockers inteligentes.

La integración de estas tres técnicas proporciona una visión integral del comportamiento del código fuente. Mientras una identifica patrones inseguros, otra sigue el recorrido de la información y la tercera verifica la lógica de ejecución. En conjunto, permiten descubrir vulnerabilidades que difícilmente podrían identificarse utilizando un único método de análisis.

Ventajas del Static Application Security Testing (SAST)

La incorporación de herramientas de **Static Application Security Testing (SAST)** dentro del ciclo de vida del desarrollo de software proporciona numerosos beneficios para la seguridad, la calidad del código y la eficiencia

del proceso de desarrollo. Debido a que el análisis se realiza directamente sobre el código fuente y no requiere ejecutar la aplicación, los desarrolladores pueden identificar y corregir vulnerabilidades desde las primeras etapas del proyecto, reduciendo considerablemente los costos asociados a su remediación.

Una de las principales ventajas de SAST es su capacidad para detectar vulnerabilidades antes de que la aplicación sea desplegada en un entorno de producción. Este enfoque preventivo permite identificar errores de programación, configuraciones inseguras y malas prácticas de desarrollo cuando todavía forman parte del proceso de construcción del software. Corregir una vulnerabilidad durante la fase de desarrollo suele requerir únicamente modificaciones en el código fuente, mientras que solucionar el mismo problema una vez que la aplicación ha sido liberada puede implicar interrupciones del servicio, actualizaciones de emergencia e incluso incidentes de seguridad que afecten a los usuarios.

Otra ventaja importante consiste en la localización precisa de los problemas detectados. Las herramientas SAST analizan directamente el código fuente y generan reportes indicando el archivo, la función e incluso la línea específica donde se encuentra la posible vulnerabilidad. Esta característica facilita considerablemente el trabajo de los desarrolladores, ya que elimina la necesidad de investigar manualmente el origen del problema y acelera el proceso de corrección.

Asimismo, el análisis estático puede ejecutarse de forma completamente automatizada dentro de un pipeline de **Integración Continua y Entrega Continua (CI/CD)**. Cada vez que un desarrollador realiza cambios en el repositorio, la herramienta puede iniciar automáticamente un nuevo análisis y verificar que las modificaciones introducidas no generen nuevas vulnerabilidades. Esta integración convierte a la seguridad en una actividad continua y no en una revisión aislada realizada al final del proyecto.

Desde el punto de vista de la calidad del software, SAST no solo detecta vulnerabilidades de seguridad, sino también problemas relacionados con mantenibilidad, duplicidad de código, complejidad ciclomática, errores de programación y malas prácticas de desarrollo. De esta manera, contribuye simultáneamente a mejorar la robustez, estabilidad y facilidad de mantenimiento del sistema.

En proyectos con múltiples desarrolladores, como **InterLockerUp**, donde participan distintos módulos correspondientes al backend, frontend, administración y componentes IoT, el análisis estático proporciona un mecanismo uniforme para evaluar la calidad y seguridad del código desarrollado por cada integrante del equipo. Esto favorece la aplicación consistente de estándares de programación y reduce la posibilidad de introducir vulnerabilidades durante la integración del sistema.

La experiencia obtenida durante el desarrollo de InterLockerUp evidencia estas ventajas. El uso de SonarQube permitió identificar vulnerabilidades relacionadas con configuraciones inseguras, manejo incorrecto de excepciones y prácticas criptográficas mejorables antes del despliegue del sistema. Gracias a ello fue posible corregir los problemas detectados y obtener un reporte final sin vulnerabilidades críticas, fortaleciendo significativamente la seguridad del backend.

Tabla 1.1. Principales ventajas del análisis SAST

Ventaja	Descripción	Beneficio para InterLockerUp
Detección temprana	Identifica vulnerabilidades antes de ejecutar la aplicación.	Reduce el riesgo de desplegar un backend con fallos de seguridad.
Corrección rápida	Localiza la línea exacta donde se encuentra el problema.	Disminuye el tiempo requerido para corregir errores durante el desarrollo.
Integración con CI/CD	Puede ejecutarse automáticamente en cada actualización del repositorio.	Favorece un proceso de desarrollo seguro y continuo.
Mejora de la calidad	Detecta problemas de mantenibilidad y malas prácticas.	Incrementa la estabilidad y mantenibilidad del código del proyecto.

Automatización	Analiza miles de líneas de código en pocos minutos.	Permite revisar de forma periódica todos los módulos del sistema.
Cumplimiento de buenas prácticas	Facilita el cumplimiento de estándares de desarrollo seguro.	Contribuye a mantener un nivel homogéneo de calidad en el equipo de desarrollo.

Aplicación práctica en InterLockerUp

En el caso de **InterLockerUp**, la utilización de análisis estático representa una ventaja significativa debido a la naturaleza crítica del sistema. La plataforma administra información personal de estudiantes, controla la asignación de casilleros inteligentes y coordina la comunicación con dispositivos ESP32 encargados del acceso físico. Un error de programación en cualquiera de estos componentes podría traducirse en accesos no autorizados, pérdida de información o interrupciones del servicio.

La implementación de SonarQube permitió revisar automáticamente el código del backend antes del despliegue, detectando oportunidades de mejora relacionadas con la seguridad y la calidad del software. Este proceso favoreció la corrección temprana de los problemas encontrados y redujo la probabilidad de que vulnerabilidades llegaran al entorno de producción, alineándose con los principios de **Shift Left Security** y **DevSecOps**, los cuales promueven integrar controles de seguridad desde las primeras fases del desarrollo.

Desventajas y limitaciones del Static Application Security Testing (SAST)

A pesar de las múltiples ventajas que ofrece el **Static Application Security Testing (SAST)**, esta metodología no constituye una solución completa para garantizar la seguridad de una aplicación. Como toda técnica de análisis, presenta limitaciones inherentes a su funcionamiento que deben ser comprendidas para utilizarla de manera adecuada dentro de una estrategia integral de desarrollo seguro. Conocer estas limitaciones permite interpretar correctamente los resultados obtenidos y comprender por qué las organizaciones modernas combinan SAST con otras metodologías de evaluación como DAST, IAST y SCA.

Una de las principales limitaciones del análisis estático es que **no ejecuta la aplicación**. Debido a ello, la herramienta únicamente puede inferir el comportamiento del software a partir de la estructura del código, sin observar cómo interactúa realmente con el sistema operativo, la red, la base de datos o los usuarios finales. En consecuencia, existen vulnerabilidades que solo se manifiestan durante la ejecución del programa y que, por su naturaleza dinámica, no pueden ser detectadas mediante análisis estático.

Otra limitación importante corresponde a la generación de **falsos positivos**. Al basarse en reglas, patrones y modelos de análisis, una herramienta SAST puede identificar determinadas construcciones del código como potencialmente inseguras aun cuando, dentro del contexto específico de la aplicación, no representen una vulnerabilidad real. Esto obliga a que los resultados sean revisados por desarrolladores o especialistas en seguridad capaces de diferenciar entre riesgos reales y hallazgos que no requieren corrección.

Asimismo, también pueden presentarse **falsos negativos**, es decir, vulnerabilidades que no son detectadas por la herramienta. Esto puede ocurrir cuando la vulnerabilidad depende del comportamiento en tiempo de ejecución, de configuraciones específicas del entorno o de interacciones complejas entre distintos componentes del sistema. Aunque los motores modernos de análisis estático incorporan algoritmos cada vez más sofisticados, ningún análisis puede garantizar la detección del cien por ciento de las vulnerabilidades existentes.

Otra desventaja está relacionada con el tiempo de análisis en proyectos de gran tamaño. A medida que aumenta la cantidad de archivos, módulos y dependencias, también se incrementa el tiempo necesario para recorrer completamente el código fuente. En aplicaciones empresariales con cientos de miles de líneas de código, un análisis exhaustivo puede requerir varios minutos o incluso horas, dependiendo de la complejidad del proyecto y de la configuración de la herramienta.

El análisis estático también depende en gran medida de la calidad y actualización de sus reglas de seguridad. Las amenazas evolucionan constantemente y aparecen nuevas vulnerabilidades, bibliotecas y técnicas de ataque. Si las reglas del motor SAST no se mantienen actualizadas, existe el riesgo de que no detecte vulnerabilidades recientemente descubiertas o

patrones de programación inseguros que aún no formen parte de su base de conocimiento.

Finalmente, SAST no evalúa aspectos relacionados con la configuración del entorno de ejecución. Por ejemplo, una aplicación puede presentar un código completamente seguro y, sin embargo, ejecutarse sobre un servidor mal configurado, utilizar certificados inválidos, exponer servicios innecesarios o implementar políticas de autenticación inadecuadas. Este tipo de problemas únicamente pueden identificarse mediante técnicas complementarias como el análisis dinámico o las auditorías de configuración.

Tabla 1.2. Principales limitaciones del análisis SAST

Limitación	Descripción	Posible impacto
No ejecuta la aplicación	Analiza únicamente el código fuente.	No detecta vulnerabilidades que dependen del entorno de ejecución.
Falsos positivos	Puede reportar problemas que realmente no representan un riesgo.	Incrementa el tiempo necesario para revisar los resultados.
Falsos negativos	Algunas vulnerabilidades pueden pasar desapercibidas.	Genera una falsa sensación de seguridad si no se complementa con otras técnicas.
Dependencia de reglas	Requiere mantener actualizadas las reglas del motor de análisis.	Puede omitir vulnerabilidades recientes o patrones emergentes.
Tiempo de análisis	En proyectos muy grandes el proceso puede ser prolongado.	Aumenta el tiempo de ejecución dentro del pipeline CI/CD.
No analiza la infraestructura	No revisa configuraciones del	Es necesario complementarlo con DAST y revisiones de infraestructura.

	servidor, red o sistema operativo.	

Limitaciones observadas en InterLockerUp

Durante el desarrollo de **InterLockerUp**, SonarQube permitió detectar diversos problemas de seguridad y calidad en el código del backend; sin embargo, el análisis estático por sí solo no podía validar aspectos como el comportamiento de la aplicación ante ataques reales, la correcta configuración del servidor web o la interacción entre los diferentes componentes desplegados. Por ejemplo, aunque el código implementara adecuadamente la autenticación mediante JWT o la validación de solicitudes, únicamente una prueba dinámica permitiría comprobar si dichas medidas resisten intentos de explotación cuando el sistema se encuentra en ejecución.

Del mismo modo, el análisis estático no evalúa la comunicación entre el backend y los dispositivos ESP32, ni verifica el comportamiento de la API REST frente a solicitudes maliciosas generadas por un atacante. Estas situaciones requieren la utilización de herramientas de análisis dinámico, como **OWASP ZAP**, que interactúan directamente con la aplicación desplegada para identificar vulnerabilidades explotables en tiempo de ejecución. La arquitectura distribuida de InterLockerUp, descrita en la documentación técnica, evidencia la necesidad de complementar el análisis del código con pruebas sobre el sistema en funcionamiento.

¿Por qué SAST no es suficiente?

La seguridad de una aplicación depende de múltiples factores: la calidad del código, las bibliotecas utilizadas, la configuración del entorno, la infraestructura de despliegue y el comportamiento del sistema durante su ejecución. Debido a ello, ninguna herramienta puede evaluar por sí sola todos estos aspectos.

SAST constituye una primera línea de defensa orientada a detectar vulnerabilidades directamente en el código fuente. Sin embargo, para obtener una evaluación integral resulta necesario combinarlo con otras metodologías:

- **DAST (Dynamic Application Security Testing):** analiza la aplicación en ejecución e identifica vulnerabilidades explotables desde el exterior.

- **IAST (Interactive Application Security Testing):** combina el análisis estático y dinámico mediante instrumentación de la aplicación durante las pruebas.
- **SCA (Software Composition Analysis):** revisa las dependencias y bibliotecas de terceros para detectar componentes con vulnerabilidades conocidas.

La integración de estas técnicas permite cubrir diferentes superficies de ataque y construir una estrategia de seguridad más completa durante todo el ciclo de vida del desarrollo.

Comparación entre SAST, DAST, IAST y SCA

¿Qué analiza cada metodología?

Cada una de las metodologías de evaluación se enfoca en una superficie de ataque distinta. Mientras algunas inspeccionan directamente el código fuente, otras observan el comportamiento de la aplicación durante su ejecución o revisan los componentes de terceros utilizados por el proyecto.

Esta diversidad de enfoques permite detectar vulnerabilidades que serían invisibles para otras herramientas. Por ello, la implementación conjunta de estas metodologías constituye una práctica recomendada dentro de los modelos de desarrollo seguro y de las estrategias DevSecOps.

Tabla 1.3. Comparación entre SAST, DAST, IAST y SCA

Característica	SAST	DAST	IAST	SCA
Objeto de análisis	Código fuente, binarios o código compilado.	Aplicación en ejecución.	Aplicación en ejecución instrumentada.	Bibliotecas y dependencias de terceros.
Momento de ejecución	Durante el desarrollo.	Después del despliegue o en ambientes de pruebas.	Durante las pruebas funcionales.	Desde el desarrollo y durante todo el ciclo de vida.
Requiere ejecutar la aplicación	No.	Sí.	Sí.	No necesariamente.
Detecta vulnerabilidades en el código propio	Sí.	Parcialmente.	Sí.	No.
Detecta vulnerabilidades en dependencias externas	No.	No.	Parcialmente.	Sí.
Detecta errores de configuración del sistema	No.	Sí.	Sí.	No.

Ubica la línea exacta del problema	Sí.	Generalmente no.	Sí.	No aplica.
Evalúa el comportamiento real del sistema	No.	Sí.	Sí.	No.
Principales herramientas	SonarQube, Semgrep, Fortify.	OWASP ZAP, Burp Suite, Acunetix.	Contrast Security, Seeker.	OWASP Dependency-Check, Snyk, Dependabot.

¿Cuándo debe utilizarse cada una?

La elección de la metodología depende directamente de la etapa del ciclo de vida del desarrollo en la que se encuentre el proyecto. Implementarlas en el momento adecuado permite detectar vulnerabilidades con mayor eficiencia y reducir el costo asociado a su corrección.

SAST: Debe ejecutarse durante el desarrollo del software, preferentemente antes de integrar nuevos cambios al repositorio principal. Su objetivo consiste en identificar vulnerabilidades directamente en el código fuente antes de que la aplicación sea compilada o desplegada.

En **InterLockerUp**, esta metodología fue utilizada mediante SonarQube para analizar el backend desarrollado en Node.js, permitiendo corregir vulnerabilidades antes de la publicación del sistema.

DAST: Se utiliza cuando la aplicación ya puede ejecutarse.

La herramienta interactúa con el sistema como si fuera un atacante externo, enviando múltiples solicitudes HTTP para descubrir vulnerabilidades explotables.

En InterLockerUp, este análisis se realizará utilizando **OWASP ZAP**, evaluando la API REST y las interfaces web una vez desplegadas.

IAST: Se ejecuta durante las pruebas funcionales.

La aplicación incorpora un agente de monitoreo que observa el comportamiento interno del software mientras los usuarios realizan pruebas normales.

Esto permite combinar la información obtenida mediante análisis estático y dinámico.

SCA: Debe ejecutarse desde las primeras etapas del desarrollo y mantenerse durante toda la vida útil del proyecto.

Su función consiste en revisar continuamente las bibliotecas de terceros utilizadas por la aplicación para identificar componentes que presenten vulnerabilidades conocidas (CVE), versiones obsoletas o dependencias sin mantenimiento.

En un proyecto como InterLockerUp, donde el backend en Node.js utiliza paquetes instalados mediante **npm** y el frontend desarrollado con React incorpora múltiples dependencias, el análisis SCA resulta indispensable para garantizar que dichos componentes no introduzcan riesgos de seguridad adicionales.

¿Por qué son complementarios?

Uno de los errores más comunes consiste en pensar que una única herramienta puede evaluar completamente la seguridad de una aplicación. En realidad, cada metodología observa únicamente una parte del sistema y, por lo tanto, presenta limitaciones que son compensadas por las demás.

Por ejemplo, SAST puede detectar una validación incorrecta implementada en el código fuente, pero no puede determinar si dicha vulnerabilidad realmente es explotable cuando la aplicación se encuentra en ejecución. Esa tarea corresponde al análisis dinámico realizado mediante DAST.

De forma similar, DAST puede descubrir una vulnerabilidad explotable, pero generalmente no puede indicar la línea específica del código donde se originó el problema. En este caso, el análisis estático facilita el proceso de remediación al localizar el punto exacto donde debe realizarse la corrección.

IAST combina ambos enfoques al observar simultáneamente el comportamiento interno del software y las solicitudes que recibe durante las pruebas funcionales, proporcionando información más precisa sobre la explotación real de las vulnerabilidades.

Finalmente, SCA complementa las metodologías anteriores al analizar un aspecto que ninguna de ellas cubre completamente: las bibliotecas y

componentes externos utilizados por la aplicación. Dado que una gran parte de las vulnerabilidades modernas provienen de dependencias de terceros, este análisis resulta indispensable para mantener actualizado el ecosistema de software del proyecto.

En conjunto, estas cuatro metodologías ofrecen una cobertura integral de la seguridad del software, permitiendo identificar vulnerabilidades tanto en el código desarrollado por el equipo como en la infraestructura de ejecución y en los componentes externos utilizados por la aplicación.

Relación con InterLockerUp

El desarrollo de **InterLockerUp** constituye un ejemplo claro de la necesidad de combinar múltiples metodologías de evaluación de seguridad. El proyecto integra un backend desarrollado en Node.js, un frontend basado en React, una base de datos PostgreSQL y dispositivos ESP32 encargados del control físico de los lockers inteligentes. Esta arquitectura implica diferentes superficies de ataque que no pueden evaluarse mediante una única herramienta. La documentación técnica del proyecto describe precisamente esta integración de componentes, lo que justifica la aplicación de distintos tipos de análisis según la naturaleza de cada uno.

En este contexto:

- **SAST** permite revisar el código fuente del backend y detectar vulnerabilidades antes del despliegue.
- **DAST** evalúa la aplicación web y la API REST una vez que el sistema está en funcionamiento.
- **IAST** puede emplearse durante las pruebas funcionales para observar el comportamiento interno de la aplicación.
- **SCA** verifica que las dependencias de Node.js y React no contengan vulnerabilidades conocidas ni versiones obsoletas.

La implementación conjunta de estas metodologías fortalece significativamente la postura de seguridad del proyecto y se alinea con las prácticas recomendadas de **DevSecOps**, donde la seguridad se integra de forma continua a lo largo de todo el ciclo de vida del software.

Dynamic Application Security Testing (DAST)

Definición de DAST

El **Dynamic Application Security Testing (DAST)** es una metodología de análisis de seguridad que consiste en evaluar una aplicación mientras se encuentra en ejecución, interactuando con ella desde el exterior de manera similar a como lo haría un usuario legítimo o un posible atacante. Su objetivo principal es identificar vulnerabilidades explotables que solo pueden manifestarse durante la operación real del sistema.

A diferencia del análisis estático, DAST trata a la aplicación como una **caja negra (Black Box Testing)**. Esto significa que la herramienta no necesita conocer la implementación interna del software ni acceder al código fuente para realizar la evaluación. En su lugar, envía solicitudes al servidor, analiza las respuestas recibidas e identifica comportamientos que puedan indicar la existencia de vulnerabilidades.

Las herramientas DAST automatizan este proceso mediante la exploración de la aplicación, el descubrimiento de recursos accesibles, el envío de solicitudes especialmente diseñadas para provocar respuestas anómalas y el análisis de dichas respuestas para determinar si existe una vulnerabilidad explotable. Como resultado, generan reportes que describen los riesgos encontrados, su nivel de severidad y recomendaciones para su mitigación.

Una de las principales fortalezas del análisis dinámico es que permite detectar problemas relacionados con la configuración del servidor, errores en la autenticación, exposición de información sensible, vulnerabilidades de sesión, inyecciones, configuraciones HTTP inseguras y múltiples fallos que únicamente pueden observarse cuando la aplicación se encuentra funcionando en un entorno real.

Objetivos del análisis DAST

El propósito del análisis dinámico no consiste únicamente en encontrar vulnerabilidades, sino en evaluar la seguridad del sistema desde la perspectiva de un atacante que interactúa con la aplicación ya desplegada. Entre sus principales objetivos destacan los siguientes:

- Identificar vulnerabilidades explotables durante la ejecución del sistema.
- Evaluar la configuración de seguridad del servidor y de la aplicación.
- Verificar la correcta implementación de mecanismos de autenticación y autorización.
- Detectar errores relacionados con sesiones, cookies y encabezados HTTP.
- Comprobar la resistencia del sistema frente a ataques comunes contra aplicaciones web.
- Validar que las medidas de seguridad implementadas durante el desarrollo funcionen correctamente en producción.
- Complementar los resultados obtenidos mediante análisis estático.

El cumplimiento de estos objetivos permite obtener una visión mucho más realista del nivel de seguridad de una aplicación antes de su puesta en producción.

Aplicación de DAST en InterLockerUp

En **InterLockerUp**, el análisis dinámico constituye una etapa esencial para verificar que las medidas de seguridad implementadas durante el desarrollo continúen siendo efectivas cuando la plataforma opera en condiciones reales. Debido a que el sistema expone múltiples servicios web y una API REST utilizada por dispositivos ESP32 para la gestión de lockers inteligentes, resulta indispensable comprobar que dichos servicios resistan intentos de explotación provenientes de usuarios no autorizados.

Durante esta fase de evaluación, una herramienta como **OWASP ZAP** puede interactuar con la aplicación desplegada enviando solicitudes HTTP hacia las diferentes rutas disponibles, incluyendo procesos de autenticación, administración de lockers, consulta de asignaciones, generación de códigos QR y comunicación con la API utilizada por los dispositivos IoT. A partir de las respuestas obtenidas, la herramienta identifica configuraciones inseguras, encabezados HTTP incorrectos, validaciones insuficientes, errores de

autenticación y otras vulnerabilidades que podrían comprometer la seguridad del sistema.

A diferencia del análisis estático realizado con SonarQube, donde se inspeccionó directamente el código fuente, el análisis dinámico permite comprobar si las correcciones implementadas realmente protegen la aplicación cuando esta se encuentra funcionando. De esta manera, ambas metodologías se complementan para ofrecer una evaluación mucho más completa de la postura de seguridad de InterLockerUp.

Funcionamiento interno del Dynamic Application Security Testing (DAST)

A diferencia del análisis estático, donde el código fuente constituye el principal objeto de evaluación, el **Dynamic Application Security Testing (DAST)** analiza el comportamiento observable de una aplicación mientras esta se encuentra en ejecución. Para lograrlo, las herramientas DAST interactúan directamente con el sistema mediante solicitudes HTTP, simulando el comportamiento de usuarios legítimos o de posibles atacantes con el propósito de descubrir vulnerabilidades explotables.

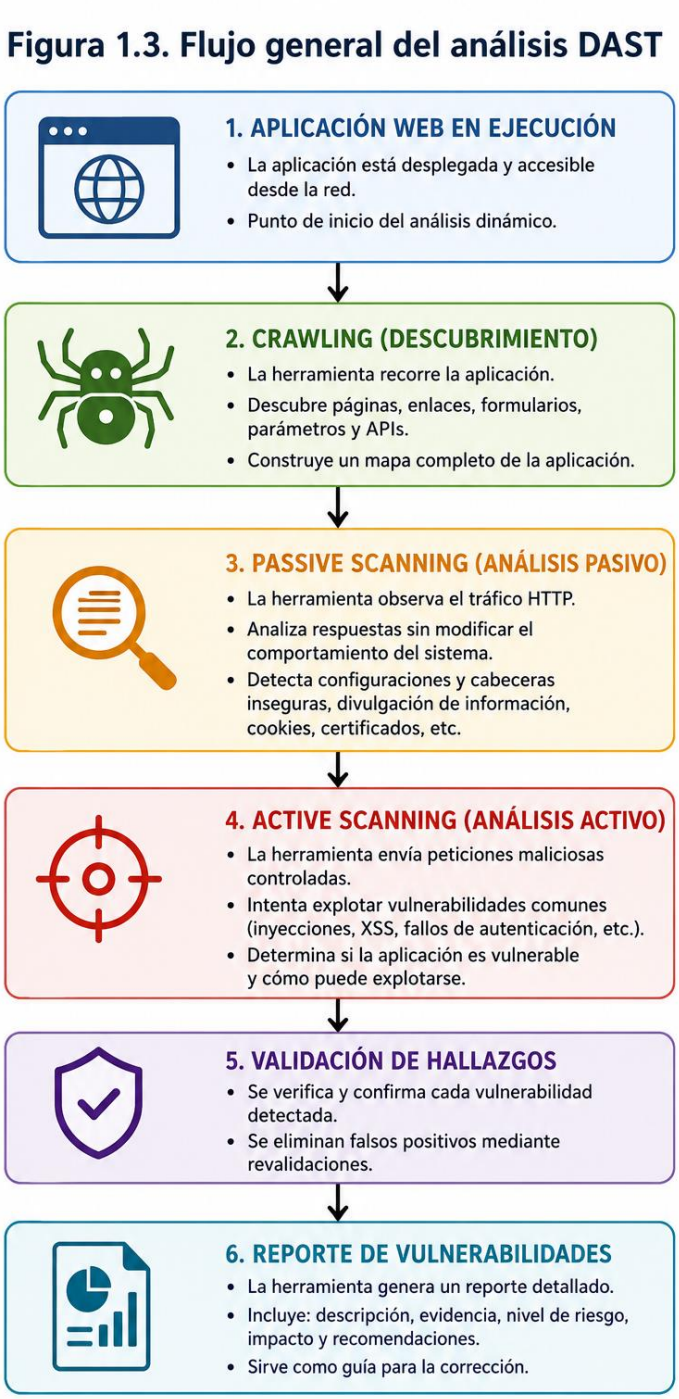
El proceso de análisis dinámico se desarrolla mediante una serie de etapas consecutivas que permiten descubrir la estructura de la aplicación, identificar los recursos disponibles, analizar la configuración de seguridad y ejecutar pruebas activas contra los diferentes componentes del sistema. Cada una de estas etapas aporta información específica que posteriormente es utilizada para construir un reporte detallado de las vulnerabilidades encontradas.

En términos generales, el funcionamiento de una herramienta DAST puede dividirse en cuatro fases principales:

1. **Descubrimiento de la aplicación (Crawling).**
2. **Análisis pasivo (Passive Scanning).**
3. **Análisis activo (Active Scanning).**
4. **Generación del reporte de vulnerabilidades.**

Estas fases trabajan de forma secuencial y permiten evaluar el comportamiento real de la aplicación desde una perspectiva externa, sin necesidad de acceder a su implementación interna.

Figura 1.3. Flujo general del análisis DAST



Crawling (Descubrimiento de Recursos)

La primera etapa del análisis dinámico corresponde al proceso de **Crawling**, también conocido como exploración o descubrimiento de recursos. Durante esta fase, la herramienta DAST recorre automáticamente la aplicación con el objetivo de identificar todas las páginas, formularios, servicios web, parámetros, enlaces, directorios y recursos disponibles para los usuarios.

El funcionamiento del Crawling es similar al de un motor de búsqueda en Internet. La herramienta comienza desde una URL inicial y sigue cada enlace disponible, construyendo progresivamente un mapa completo de la aplicación. Conforme descubre nuevos recursos, continúa explorándolos hasta identificar todas las rutas accesibles.

Durante esta etapa también se detectan elementos como:

- Formularios de autenticación.
- Parámetros enviados mediante GET y POST.
- Cookies.
- Tokens de sesión.
- APIs REST.
- Archivos descargables.
- Recursos estáticos.
- Encabezados HTTP.

El resultado del proceso es una representación estructurada de la superficie de ataque de la aplicación, la cual será utilizada posteriormente durante las fases de análisis pasivo y activo.

En **InterLockerUp**, esta etapa permitiría descubrir automáticamente las diferentes interfaces disponibles para estudiantes y administradores, así como las rutas de la API REST utilizadas por los dispositivos ESP32 para validar códigos QR y gestionar la apertura de los lockers inteligentes. La documentación técnica describe precisamente esta interacción entre clientes web, servicios backend y componentes IoT, lo que evidencia la importancia de identificar correctamente todos los puntos de acceso antes de iniciar las pruebas de seguridad.

Passive Scanning (Análisis Pasivo)

Una vez identificada la estructura general de la aplicación, la herramienta inicia el proceso denominado **Passive Scanning** o análisis pasivo.

Durante esta fase **no se modifica el comportamiento del sistema ni se intenta explotar ninguna vulnerabilidad**. La herramienta simplemente observa todas las respuestas HTTP generadas por la aplicación mientras continúa navegando por sus diferentes recursos.

El objetivo consiste en identificar configuraciones inseguras que puedan detectarse únicamente mediante la inspección del tráfico intercambiado entre el cliente y el servidor.

Entre los aspectos evaluados durante el análisis pasivo destacan:

- Encabezados HTTP inseguros.
- Cookies sin atributos Secure o HttpOnly.
- Versiones expuestas del servidor.
- Divulgación de información sensible.
- Certificados incorrectos.
- Políticas CSP inexistentes.
- Configuración de CORS.
- Métodos HTTP habilitados.

Debido a que no realiza ataques directos contra la aplicación, esta etapa presenta un riesgo prácticamente nulo para el funcionamiento del sistema y puede ejecutarse incluso sobre entornos productivos cuando se toman las precauciones adecuadas.

En el caso de **InterLockerUp**, el análisis pasivo permitiría comprobar, por ejemplo, que las respuestas HTTP generadas por el backend incluyan los encabezados de seguridad adecuados, que las cookies de autenticación mediante JWT se gestionen correctamente y que el servidor no revele información innecesaria sobre la infraestructura utilizada.

Active Scanning (Análisis Activo)

Una vez finalizadas las etapas de descubrimiento de recursos y análisis pasivo, la herramienta DAST inicia el proceso conocido como **Active Scanning** o análisis activo. Esta fase representa el núcleo del análisis dinámico, ya que consiste en enviar solicitudes especialmente diseñadas para comprobar si la aplicación presenta vulnerabilidades explotables.

A diferencia del análisis pasivo, donde la herramienta únicamente observa las respuestas generadas por el sistema sin modificar su comportamiento, el análisis activo interactúa deliberadamente con la aplicación utilizando entradas maliciosas o inesperadas. El objetivo consiste en provocar respuestas que permitan determinar si existen fallos de seguridad susceptibles de ser aprovechados por un atacante.

Durante esta etapa, la herramienta automatiza cientos o incluso miles de pruebas contra los diferentes recursos identificados durante el proceso de Crawling. Cada solicitud enviada contiene valores cuidadosamente seleccionados para verificar la existencia de vulnerabilidades específicas. Posteriormente, la herramienta analiza las respuestas del servidor buscando comportamientos anómalos que indiquen la presencia de una debilidad de seguridad.

El análisis activo constituye una simulación controlada de múltiples técnicas de ataque ampliamente documentadas dentro del ámbito de la seguridad informática. Debido a ello, suele ejecutarse únicamente sobre ambientes de desarrollo, pruebas o preproducción, ya que algunas pruebas podrían afectar temporalmente el funcionamiento normal de la aplicación.

Objetivos del Active Scanning

El propósito principal del análisis activo consiste en comprobar si una vulnerabilidad puede explotarse realmente dentro del entorno donde se ejecuta la aplicación. Para ello, la herramienta intenta interactuar con los diferentes componentes del sistema reproduciendo el comportamiento que seguiría un atacante durante un proceso de reconocimiento y explotación.

Entre los objetivos más importantes del análisis activo destacan:

- Detectar vulnerabilidades explotables durante la ejecución del sistema.
- Validar mecanismos de autenticación y autorización.
- Comprobar la correcta validación de entradas proporcionadas por los usuarios.
- Evaluar la resistencia de la aplicación frente a ataques automatizados.
- Identificar errores de configuración del servidor.
- Detectar exposición de información sensible.
- Confirmar la existencia real de vulnerabilidades reportadas previamente.

Gracias a estos objetivos, el análisis activo proporciona evidencia mucho más cercana al comportamiento observado durante una auditoría de penetración.

Principales vulnerabilidades detectadas mediante Active Scanning

Las herramientas DAST incorporan un amplio conjunto de pruebas diseñadas para identificar las vulnerabilidades más frecuentes presentes en aplicaciones web modernas. Algunas de las más relevantes son las siguientes:

SQL Injection (SQLi): Consiste en enviar consultas manipuladas con el propósito de alterar el comportamiento normal de la base de datos.

Si la aplicación no valida correctamente las entradas del usuario, el atacante podría consultar, modificar o eliminar información almacenada en el sistema.

En **InterLockerUp**, esta vulnerabilidad podría comprometer información relacionada con usuarios, asignaciones de lockers, códigos QR temporales o registros de acceso si existieran consultas construidas de forma insegura.

Cross-Site Scripting (XSS): El análisis activo intenta insertar código JavaScript en formularios, parámetros URL o campos de entrada para comprobar si dicho código es posteriormente ejecutado dentro del navegador de otros usuarios.

La explotación de esta vulnerabilidad puede permitir el robo de cookies de sesión, credenciales o información sensible.

Broken Authentication: La herramienta analiza el comportamiento de los procesos de autenticación intentando detectar:

sesiones predecibles;

- expiración incorrecta de tokens;
- reutilización de sesiones;
- gestión insegura de cookies;
- bypass de autenticación.

En un sistema como **InterLockerUp**, donde la autenticación mediante JWT protege tanto el acceso de estudiantes como de administradores, este tipo de pruebas resulta especialmente importante.

Broken Access Control: La herramienta intenta acceder a recursos para los cuales el usuario no posee autorización.

Por ejemplo:

- acceder a funciones administrativas utilizando una cuenta de estudiante;
- consultar lockers pertenecientes a otros usuarios;
- modificar solicitudes ajenas.

Este tipo de pruebas permite verificar que la autorización implementada por el backend realmente impide accesos indebidos.

Security Misconfiguration: Durante el análisis activo también se evalúan aspectos relacionados con la configuración del servidor, entre ellos:

- directorios expuestos;
- páginas administrativas accesibles;
- versiones del servidor;
- configuración TLS;
- encabezados HTTP.

Muchas de estas vulnerabilidades no dependen directamente del código fuente, sino del entorno donde la aplicación se encuentra desplegada.

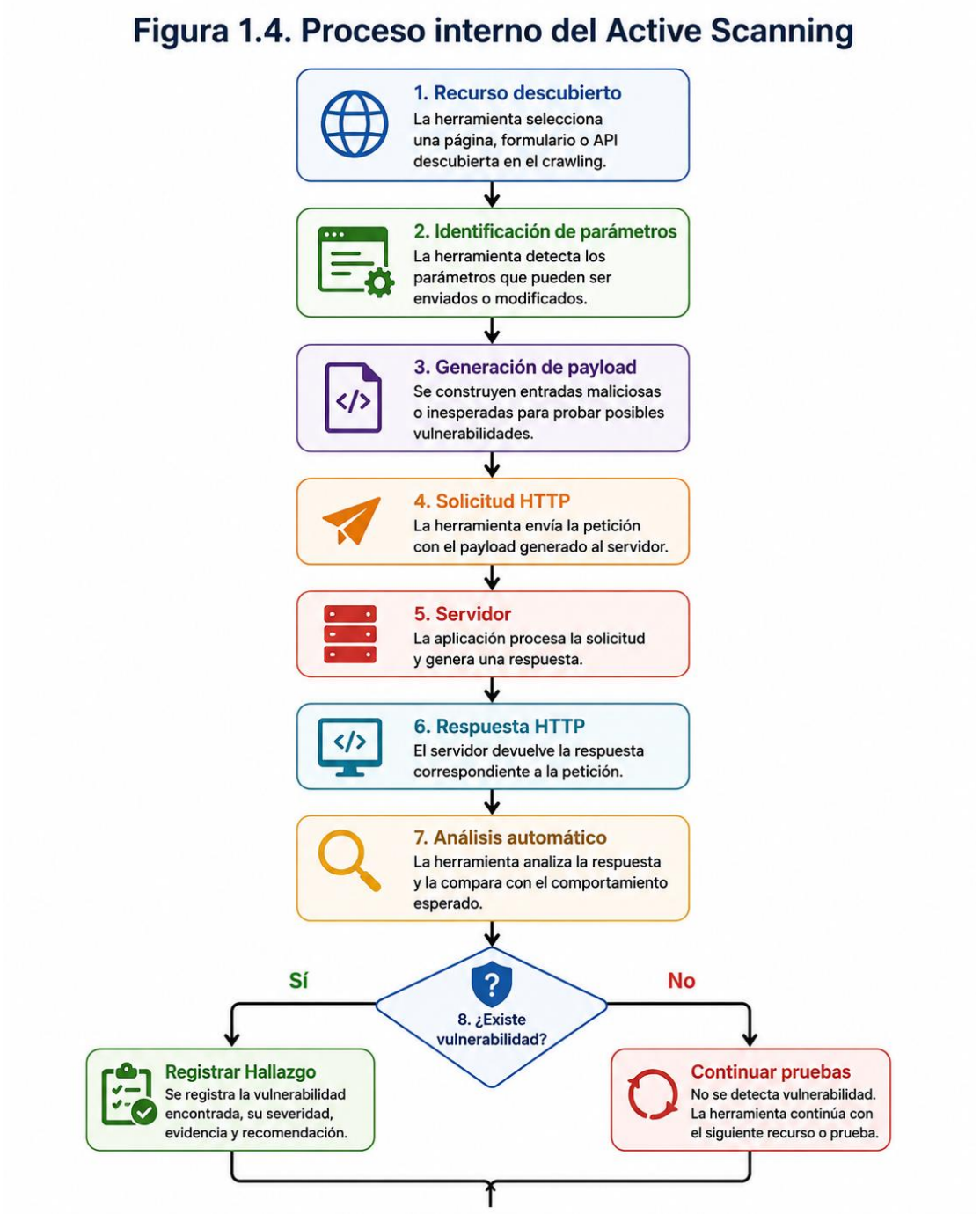
¿Cómo funciona internamente el Active Scanning?

El proceso interno seguido por una herramienta DAST puede resumirse mediante las siguientes etapas:

1. Seleccionar un recurso descubierto durante el Crawling.
2. Identificar los parámetros que pueden modificarse.
3. Elegir una prueba de seguridad específica.
4. Construir una solicitud maliciosa.
5. Enviar la petición al servidor.
6. Analizar la respuesta obtenida.
7. Comparar el resultado con el comportamiento esperado.
8. Registrar el hallazgo si se identifica una vulnerabilidad.

Este procedimiento se repite automáticamente cientos o miles de veces para cada uno de los recursos descubiertos durante la exploración inicial.

Figura 1.4. Proceso interno del Active Scanning



Aplicación en InterLockerUp

Durante la auditoría de seguridad de **InterLockerUp**, el análisis activo se realizará utilizando **OWASP ZAP** sobre la aplicación desplegada. La herramienta enviará solicitudes controladas hacia el backend desarrollado en Node.js, evaluando tanto las interfaces web como la API REST utilizada por los dispositivos ESP32.

Las pruebas estarán orientadas a verificar la correcta implementación de controles relacionados con autenticación, autorización, validación de entradas, encabezados HTTP y manejo de sesiones. Asimismo, se comprobará que las vulnerabilidades corregidas previamente mediante el análisis estático con SonarQube no sean explotables cuando el sistema se encuentre en funcionamiento.

Este enfoque permitirá validar que las medidas de seguridad implementadas durante el desarrollo se mantienen efectivas en un entorno operativo, proporcionando una evaluación mucho más cercana a las condiciones reales de uso del sistema.

Fuzzing (Pruebas mediante entradas aleatorias)

El **Fuzzing**, también conocido como **Fuzz Testing**, constituye una de las técnicas más utilizadas dentro del análisis dinámico para identificar vulnerabilidades relacionadas con el procesamiento incorrecto de datos de entrada. Su funcionamiento consiste en enviar de forma automática una gran cantidad de valores inesperados, aleatorios o especialmente diseñados hacia los diferentes componentes de una aplicación con el propósito de observar su comportamiento ante situaciones no previstas por los desarrolladores.

A diferencia de otras técnicas de evaluación que utilizan únicamente datos válidos, el Fuzzing intenta romper las suposiciones realizadas durante el desarrollo del software. Para ello genera entradas con formatos incorrectos, tamaños excesivos, caracteres especiales, valores nulos, secuencias repetitivas o combinaciones poco comunes que podrían provocar errores de ejecución, fallos de validación o vulnerabilidades explotables.

El principio fundamental del Fuzzing es que un sistema robusto debe ser capaz de procesar correctamente cualquier entrada recibida, incluso cuando dicha información sea inválida o maliciosa. Si durante estas pruebas la aplicación genera excepciones inesperadas, mensajes de error inusuales,

bloqueos o comportamientos inconsistentes, la herramienta registra estos eventos como posibles indicadores de vulnerabilidades.

Debido a que el proceso puede ejecutarse automáticamente sobre miles de parámetros en pocos minutos, el Fuzzing representa uno de los mecanismos más eficaces para descubrir fallos que difícilmente serían detectados mediante pruebas manuales.

Objetivos del Fuzzing

La finalidad principal del Fuzzing consiste en evaluar la capacidad de una aplicación para manejar entradas inesperadas sin comprometer su funcionamiento ni su seguridad.

Entre sus objetivos más importantes destacan:

- Detectar errores de validación de datos.
- Identificar vulnerabilidades de inyección.
- Descubrir desbordamientos de memoria (en aplicaciones nativas).
- Evaluar la estabilidad del sistema.
- Verificar el manejo adecuado de excepciones.
- Detectar bloqueos inesperados.
- Identificar condiciones de error que puedan convertirse en vulnerabilidades explotables.

En aplicaciones web modernas, estas pruebas permiten comprobar que todos los datos enviados por los usuarios sean correctamente validados antes de ser procesados por el servidor.

¿Cómo funciona internamente el Fuzzing?

El funcionamiento interno del Fuzzing puede dividirse en una serie de etapas secuenciales.

Inicialmente, la herramienta identifica un punto de entrada de la aplicación, como un formulario, un parámetro de una URL, un campo JSON enviado mediante una API REST o cualquier otro dato controlado por el usuario.

Posteriormente genera automáticamente cientos o miles de valores diferentes utilizando múltiples estrategias de mutación. Algunas entradas consisten en cadenas extremadamente largas, caracteres especiales, valores fuera del rango esperado, estructuras JSON incompletas, números

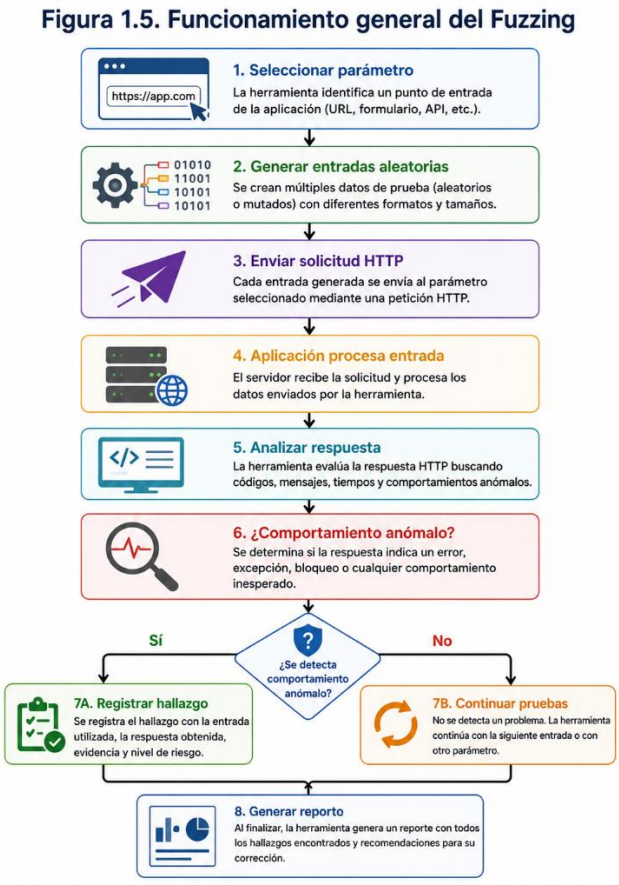
negativos donde únicamente deberían existir valores positivos o secuencias especialmente diseñadas para provocar errores de procesamiento.

Cada uno de estos valores es enviado al servidor mediante solicitudes HTTP. Después de cada petición, la herramienta analiza cuidadosamente la respuesta obtenida buscando indicadores como:

- códigos HTTP inesperados;
- mensajes internos de error;
- excepciones no controladas;
- tiempos de respuesta anormalmente elevados;
- reinicios del servicio;
- pérdida de información;
- comportamientos inconsistentes.

Cuando alguno de estos eventos ocurre, el sistema registra la entrada utilizada y la respuesta recibida para que posteriormente pueda ser analizada por el equipo de desarrollo.

Figura 1.5. Funcionamiento general del Fuzzing



Tipos de entradas utilizadas durante el Fuzzing

Las herramientas DAST modernas utilizan diferentes estrategias para generar datos de prueba. Algunas de las más comunes son:

Tipo de entrada	Ejemplo	Objetivo
Cadenas extremadamente largas	"AAAAAAAAAAAA..."	Detectar errores de validación o consumo excesivo de recursos.
Caracteres especiales	<script>, ', ", %00	Buscar vulnerabilidades como XSS o SQL Injection.
Valores nulos	NULL, vacío	Verificar el manejo correcto de datos faltantes.
Números fuera de rango	-999999, 999999999	Detectar errores lógicos o validaciones insuficientes.
JSON mal formado	{ "user": }	Evaluar el manejo de estructuras inválidas.
Datos aleatorios	Secuencias generadas automáticamente	Descubrir fallos no previstos durante el desarrollo.

Aplicación del Fuzzing en InterLockerUp

En InterLockerUp, la técnica de Fuzzing puede aplicarse sobre diversos puntos de entrada expuestos por la aplicación. Entre ellos destacan los formularios de autenticación, las solicitudes para la asignación de lockers, los servicios de generación y validación de códigos QR, las rutas de la API REST utilizadas por los dispositivos ESP32 y los formularios de administración disponibles para los usuarios con privilegios elevados.

Durante la auditoría utilizando OWASP ZAP, la herramienta enviará automáticamente múltiples variaciones de datos hacia estos componentes con el objetivo de verificar que el backend desarrollado en Node.js valide

correctamente toda la información recibida antes de procesarla o almacenarla en PostgreSQL. Esta estrategia permitirá comprobar la resistencia del sistema frente a entradas inesperadas y confirmar que los mecanismos de validación implementados durante el desarrollo sean suficientes para prevenir vulnerabilidades relacionadas con inyecciones, errores de validación y fallos de procesamiento.

Ventajas del Fuzzing

La utilización de pruebas automatizadas mediante Fuzzing proporciona múltiples beneficios dentro de una auditoría de seguridad.

Entre las ventajas más importantes destacan:

- Permite descubrir vulnerabilidades difíciles de identificar manualmente.
- Automatiza miles de pruebas en poco tiempo.
- Incrementa significativamente la cobertura del análisis dinámico.
- Evalúa la robustez de los mecanismos de validación.
- Detecta errores inesperados durante el procesamiento de datos.
- Complementa las pruebas realizadas mediante Active Scanning.

Limitaciones del Fuzzing

A pesar de sus ventajas, esta técnica también presenta ciertas limitaciones.

Entre ellas pueden mencionarse:

- No garantiza encontrar todas las vulnerabilidades.
- Puede generar grandes volúmenes de información para analizar.
- Algunas pruebas pueden afectar el rendimiento del sistema.
- Requiere interpretar correctamente los resultados para evitar falsos positivos.
- Resulta más efectivo cuando se combina con otras técnicas de análisis dinámico.

Ventajas del Dynamic Application Security Testing (DAST)

El **Dynamic Application Security Testing (DAST)** constituye una de las metodologías más utilizadas dentro de las auditorías de seguridad debido a su capacidad para evaluar el comportamiento real de una aplicación cuando se encuentra en ejecución. A diferencia del análisis estático, que examina únicamente el código fuente, DAST interactúa directamente con el sistema desplegado y verifica si las vulnerabilidades realmente pueden ser

explotadas desde el exterior. Esta característica convierte al análisis dinámico en una herramienta indispensable para validar la efectividad de los controles de seguridad implementados durante el desarrollo.

Una de las principales ventajas del análisis dinámico es que permite identificar vulnerabilidades que únicamente aparecen durante la ejecución del sistema. Muchos problemas relacionados con autenticación, autorización, manejo de sesiones, configuración del servidor o procesamiento de solicitudes HTTP no pueden detectarse mediante el análisis del código fuente, ya que dependen del comportamiento conjunto de todos los componentes que conforman la aplicación.

Otra ventaja importante consiste en que DAST evalúa la aplicación desde la perspectiva de un atacante externo. Al no requerir acceso al código fuente, la herramienta reproduce un escenario muy similar al que enfrentaría un usuario malicioso que intenta comprometer la seguridad del sistema mediante solicitudes HTTP, manipulación de parámetros o explotación de servicios expuestos.

Asimismo, el análisis dinámico permite validar la correcta configuración del entorno donde se ejecuta la aplicación. Aspectos como encabezados HTTP, certificados digitales, políticas de seguridad del navegador, configuración TLS, cookies, mecanismos de autenticación y autorización pueden analizarse directamente durante la ejecución del sistema, proporcionando una visión más completa de la postura de seguridad.

Otra característica relevante es que las herramientas DAST automatizan una gran cantidad de pruebas que manualmente requerirían muchas horas de trabajo. Mediante procesos como Crawling, Active Scanning y Fuzzing, es posible evaluar cientos de rutas, formularios, parámetros y servicios web en pocos minutos, incrementando significativamente la cobertura de la auditoría.

Desde la perspectiva del desarrollo seguro, DAST también facilita la validación de las correcciones realizadas durante el análisis estático. Después de corregir una vulnerabilidad detectada por herramientas como SonarQube, el análisis dinámico permite comprobar que dicha corrección realmente impida la explotación del problema cuando la aplicación se encuentra funcionando.

En proyectos como **InterLockerUp**, donde la plataforma integra un backend desarrollado en Node.js, un frontend web basado en React, una API REST utilizada por dispositivos ESP32 y una base de datos PostgreSQL, el análisis dinámico permite evaluar simultáneamente la interacción entre todos estos componentes, proporcionando una visión integral del estado de seguridad del sistema.

Tabla 1.4. Principales ventajas del análisis DAST

Ventaja	**Descripción**	**Beneficio para InterLockerUp**
Evaluación en tiempo de ejecución	Analiza la aplicación mientras se encuentra funcionando.	Permite verificar que los controles implementados realmente protejan el sistema.
Perspectiva del atacante	Simula ataques externos sobre la aplicación.	Evalúa la seguridad desde un escenario realista.
No requiere código fuente	Funciona únicamente mediante solicitudes HTTP.	Puede analizar aplicaciones completas sin conocer su implementación interna.
Revisión de configuración	Detecta errores en encabezados HTTP, TLS, cookies y sesiones.	Verifica que el entorno de despliegue sea seguro.
Automatización	Ejecuta cientos de pruebas de manera automática.	Incrementa la cobertura de la auditoría.
Validación de correcciones	Comprueba que las vulnerabilidades corregidas ya no sean explotables.	Complementa los resultados obtenidos mediante SAST.

Aplicación práctica en InterLockerUp

La implementación de análisis dinámico aporta un beneficio considerable para **InterLockerUp**, ya que permite validar el comportamiento de los diferentes componentes del sistema una vez desplegados. La plataforma expone interfaces de autenticación, servicios administrativos, procesos de asignación de lockers y una API REST utilizada por dispositivos ESP32. Todos estos componentes pueden ser evaluados mediante herramientas como **OWASP ZAP**, verificando que los mecanismos de autenticación, autorización y validación de entradas funcionen correctamente en condiciones reales de operación.

Adicionalmente, el análisis dinámico facilita comprobar que las vulnerabilidades identificadas previamente mediante SonarQube fueron corregidas de forma efectiva y que no existen nuevas debilidades introducidas durante la integración del sistema.

Desventajas y limitaciones del Dynamic Application Security Testing (DAST)

Aunque el análisis dinámico representa una herramienta muy poderosa para evaluar aplicaciones en funcionamiento, también presenta diversas limitaciones que deben considerarse durante una auditoría de seguridad. Comprender estas limitaciones resulta fundamental para interpretar correctamente los resultados obtenidos y evitar asumir que el análisis dinámico, por sí solo, garantiza la seguridad completa de una aplicación.

La principal limitación de DAST es que únicamente puede analizar funcionalidades que realmente se encuentran disponibles durante la ejecución del sistema. Si determinadas rutas, servicios o componentes no son accesibles durante el análisis, la herramienta no podrá evaluarlos, independientemente de que existan vulnerabilidades en ellos.

Otra limitación importante consiste en que DAST no identifica la ubicación exacta de la vulnerabilidad dentro del código fuente. Aunque puede demostrar que una debilidad existe y es explotable, normalmente no proporciona información suficiente para localizar la línea específica donde debe realizarse la corrección, por lo que resulta necesario complementar los resultados con herramientas SAST.

Asimismo, algunas pruebas ejecutadas durante el análisis activo pueden afectar temporalmente el rendimiento de la aplicación. Procesos como Active Scanning o Fuzzing generan una gran cantidad de solicitudes HTTP que incrementan la carga sobre el servidor. Por esta razón, estas pruebas suelen ejecutarse sobre ambientes de desarrollo, pruebas o preproducción y no directamente sobre sistemas críticos en producción.

Otra limitación relevante es la posibilidad de generar falsos positivos y falsos negativos. Algunas respuestas inesperadas pueden interpretarse incorrectamente como vulnerabilidades, mientras que otras debilidades podrían no detectarse debido a restricciones del entorno o limitaciones propias del motor de análisis.

Finalmente, DAST depende del estado de la aplicación durante el momento de la evaluación. Si ciertas funcionalidades requieren autenticación, configuraciones específicas o datos previamente cargados, será necesario preparar adecuadamente el entorno para obtener una cobertura completa del análisis.

Tabla 1.5. Principales limitaciones del análisis DAST

Limitación	Descripción	Impacto
Requiere la aplicación en ejecución	Solo analiza sistemas desplegados.	No puede utilizarse durante las primeras etapas del desarrollo.
No identifica la línea del código	Detecta la vulnerabilidad pero no su ubicación exacta.	Requiere apoyo de herramientas SAST para facilitar la corrección.
Cobertura limitada	Solo analiza recursos accesibles.	Funcionalidades ocultas pueden quedar fuera del análisis.
Posible impacto sobre el rendimiento	Active Scanning genera numerosas solicitudes HTTP.	Puede afectar temporalmente el desempeño del servidor.
Falsos positivos y falsos negativos	Algunas vulnerabilidades pueden ser reportadas incorrectamente o pasar desapercibidas.	Requiere validación por parte del equipo de seguridad.

Comparación entre SAST y DAST

El análisis estático y el análisis dinámico representan dos enfoques complementarios para evaluar la seguridad del software. Mientras SAST inspecciona el código fuente antes de ejecutar la aplicación, DAST analiza el comportamiento del sistema cuando ya se encuentra funcionando. Debido a estas diferencias, ambos métodos permiten descubrir vulnerabilidades distintas y aportan información complementaria durante una auditoría de seguridad.

En **InterLockerUp**, la combinación de ambas metodologías permite obtener una visión integral de la seguridad del sistema. El análisis estático realizado mediante **SonarQube** identifica problemas directamente en el código fuente del backend desarrollado en Node.js, mientras que el análisis dinámico mediante **OWASP ZAP** verifica que la aplicación desplegada resista ataques dirigidos contra sus interfaces web y servicios REST.

Tabla 1.6. Comparación entre SAST y DAST

Característica	SAST	DAST
Tipo de análisis	Estático	Dinámico
Requiere ejecutar la aplicación	No	Sí
Requiere acceso al código fuente	Sí	No
Momento de ejecución	Durante el desarrollo	Después del despliegue
Detecta vulnerabilidades en el código	Sí	Parcialmente
Detecta errores de configuración	No	Sí
Localiza la línea exacta del problema	Sí	No
Evalúa el comportamiento real	No	Sí
Herramientas utilizadas	SonarQube, Semgrep	OWASP ZAP, Burp Suite

Interactive Application Security Testing (IAST)

Definición de IAST

El **Interactive Application Security Testing (IAST)** es una metodología de evaluación de seguridad que analiza una aplicación mientras esta se encuentra en ejecución mediante la incorporación de agentes de monitoreo directamente dentro del entorno de ejecución del software.

Estos agentes observan continuamente el flujo interno del programa, registrando información relacionada con funciones ejecutadas, consultas realizadas a la base de datos, llamadas a bibliotecas, procesamiento de datos, validaciones, autenticación y demás componentes que intervienen durante la operación normal de la aplicación.

Gracias a este nivel de observación, IAST combina características propias del análisis estático y dinámico. Por una parte, posee conocimiento del código y del flujo interno de ejecución; por otra, analiza el comportamiento real del sistema mientras responde a solicitudes generadas durante pruebas funcionales o automatizadas.

El resultado es un análisis altamente preciso que permite identificar vulnerabilidades con menor cantidad de falsos positivos y localizar de forma casi inmediata el punto exacto donde debe realizarse la corrección.

Objetivos del análisis IAST

La finalidad principal del análisis interactivo consiste en proporcionar una evaluación continua de la seguridad de la aplicación mientras esta ejecuta sus funcionalidades normales.

Entre sus principales objetivos destacan:

- Supervisar el comportamiento interno del software durante su ejecución.
- Detectar vulnerabilidades explotables con mayor precisión.
- Reducir la cantidad de falsos positivos.
- Identificar la ubicación exacta del problema dentro del código.
- Complementar los resultados obtenidos mediante SAST y DAST.
- Integrarse con pruebas funcionales automatizadas.
- Proporcionar retroalimentación inmediata al equipo de desarrollo.

A diferencia del análisis dinámico tradicional, IAST no depende exclusivamente de las respuestas HTTP, sino que también observa directamente el procesamiento interno realizado por la aplicación.

¿Cómo funciona IAST?

El funcionamiento del **Interactive Application Security Testing** se basa en la incorporación de un **agente de instrumentación** dentro de la propia aplicación.

Este agente permanece activo mientras el sistema ejecuta pruebas funcionales, monitoreando continuamente todas las operaciones internas realizadas por el software.

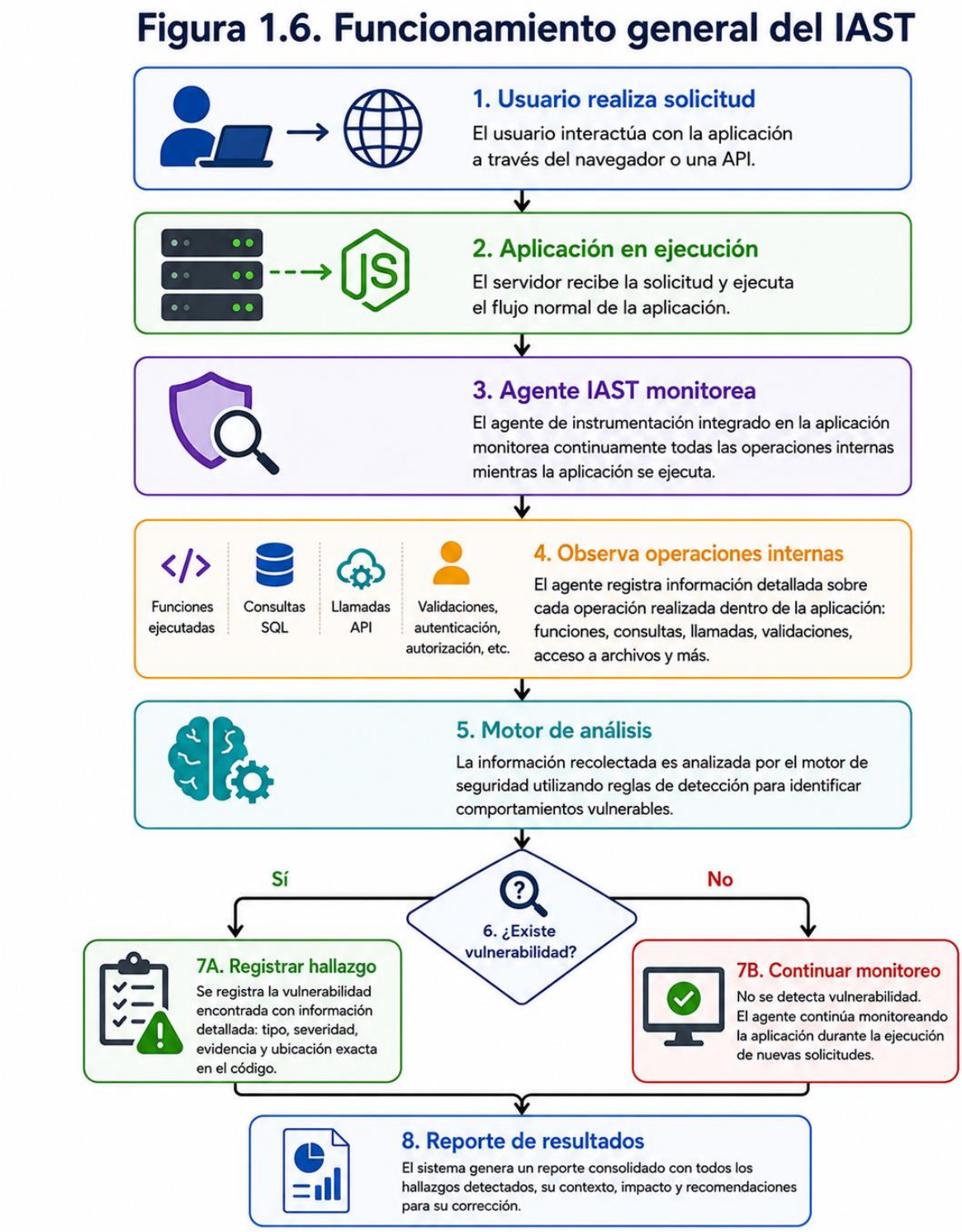
Cuando un usuario envía una solicitud al servidor, el agente registra información relacionada con:

- funciones ejecutadas;
- parámetros recibidos;
- consultas SQL;
- llamadas al sistema;
- acceso a archivos;
- comunicación con APIs;
- validaciones realizadas;
- autenticación;
- autorización.

Posteriormente, esta información es analizada utilizando reglas de seguridad similares a las empleadas por herramientas SAST y DAST.

Sin embargo, al disponer de información mucho más detallada sobre el contexto de ejecución, IAST puede determinar con mayor precisión si realmente existe una vulnerabilidad y cuál es su origen exacto.

Figura 1.6. Funcionamiento general del IAST



Diferencias entre IAST y DAST

Aunque ambas metodologías analizan aplicaciones en ejecución, existen diferencias importantes entre ellas.

DAST observa únicamente aquello que puede verse desde el exterior mediante solicitudes HTTP.

IAST, en cambio, observa directamente el interior de la aplicación.

Esto significa que puede conocer exactamente qué función fue ejecutada, qué consulta SQL se realizó, qué variable recibió un valor determinado y qué validaciones fueron aplicadas antes de producirse una vulnerabilidad.

Como consecuencia, IAST genera menos falsos positivos y facilita considerablemente el proceso de corrección.

Tabla 1.7. Comparación entre DAST e IAST

Característica	**DAST**	**IAST**
Analiza la aplicación en ejecución	Sí	Sí
Observa el interior del programa	No	Sí
Requiere agente dentro de la aplicación	No	Sí
Localiza la línea del código	No	Sí
Precisión	Media	Alta
Falsos positivos	Más frecuentes	Menos frecuentes
Integración con pruebas funcionales	Parcial	Completa

Aplicación en InterLockerUp

Aunque durante el desarrollo de **InterLockerUp** no se implementó una herramienta específica de **IAST**, esta metodología representa una alternativa interesante para futuras etapas de evolución del proyecto. La incorporación de un agente de instrumentación permitiría monitorear en tiempo real procesos críticos como la autenticación mediante JWT, la validación de códigos QR dinámicos, la comunicación entre el backend y

los dispositivos ESP32, así como las consultas realizadas sobre la base de datos PostgreSQL.

Durante las pruebas funcionales del sistema, el agente podría identificar si las solicitudes enviadas desde la interfaz web o desde la API REST alcanzan operaciones sensibles sin haber sido correctamente validadas, proporcionando información detallada sobre el flujo de ejecución y la ubicación exacta de posibles vulnerabilidades. Este nivel de observación complementaría los análisis realizados previamente mediante SonarQube y OWASP ZAP, ofreciendo una cobertura más amplia sobre el comportamiento interno de la aplicación.

Ventajas del IAST

Entre las principales ventajas del análisis interactivo destacan:

- Combina características de SAST y DAST.
- Reduce significativamente los falsos positivos.
- Analiza el comportamiento real de la aplicación.
- Identifica la ubicación exacta de las vulnerabilidades.
- Se integra fácilmente con pruebas funcionales automatizadas.
- Proporciona retroalimentación continua durante el desarrollo.
- Incrementa la precisión del análisis de seguridad.

Desventajas del IAST

A pesar de sus beneficios, IAST también presenta algunas limitaciones:

- Requiere instalar un agente dentro de la aplicación.
- Puede incrementar ligeramente el consumo de recursos durante las pruebas.
- No todas las plataformas ofrecen soporte para instrumentación.
- Generalmente implica un costo superior respecto a otras herramientas.
- Su utilización depende de la compatibilidad con el entorno de desarrollo.

Software Composition Analysis (SCA)

Definición de Software Composition Analysis (SCA)

El **Software Composition Analysis (SCA)** es una metodología de seguridad orientada al análisis automático de los componentes de terceros utilizados por una aplicación. Su finalidad consiste en identificar bibliotecas, frameworks, paquetes y demás dependencias externas que presenten vulnerabilidades conocidas, versiones obsoletas, problemas de licenciamiento o riesgos asociados a la cadena de suministro del software.

A diferencia de SAST, que analiza el código desarrollado por el equipo de programación, y de DAST, que evalúa el comportamiento de la aplicación en ejecución, SCA se concentra exclusivamente en los componentes reutilizados provenientes de proyectos externos.

Para ello, las herramientas SCA inspeccionan archivos como:

- package.json
- package-lock.json
- pom.xml
- requirements.txt
- composer.json
- *.csproj

Posteriormente comparan las versiones encontradas con bases de datos internacionales de vulnerabilidades, como:

- **CVE (Common Vulnerabilities and Exposures)**.
- **NVD (National Vulnerability Database)**.
- **GitHub Security Advisories**.
- **OSV (Open Source Vulnerabilities)**.

Cuando identifican una dependencia vulnerable, generan un reporte indicando:

- componente afectado;
- versión instalada;
- versión recomendada;
- severidad;
- CVE asociado;
- recomendaciones de actualización.

Objetivos del SCA

La implementación de Software Composition Analysis persigue diversos objetivos relacionados con la seguridad de la cadena de suministro del software.

Entre los más importantes destacan:

- Identificar bibliotecas con vulnerabilidades conocidas.
- Detectar versiones obsoletas.
- Recomendar actualizaciones seguras.
- Supervisar continuamente las dependencias del proyecto.
- Reducir riesgos asociados al software de terceros.
- Verificar compatibilidad de licencias.
- Fortalecer la seguridad del ciclo de desarrollo.

Estos objetivos convierten a SCA en un elemento esencial dentro de las estrategias modernas de **DevSecOps**.

¿Cómo funciona el SCA?

El funcionamiento del **Software Composition Analysis** se basa en la inspección automática de las dependencias declaradas por un proyecto.

Inicialmente, la herramienta identifica los archivos donde se encuentran definidas las bibliotecas utilizadas por la aplicación. Posteriormente construye un inventario completo de todos los componentes, incluyendo dependencias directas y dependencias transitivas.

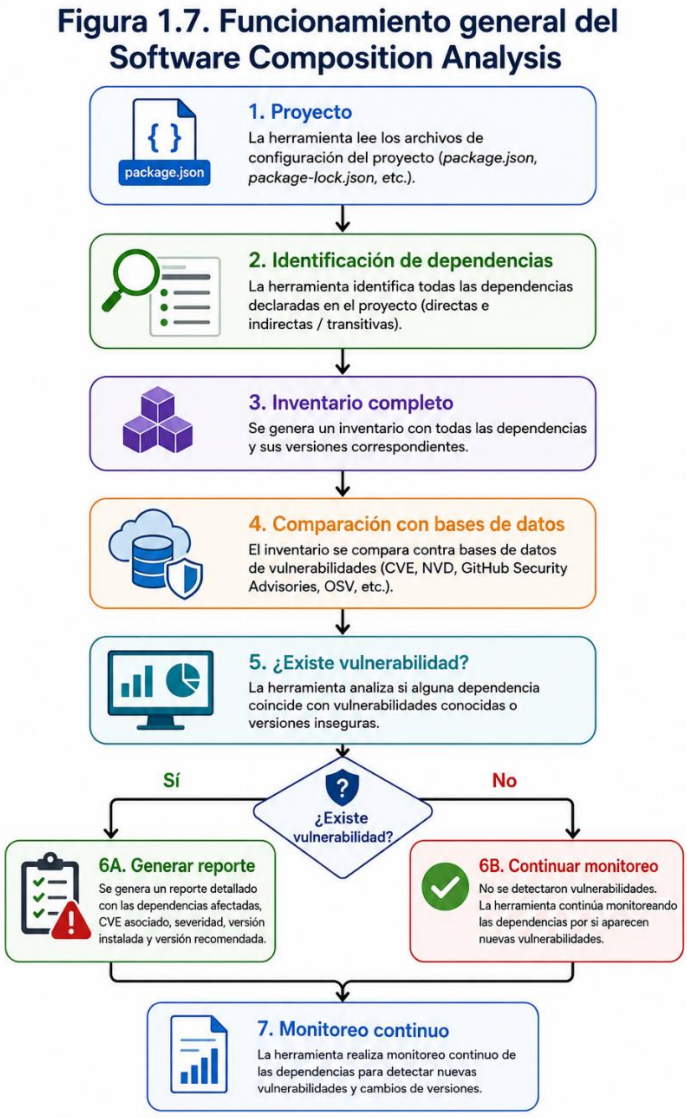
Una vez generado este inventario, la herramienta compara las versiones encontradas contra múltiples bases de datos de vulnerabilidades conocidas.

Cuando identifica una coincidencia, recupera información relacionada con:

- identificador CVE;
- descripción del riesgo;
- nivel de severidad;
- versión afectada;
- versión corregida;
- referencias oficiales.

Finalmente, genera un reporte priorizando las vulnerabilidades según su impacto y proponiendo acciones para su mitigación.

Figura 1.7. Funcionamiento general del Software Composition Analysis



¿Qué vulnerabilidades detecta SCA? Las herramientas SCA permiten identificar diversos riesgos relacionados con el uso de componentes externos.

Entre ellos destacan:

- **Vulnerabilidades conocidas (CVE):** Bibliotecas que presentan vulnerabilidades publicadas oficialmente.
- **Dependencias obsoletas:** Versiones antiguas que ya no reciben mantenimiento.
- **Dependencias transitivas:** Bibliotecas instaladas indirectamente por otros paquetes. Muchas veces estas representan el mayor riesgo.
- **Problemas de licencias:** Uso de componentes incompatibles con la política de licenciamiento del proyecto.

Riesgos de Supply Chain

Ataques dirigidos contra la cadena de suministro del software.

Por ejemplo:

- paquetes maliciosos;
- dependencias comprometidas;
- bibliotecas abandonadas.

Aplicación del SCA en InterLockerUp

En **InterLockerUp**, el análisis de composición de software resulta especialmente importante debido al uso de tecnologías basadas en el ecosistema JavaScript. Tanto el backend desarrollado en Node.js como el frontend implementado con React dependen de múltiples paquetes distribuidos mediante **npm**, los cuales incorporan funcionalidades esenciales para la autenticación, el acceso a la base de datos, la comunicación con APIs, el manejo de sesiones y la generación de códigos QR.

La ejecución de un análisis SCA permitirá verificar que dichas dependencias no presenten vulnerabilidades conocidas registradas en bases de datos internacionales como el **National Vulnerability Database (NVD)** o los **GitHub Security Advisories**. En caso de detectarse una biblioteca vulnerable, el reporte indicará la versión afectada, el identificador CVE correspondiente y la versión recomendada para su actualización.

Durante la auditoría de seguridad del proyecto se empleará una herramienta de análisis de dependencias con el propósito de evaluar la integridad del ecosistema de paquetes utilizado por InterLockerUp y reducir los riesgos asociados a componentes de terceros.

Ventajas del SCA

Las principales ventajas de esta metodología son:

- Detecta vulnerabilidades conocidas automáticamente.
- Reduce riesgos de la cadena de suministro.
- Facilita la actualización de dependencias.
- Automatiza el monitoreo continuo.
- Mejora el cumplimiento normativo.
- Se integra fácilmente con pipelines DevSecOps.
- Complementa SAST y DAST.

Desventajas del SCA

Entre sus principales limitaciones destacan:

- Solo detecta vulnerabilidades conocidas.
- No analiza el código desarrollado por el equipo.
- Requiere mantener actualizadas las bases de datos.
- Puede generar alertas sobre dependencias que realmente no son utilizadas durante la ejecución.
- No sustituye otras metodologías de análisis.

Tabla 1.8 Comparación general entre SAST, DAST, IAST y SCA

Característica	SAST	DAST	IAST	SCA
Analiza código fuente	✓	✗	✓	✗
Analiza aplicación en ejecución	✗	✓	✓	✗
Analiza dependencias externas	✗	✗	Parcial	✓
Requiere aplicación desplegada	✗	✓	✓	✗
Detecta CVE	✗	✗	✗	✓
Detecta vulnerabilidades explotables	Parcial	✓	✓	Parcial
Localiza la línea del código	✓	✗	✓	✗
Herramientas	SonarQube, Semgrep	OWASP ZAP	Contrast, Seeker	OWASP Dependency-Check, Snyk, Dependabot

Modelo de amenazas STRIDE

Definición de STRIDE

STRIDE es un modelo de clasificación de amenazas utilizado durante el diseño y análisis de sistemas de información. Su objetivo consiste en ayudar a los equipos de desarrollo a identificar posibles riesgos de seguridad mediante seis categorías que representan diferentes formas de ataque.

El nombre STRIDE se forma a partir de las iniciales de las siguientes amenazas:

- **S — Spoofing:** suplantación de identidad.
- **T — Tampering:** alteración o manipulación de información.
- **R — Repudiation:** negación de una acción realizada.
- **I — Information Disclosure:** divulgación de información.
- **D — Denial of Service:** denegación del servicio.
- **E — Elevation of Privilege:** elevación de privilegios.

Cada una de estas categorías se relaciona con una propiedad de seguridad que puede verse comprometida.

Tabla 1.9. Relación entre STRIDE y las propiedades de seguridad

Categoría STRIDE	Traducción	Propiedad afectada
Spoofing	Suplantación de identidad	Autenticidad
Tampering	Alteración de información	Integridad
Repudiation	Repudio de acciones	No repudio y trazabilidad
Information Disclosure	Divulgación de información	Confidencialidad
Denial of Service	Denegación del servicio	Disponibilidad
Elevation of Privilege	Elevación de privilegios	Autorización

El valor de este modelo radica en que permite revisar cada elemento de una arquitectura y preguntarse si puede ser objeto de alguna de estas amenazas. De esta manera, se obtiene una visión organizada de los riesgos potenciales y se facilita la definición de controles de seguridad.

Spoofing — Suplantación de identidad

Definición: Spoofing ocurre cuando una persona, aplicación o dispositivo se hace pasar por una identidad legítima con el propósito de obtener acceso a recursos protegidos. Esta amenaza afecta directamente la autenticidad, debido a que el sistema deja de ser capaz de comprobar con certeza quién realiza una determinada acción.

La suplantación puede producirse mediante el robo de credenciales, tokens de sesión, códigos QR, cookies, claves criptográficas o identificadores de dispositivos. También puede involucrar la creación de cuentas falsas o la manipulación de información utilizada durante el proceso de autenticación.

Ejemplo aplicado a InterLockerUp

En InterLockerUp, un atacante podría obtener el **JWT de un administrador** y reutilizarlo para ingresar al panel administrativo. Si el token sigue siendo válido y el backend no aplica controles adicionales, el atacante podría consultar datos de usuarios, gestionar lockers, revisar incidencias o modificar asignaciones como si fuera el administrador legítimo.

Otro escenario de Spoofing consistiría en copiar un **código QR dinámico** perteneciente a un estudiante y presentarlo frente al lector del locker. El sistema documenta que el QR se encuentra asociado al alumno, al locker y a un periodo de vigencia; por ello, su robo o reutilización indebida constituye un riesgo de suplantación que debe considerarse dentro del modelo de amenazas.

También existe una posible amenaza contra el componente IoT. Si una persona obtuviera las credenciales de un dispositivo ESP32 autorizado, podría intentar hacerse pasar por el hardware legítimo y enviar solicitudes al backend para validar accesos o registrar eventos fraudulentos.

Controles de mitigación

Los controles más adecuados incluyen:

- expiración limitada de tokens JWT;
- verificación de firma;
- autenticación multifactor mediante OTP;
- revocación de sesiones;
- asociación del token con el dispositivo;
- códigos QR de un solo uso o corta duración;

- autenticación individual de cada dispositivo IoT;
- comunicación mediante HTTPS;
- rotación periódica de secretos.

La documentación técnica indica que InterLockerUp utiliza JWT, verificación de dispositivo, OTP para equipos no reconocidos y códigos temporales, los cuales constituyen medidas relevantes contra la suplantación.

Tampering — Alteración de información

Definición: Tampering se refiere a la modificación no autorizada de datos, mensajes, configuraciones, archivos o instrucciones dentro de un sistema. Esta categoría afecta la integridad de la información, ya que los datos dejan de representar su estado original o legítimo.

La alteración puede ocurrir durante el almacenamiento, la transmisión o el procesamiento de la información. Un atacante podría modificar solicitudes HTTP, registros de base de datos, archivos subidos por usuarios, tokens, respuestas del servidor o comandos enviados hacia un dispositivo físico.

Ejemplo aplicado a InterLockerUp

Un estudiante podría interceptar o modificar una solicitud enviada desde el frontend hacia el backend para intentar alterar el identificador del locker solicitado. Aunque la interfaz muestre únicamente opciones permitidas, un usuario malicioso podría modificar manualmente el cuerpo JSON y sustituir el número del locker por otro diferente.

Otro ejemplo consiste en alterar la respuesta que el backend envía al ESP32. Si la comunicación entre ambos componentes no está protegida, un atacante ubicado dentro de la misma red podría intentar cambiar una respuesta de acceso denegado por una respuesta de autorización y provocar la apertura de la cerradura.

La implementación IoT utiliza solicitudes HTTP en formato JSON entre el ESP32 y el backend dentro de una red local. Este flujo debe considerarse un punto sensible, ya que la manipulación del contenido transmitido podría comprometer directamente el acceso físico al locker.

También podría producirse Tampering sobre los registros de auditoría, tokens QR, solicitudes de lockers o evidencias fotográficas de incidencias. La documentación administrativa menciona el uso de validación de integridad

SHA-256 para archivos, lo cual constituye un control dirigido precisamente a detectar modificaciones no autorizadas.

Controles de mitigación

- validación de todos los datos en el backend;
- uso de HTTPS/TLS;
- firmas digitales o HMAC;
- hashes de integridad;
- consultas parametrizadas;
- controles de autorización;
- validación de estados y reglas de negocio;
- permisos mínimos sobre la base de datos;
- registros inmutables o protegidos.

Repudiation — Repudio

Definición: Repudiation ocurre cuando un usuario niega haber realizado una acción y el sistema no posee evidencia suficiente para demostrar lo contrario. Esta amenaza afecta la trazabilidad, la responsabilidad y el principio de no repudio.

Un sistema vulnerable al repudio carece de registros confiables, marcas de tiempo, identificación de usuarios o mecanismos de auditoría capaces de reconstruir lo ocurrido durante un incidente.

Ejemplo aplicado a InterLockerUp

Un administrador podría liberar un locker, rechazar una solicitud o modificar el estado de una incidencia y posteriormente afirmar que nunca realizó dicha operación. Si el sistema no almacena el identificador del administrador, la fecha, la hora, la dirección IP y la acción ejecutada, resultaría difícil demostrar quién realizó el cambio.

De forma similar, un estudiante podría negar haber abierto un locker. El sistema contempla una tabla de access_logs y registra tanto accesos permitidos como denegados, lo cual proporciona evidencia para reconstruir el evento.

El script de base de datos también separa los registros operativos dentro del esquema iot y permite que los administradores tengan acceso de lectura para auditoría. Esta organización puede contribuir a mantener evidencia de

los eventos, aunque debe protegerse contra borrados o modificaciones no autorizadas.

Controles de mitigación

- logs de auditoría detallados;
- marcas de tiempo confiables;
- identificación del usuario y del dispositivo;
- registro de dirección IP;
- protección contra modificación o eliminación;
- retención adecuada;
- centralización de registros;
- alertas sobre acciones críticas;
- firmas digitales cuando corresponda.

Information Disclosure — Divulgación de información

Definición: Information Disclosure ocurre cuando información sensible es expuesta a usuarios, aplicaciones o dispositivos que no cuentan con autorización para consultarla. Esta amenaza compromete la confidencialidad del sistema.

La divulgación puede producirse mediante errores detallados, APIs sin protección, bases de datos mal configuradas, archivos públicos, tokens visibles, credenciales almacenadas en repositorios o respuestas que contienen más información de la necesaria.

Ejemplo aplicado a InterLockerUp

InterLockerUp procesa información como nombres, matrículas, correos institucionales, asignaciones de lockers, registros de acceso, incidencias y datos de administradores. Si una ruta del backend devolviera información de todos los estudiantes a un usuario común, se produciría una divulgación de datos personales.

Otro escenario sería la exposición de variables como JWT_SECRET, DB_PASSWORD o claves de servicios externos en el repositorio. Los manuales técnicos indican que estas configuraciones deben almacenarse mediante variables de entorno, lo cual constituye una práctica adecuada siempre que el archivo .env no se publique.

También existe riesgo si el servidor devuelve errores internos con rutas, consultas SQL, versiones del software o fragmentos de código. El análisis SAST

documentado previamente detectó la exposición de información mediante X-Powered-By, demostrando que este tipo de amenaza ya fue considerado durante el desarrollo.

Controles de mitigación

- control de acceso por rol;
- cifrado de datos en tránsito y reposo;
- mensajes de error genéricos;
- variables de entorno;
- exclusión de secretos del repositorio;
- minimización de datos;
- consultas limitadas;
- clasificación de información;
- políticas de privacidad;
- uso de HTTPS.

Denial of Service — Denegación del servicio

Definición: Denial of Service (DoS) consiste en afectar la disponibilidad de una aplicación, servicio o recurso mediante el agotamiento de su capacidad, la interrupción de sus comunicaciones o la explotación de fallos que provoquen bloqueos.

El objetivo del atacante no siempre es obtener información. En muchos casos, busca impedir que los usuarios legítimos puedan utilizar el sistema.

Ejemplo aplicado a InterLockerUp

Un atacante podría enviar miles de solicitudes al endpoint encargado de validar códigos QR o PIN temporales. Si el backend no implementa límites de solicitudes, el servidor podría consumir excesivos recursos y dejar de responder.

La disponibilidad también puede verse comprometida por la dependencia de la red WiFi. La documentación de implementación reconoce que la estabilidad de la conexión es un factor crítico para el funcionamiento del prototipo IoT. Si el ESP32 pierde comunicación con el backend, el locker no podrá validar códigos ni recibir instrucciones de apertura.

También podría producirse una denegación de servicio mediante el envío de archivos excesivamente grandes al módulo de incidencias. El análisis de

SonarQube detectó precisamente la ausencia de límites en multer, posteriormente corregida mediante una restricción de tamaño.

Controles de mitigación

- rate limiting;
- límites de tamaño de archivos;
- timeouts;
- validación de cargas;
- colas de procesamiento;
- monitoreo del consumo de recursos;
- redundancia;
- respaldos;
- manejo de errores;
- mecanismos de recuperación;
- disponibilidad alternativa para eventos críticos.

Elevation of Privilege — Elevación de privilegios

Definición: Elevation of Privilege ocurre cuando un usuario obtiene permisos superiores a los que originalmente le fueron asignados. Esta amenaza afecta la autorización y el principio de mínimo privilegio.

Puede originarse por controles de acceso insuficientes, validaciones realizadas únicamente en el frontend, errores en la asignación de roles o permisos excesivos en la base de datos.

Ejemplo aplicado a InterLockerUp

Un estudiante autenticado podría modificar una solicitud HTTP para intentar acceder a un endpoint administrativo, consultar datos globales o asignar lockers sin autorización. Aunque la interfaz no muestre estas funciones, el backend debe comprobar el rol en cada ruta protegida.

Otro escenario consistiría en modificar el campo role incluido en un token o en una solicitud para intentar convertirse en administrador. El sistema utiliza JWT que incluyen el identificador y el rol del usuario; por lo tanto, la firma del token y la validación estricta del rol son controles fundamentales.

La base de datos cuenta con grupos separados para administradores, alumnos y dispositivos IoT, así como permisos distintos sobre los esquemas gestion e iot. Este diseño RBAC representa una aplicación directa del

principio de mínimo privilegio y reduce el impacto de una posible elevación de permisos.

Controles de mitigación

- autorización en el backend;
- RBAC;
- mínimo privilegio;
- validación de roles en cada endpoint;
- separación de funciones;
- tokens firmados;
- revisión de permisos de base de datos;
- prohibición de confiar en datos enviados por el cliente;
- auditoría de cambios de privilegios.

Resumen de amenazas STRIDE aplicadas a InterLockerUp

Tabla 1.10. Ejemplos propios de STRIDE para InterLockerUp

Categoría	Ejemplo propio	Activo afectado	Control principal
Spoofing	Robo de JWT, OTP o QR para hacerse pasar por un alumno o administrador.	Identidad y acceso al locker	MFA, expiración, firmas y revocación
Tampering	Modificación de una petición HTTP o de la respuesta enviada al ESP32.	Integridad del acceso	TLS, validación en backend y firmas
Repudiation	Un usuario niega haber abierto o liberado un locker.	Trazabilidad	Logs protegidos y marcas de tiempo
Information Disclosure	Exposición de matrículas, correos, tokens o secretos.	Datos personales y credenciales	Control de acceso, cifrado y variables de entorno

Denial of Service	Saturación de endpoints o interrupción de la red WiFi.	Disponibilidad del servicio	Rate limiting, monitoreo y redundancia
Elevation of Privilege	Un alumno accede a funciones administrativas.	Autorización y administración	RBAC y validación de roles

Importancia de STRIDE en InterLockerUp

La importancia de STRIDE para InterLockerUp radica en que el proyecto no se limita a una aplicación web convencional. El sistema conecta procesos digitales con acciones físicas, por lo que una vulnerabilidad puede tener consecuencias directas sobre la apertura de un locker, la privacidad de los estudiantes o la continuidad del servicio.

El modelo permite analizar de forma ordenada cada componente de la arquitectura:

- usuarios;
- módulos web;
- backend;
- base de datos;
- servicio de correo;
- dispositivos ESP32;
- lector QR;
- relay;
- cerradura electrónica;
- red WiFi;
- registros de auditoría.

Cada uno de estos elementos puede relacionarse con una o varias amenazas STRIDE. Esto permitirá que, durante **SA.2**, se construya un Diagrama de Flujo de Datos, se definan límites de confianza y se elabore una tabla con amenazas específicas y mitigaciones.

Shift Left Security

Definición de Shift Left Security

El término **Shift Left Security** puede traducirse como **"desplazar la seguridad hacia la izquierda"**, haciendo referencia a la línea de tiempo del ciclo de vida del desarrollo de software (Software Development Life Cycle, SDLC). En un diagrama tradicional del SDLC, las primeras fases del proyecto se representan a la izquierda (planificación, análisis, diseño e implementación), mientras que las etapas finales (pruebas, despliegue y mantenimiento) se ubican hacia la derecha.

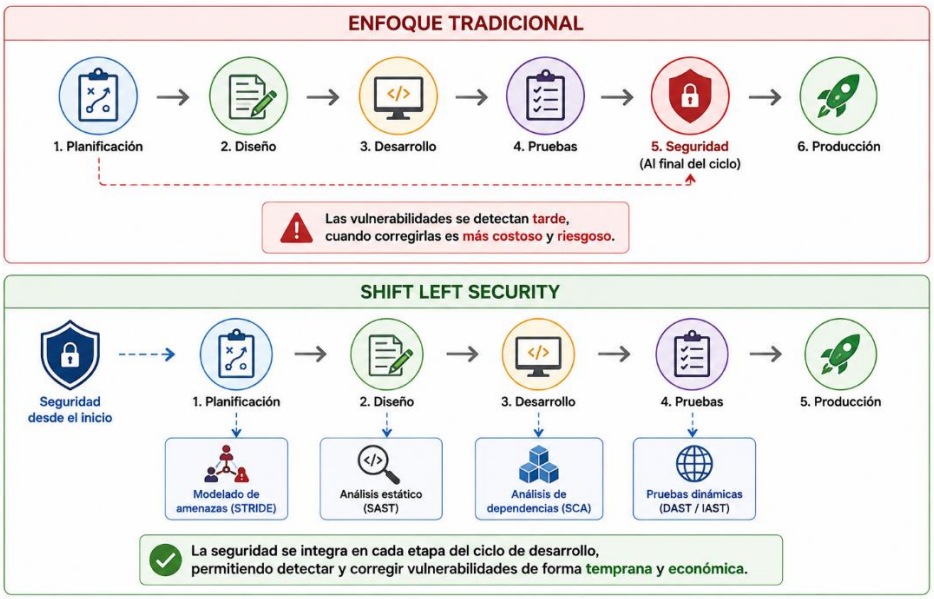
El objetivo principal de esta estrategia consiste en incorporar controles de seguridad desde las primeras fases del desarrollo, permitiendo detectar y corregir vulnerabilidades antes de que el sistema sea desplegado o entre en operación.

Bajo este enfoque, actividades como el modelado de amenazas, el análisis estático del código (SAST), la revisión de dependencias (SCA), las revisiones entre pares (*code reviews*) y las pruebas automatizadas forman parte del proceso habitual de desarrollo y no constituyen actividades aisladas realizadas únicamente al finalizar el proyecto.

Esta integración temprana favorece la construcción de software más seguro, reduce los costos asociados a la corrección de vulnerabilidades y mejora la calidad general del producto.

Figura 1.8. Comparación entre el enfoque tradicional y Shift Left Security

Figura 1.8. Comparación entre el enfoque tradicional y Shift Left Security



Objetivos de Shift Left Security

La estrategia Shift Left Security persigue diversos objetivos orientados a fortalecer la seguridad del software desde su origen.

Entre los más importantes destacan:

- Detectar vulnerabilidades durante las primeras etapas del desarrollo.
- Reducir el costo asociado a la corrección de errores.
- Integrar la seguridad como parte del proceso de desarrollo.
- Automatizar controles de seguridad dentro del ciclo de integración continua.
- Incrementar la calidad del software.
- Reducir la probabilidad de incidentes en producción.
- Fomentar una cultura de desarrollo seguro dentro de los equipos de trabajo.

Estos objetivos permiten que la seguridad deje de ser una actividad reactiva y se convierta en un proceso continuo presente durante todo el ciclo de vida del software.

Beneficios de Shift Left Security

La implementación de Shift Left Security ofrece ventajas tanto para el equipo de desarrollo como para la organización.

Uno de los beneficios más importantes consiste en la **detección temprana de vulnerabilidades**. Identificar un problema durante la implementación permite corregirlo rápidamente, evitando que el error continúe propagándose hacia otras fases del proyecto.

Otro beneficio significativo es la **reducción de costos**. Diversos estudios de ingeniería de software han demostrado que corregir una vulnerabilidad durante el desarrollo resulta considerablemente menos costoso que hacerlo después del despliegue, cuando ya existen usuarios utilizando la aplicación y los cambios requieren procesos adicionales de validación y liberación.

Asimismo, este enfoque favorece la **automatización** de actividades de seguridad. Herramientas como SonarQube, OWASP Dependency-Check, Snyk o GitHub Dependabot pueden ejecutarse automáticamente cada vez que un desarrollador realiza cambios en el repositorio, proporcionando retroalimentación inmediata sobre la calidad y seguridad del código.

Desde el punto de vista organizacional, Shift Left Security también promueve una **cultura de responsabilidad compartida**, donde la seguridad deja de depender exclusivamente de un equipo especializado y pasa a formar parte del trabajo cotidiano de desarrolladores, arquitectos, analistas y responsables de calidad.

Tabla 1.11. Beneficios de Shift Left Security

Beneficio	Impacto en el desarrollo
Detección temprana	Reduce el número de vulnerabilidades que llegan a producción.
Menor costo de corrección	Disminuye el tiempo y esfuerzo necesarios para resolver problemas.
Automatización	Facilita la ejecución continua de pruebas de seguridad.
Mayor calidad del software	Incrementa la estabilidad y mantenibilidad del sistema.
Desarrollo seguro	Integra la seguridad como parte del trabajo diario del equipo.

Reducción de riesgos	Disminuye la probabilidad de incidentes de seguridad en producción.

Aplicación de Shift Left Security en InterLockerUp

El proyecto **InterLockerUp** constituye un ejemplo adecuado para aplicar los principios de Shift Left Security, ya que integra diversos componentes tecnológicos que deben protegerse desde las primeras etapas del desarrollo. La plataforma incorpora un backend desarrollado en Node.js, un frontend basado en React, una base de datos PostgreSQL, una API REST y dispositivos ESP32 encargados de controlar la apertura de lockers inteligentes. Cada uno de estos elementos representa una posible superficie de ataque que debe evaluarse antes del despliegue del sistema.

Durante el desarrollo del proyecto ya se implementaron actividades que reflejan este enfoque. La utilización de **SonarQube** permitió ejecutar análisis estáticos del código fuente para detectar vulnerabilidades y problemas de calidad antes de liberar nuevas versiones del backend. De igual forma, la revisión de dependencias mediante herramientas SCA ayuda a identificar bibliotecas vulnerables antes de incorporarlas al proyecto, mientras que el modelado de amenazas basado en STRIDE facilita anticipar riesgos desde la fase de diseño.

Posteriormente, durante la etapa de pruebas, la ejecución de **OWASP ZAP** permitirá validar que la aplicación desplegada resista ataques sobre sus interfaces web y servicios REST. Esta combinación de actividades demuestra cómo la seguridad puede integrarse progresivamente a lo largo del ciclo de vida del proyecto, en lugar de concentrarse únicamente al final del desarrollo.

Relación entre Shift Left Security y las metodologías de análisis

La estrategia Shift Left Security no reemplaza las metodologías de análisis de seguridad; por el contrario, las organiza dentro del ciclo de vida del desarrollo para que se ejecuten en el momento más oportuno.

En este contexto:

- **SAST** se aplica durante la implementación para revisar el código fuente antes de su integración.

- **SCA** analiza continuamente las dependencias incorporadas al proyecto.
- **STRIDE** se utiliza durante el análisis y diseño para identificar amenazas potenciales.
- **DAST** se ejecuta una vez que la aplicación puede desplegarse en un entorno de pruebas.
- **IAST** complementa las pruebas funcionales mediante la observación del comportamiento interno del sistema.

La integración de estas metodologías dentro de un enfoque Shift Left Security permite construir aplicaciones más seguras desde su origen y reducir considerablemente el riesgo de vulnerabilidades en producción.

DevSecOps y el pipeline de desarrollo seguro

Definición de DevSecOps

DevSecOps es una metodología de desarrollo de software que integra de forma continua las actividades de **desarrollo (Development)**, **seguridad (Security)** y **operaciones (Operations)** dentro de un proceso automatizado conocido como **pipeline**.

Su objetivo principal consiste en incorporar controles de seguridad en cada etapa del ciclo de vida del desarrollo, evitando que las vulnerabilidades sean detectadas únicamente al finalizar el proyecto. Para lograrlo, DevSecOps automatiza tareas como el análisis estático del código, la revisión de dependencias, la ejecución de pruebas dinámicas, el análisis de infraestructura y la validación de configuraciones antes de autorizar el despliegue de una nueva versión.

Este enfoque se encuentra estrechamente relacionado con el principio de **Shift Left Security**, ya que ambos promueven la detección temprana de vulnerabilidades y la integración continua de mecanismos de protección.

¿Qué es un pipeline DevSecOps?

Un **pipeline DevSecOps** es una secuencia automatizada de actividades que se ejecutan cada vez que un desarrollador realiza cambios en el código fuente del proyecto. Su finalidad consiste en verificar que las modificaciones introducidas cumplan con los estándares de calidad, funcionamiento y seguridad antes de ser integradas al sistema principal o desplegadas en un entorno de producción.

Cada etapa del pipeline ejecuta tareas específicas de forma automática. Si alguna de estas actividades detecta un problema crítico, el proceso puede detenerse hasta que la vulnerabilidad sea corregida. Esto evita que código inseguro continúe avanzando hacia las siguientes fases del ciclo de desarrollo.

Gracias a este mecanismo, la seguridad deja de depender exclusivamente de revisiones manuales y pasa a formar parte del proceso cotidiano de construcción del software.

Etapas del pipeline DevSecOps

Un pipeline DevSecOps puede variar según las necesidades de cada organización; sin embargo, generalmente incorpora las siguientes etapas:

Pre-commit: Corresponde al momento en que el desarrollador realiza cambios sobre su copia local del proyecto. Antes de enviar el código al repositorio, pueden ejecutarse herramientas de análisis estático, revisión de formato, pruebas unitarias y validaciones básicas.

En esta etapa es recomendable ejecutar herramientas como:

- SonarLint.
- ESLint.
- Prettier.
- Git Hooks.
- Secret Scanners.

El objetivo consiste en detectar problemas antes de que el código sea compartido con el resto del equipo.

Repositorio Git: Una vez verificados los cambios localmente, el desarrollador realiza un **commit** y posteriormente envía el código al repositorio central utilizando plataformas como GitHub, GitLab o Azure DevOps.

Cada actualización del repositorio constituye el punto de partida para ejecutar automáticamente el pipeline de integración continua.

Integración Continua (Continuous Integration): Durante esta fase se construye automáticamente la aplicación y se ejecutan diferentes procesos de validación.

Entre ellos destacan:

- compilación;
- pruebas unitarias;
- análisis SAST;
- revisión SCA;
- validación de calidad;
- generación de artefactos.

Si alguna prueba falla, el pipeline se detiene hasta que el problema sea corregido.

Entorno de Staging: Cuando el proyecto supera las validaciones iniciales, la aplicación es desplegada automáticamente en un entorno de **Staging**.

Este ambiente reproduce las condiciones de producción sin afectar a los usuarios finales y permite ejecutar pruebas adicionales de funcionamiento y seguridad.

Pruebas DAST: Una vez desplegada la aplicación en Staging, se ejecutan herramientas de análisis dinámico como **OWASP ZAP**.

Durante esta etapa se realizan:

- Crawling.
- Passive Scanning.
- Active Scanning.
- Fuzzing.

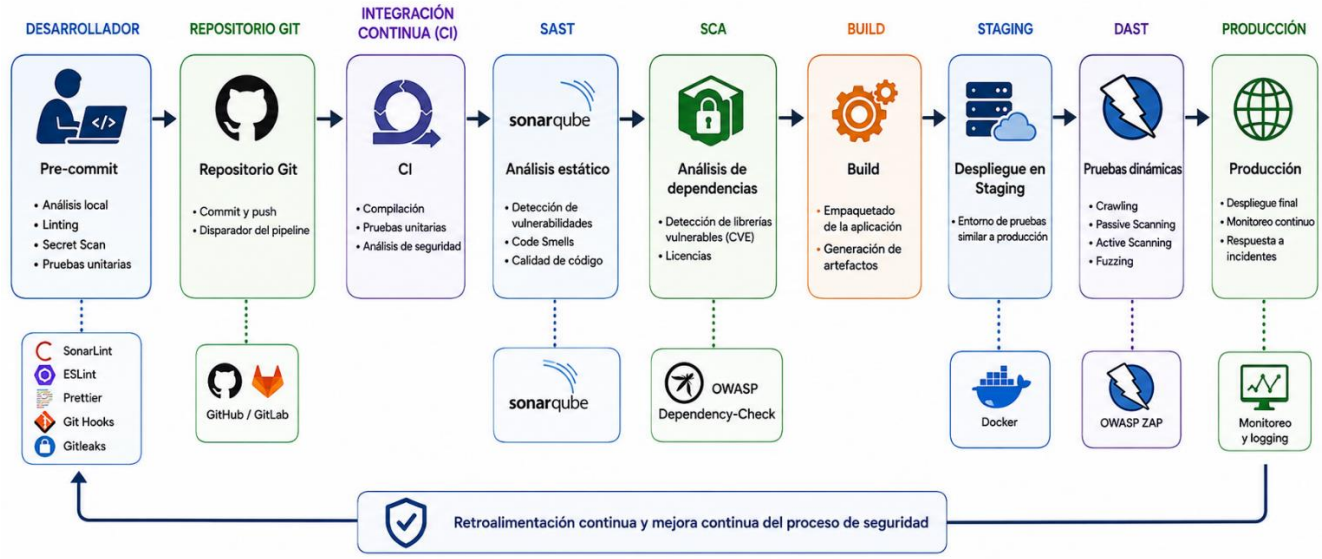
Estas pruebas permiten comprobar que la aplicación desplegada resista ataques reales antes de ser publicada.

Producción: Únicamente cuando todas las etapas anteriores concluyen satisfactoriamente, la aplicación puede ser desplegada en el entorno de producción.

Este proceso reduce considerablemente la probabilidad de liberar versiones que contengan vulnerabilidades críticas.

Figura 1.9. Pipeline DevSecOps aplicado al desarrollo seguro

Figura 1.9. Pipeline DevSecOps aplicado al desarrollo seguro



Aplicación del pipeline DevSecOps en InterLockerUp

La arquitectura de **InterLockerUp** permite implementar un pipeline DevSecOps que integre controles automáticos de seguridad durante todo el ciclo de desarrollo. Cada modificación realizada sobre el backend desarrollado en Node.js o el frontend basado en React puede iniciar automáticamente un proceso de validación que incluya análisis estático del código, revisión de dependencias y pruebas dinámicas antes de aprobar su despliegue.

En un escenario de implementación continua, el pipeline podría funcionar de la siguiente manera:

1. El desarrollador realiza cambios sobre el código del proyecto.
2. Antes del commit, se ejecutan herramientas de validación local y análisis estático.
3. Al enviar los cambios al repositorio Git, el servidor de integración continua compila la aplicación y ejecuta **SonarQube** para detectar vulnerabilidades en el código fuente.
4. Posteriormente se ejecuta una herramienta **SCA** para revisar las dependencias instaladas mediante **npm** y verificar que no existan bibliotecas con vulnerabilidades conocidas.
5. Si todas las validaciones son satisfactorias, la aplicación se despliega automáticamente en un entorno de **Staging**.

6. Finalmente, **OWASP ZAP** realiza pruebas dinámicas sobre las interfaces web y la API REST utilizada por los dispositivos ESP32.
7. Solo cuando todas las verificaciones concluyen exitosamente, la nueva versión es liberada al entorno de producción.

Este flujo garantiza que los controles de seguridad se ejecuten de forma automática y continua, reduciendo la posibilidad de introducir vulnerabilidades durante la evolución del proyecto.

Beneficios del pipeline DevSecOps

La adopción de un pipeline DevSecOps proporciona múltiples beneficios para el desarrollo seguro de software.

Entre los más importantes destacan:

- Automatización de controles de seguridad.
- Detección temprana de vulnerabilidades.
- Integración continua de pruebas SAST, SCA y DAST.
- Reducción de errores humanos.
- Mayor velocidad en el proceso de liberación.
- Mejor calidad del software.
- Menor costo de corrección.
- Mayor confianza antes del despliegue.

Tabla 1.12. Beneficios del pipeline DevSecOps

Beneficio	Impacto en el proyecto
Automatización	Reduce la intervención manual durante las revisiones de seguridad.
Integración continua	Ejecuta controles de seguridad en cada cambio del código.
Detección temprana	Identifica vulnerabilidades antes del despliegue.
Calidad del software	Favorece la estabilidad y mantenibilidad del sistema.
Reducción de riesgos	Disminuye la probabilidad de incidentes en producción.

Entrega continua	Permite liberar nuevas versiones con mayor confianza.

Falsos positivos y falsos negativos en las herramientas de análisis de seguridad

Definición de falso positivo

Un **falso positivo** ocurre cuando una herramienta de análisis identifica una posible vulnerabilidad que, después de ser revisada por el equipo de desarrollo, se determina que no representa un riesgo real para la aplicación.

En otras palabras, el sistema de análisis interpreta determinadas características del código o del comportamiento de la aplicación como si fueran inseguras, aunque en el contexto específico del proyecto existan controles adicionales que impidan su explotación.

Los falsos positivos suelen producirse porque las herramientas trabajan mediante reglas generales que no siempre consideran toda la lógica de negocio implementada por los desarrolladores.

Por ejemplo, una herramienta SAST podría detectar una consulta SQL construida dinámicamente y reportarla como una posible vulnerabilidad de **SQL Injection**. Sin embargo, después de revisar el código, el equipo puede comprobar que todos los parámetros son previamente validados y que la consulta no puede ser manipulada por un atacante.

En este caso, el reporte constituye un falso positivo.

Características de un falso positivo

Los falsos positivos presentan generalmente las siguientes características:

- La herramienta reporta una vulnerabilidad.
- Después de la revisión manual, se concluye que no existe un riesgo real.
- No implica necesariamente una falla del software.
- Requiere tiempo adicional para validar el hallazgo.
- Puede disminuir la confianza del equipo en la herramienta si ocurre con demasiada frecuencia.

Definición de falso negativo

Un **falso negativo** ocurre cuando una herramienta de análisis no detecta una vulnerabilidad que realmente existe dentro de la aplicación.

En este escenario, el sistema de análisis concluye incorrectamente que el software es seguro, permitiendo que una vulnerabilidad permanezca oculta y pueda ser explotada posteriormente por un atacante.

Los falsos negativos representan el tipo de error más peligroso dentro de una auditoría de seguridad, ya que generan una falsa sensación de protección. El equipo de desarrollo puede asumir que la aplicación no presenta problemas importantes y proceder con su despliegue, cuando en realidad todavía existen vulnerabilidades críticas sin identificar.

Este tipo de situaciones puede ocurrir cuando la vulnerabilidad depende de condiciones muy específicas de ejecución, configuraciones particulares del entorno o patrones que aún no forman parte de las reglas utilizadas por la herramienta.

Características de un falso negativo

Los falsos negativos presentan las siguientes características:

- Existe una vulnerabilidad real.
- La herramienta no la detecta.
- El sistema aparentemente supera todas las verificaciones.
- La aplicación puede ser desplegada con fallos de seguridad ocultos.
- Incrementa significativamente el riesgo para la organización.

Comparación entre falso positivo y falso negativo

Aunque ambos conceptos representan errores de clasificación, sus consecuencias son muy diferentes.

Tabla 1.13. Comparación entre falsos positivos y falsos negativos

Característica	Falso positivo	Falso negativo
¿Existe realmente la vulnerabilidad?	No	Sí
¿La herramienta la reporta?	Sí	No

Consecuencia principal	Tiempo adicional de revisión	Vulnerabilidad permanece sin corregir
Impacto en la seguridad	Bajo	Alto
Riesgo para la organización	Moderado	Muy elevado

¿Por qué un falso negativo es más peligroso?

Desde el punto de vista de la ciberseguridad, un falso negativo representa un riesgo considerablemente mayor que un falso positivo.

Cuando ocurre un falso positivo, el principal efecto consiste en invertir tiempo adicional revisando un problema que finalmente no requiere corrección. Aunque esto puede afectar la productividad del equipo, la seguridad del sistema permanece intacta.

En cambio, un falso negativo permite que una vulnerabilidad real permanezca oculta durante el proceso de desarrollo. Como consecuencia, la aplicación puede ser liberada con una debilidad explotable que posteriormente sea utilizada por un atacante para comprometer la confidencialidad, integridad o disponibilidad del sistema.

Por esta razón, las organizaciones suelen preferir herramientas que generen algunos falsos positivos antes que herramientas que presenten una elevada tasa de falsos negativos. La revisión manual de un hallazgo inexistente implica un costo relativamente bajo, mientras que la explotación de una vulnerabilidad no detectada puede ocasionar pérdida de información, interrupción de servicios, sanciones legales y afectaciones económicas.

Ejemplos aplicados a InterLockerUp

La diferencia entre ambos conceptos puede comprenderse mejor mediante ejemplos relacionados con **InterLockerUp**.

Ejemplo de falso positivo: Durante el análisis estático del backend con **SonarQube**, la herramienta podría advertir sobre el uso de una función considerada potencialmente insegura para construir una consulta. Sin embargo, al revisar el código, el equipo confirma que todos los parámetros utilizados provienen de listas controladas y no pueden ser modificados por

los usuarios. En este caso, el hallazgo no representa una vulnerabilidad explotable y corresponde a un falso positivo.

Otro ejemplo podría presentarse durante un análisis con **OWASP ZAP**, cuando la herramienta detecta un encabezado HTTP faltante y lo clasifica como una vulnerabilidad de severidad media. Después del análisis manual, se determina que dicho encabezado no aplica al entorno específico del proyecto y que no existe un riesgo real.

Ejemplo de falso negativo: Supóngase que el backend de **InterLockerUp** contiene una validación insuficiente en un endpoint encargado de gestionar solicitudes de lockers. Si ninguna herramienta identifica esta condición durante las pruebas, la aplicación podría ser desplegada con una vulnerabilidad que permita modificar información sin autorización.

Otro escenario sería una dependencia instalada mediante **npm** que incorpora una vulnerabilidad recientemente publicada y que todavía no ha sido incluida en las bases de datos utilizadas por la herramienta SCA. Aunque el análisis no reporte problemas, la vulnerabilidad continúa presente y podría ser explotada posteriormente.

Estos ejemplos muestran por qué resulta indispensable complementar diferentes metodologías de análisis y realizar revisiones manuales cuando sea necesario.

Estrategias para reducir falsos positivos y falsos negativos

Aunque no es posible eliminarlos completamente, existen diversas prácticas que permiten disminuir la probabilidad de obtener resultados incorrectos.

Entre las más importantes destacan:

- Mantener actualizadas las herramientas de análisis.
- Utilizar reglas adaptadas al lenguaje y arquitectura del proyecto.
- Combinar metodologías SAST, DAST, IAST y SCA.
- Ejecutar revisiones manuales sobre hallazgos críticos.
- Integrar pruebas funcionales y de seguridad dentro del pipeline DevSecOps.
- Capacitar al equipo para interpretar correctamente los resultados.
- Actualizar periódicamente las bases de datos de vulnerabilidades.

La combinación de estas estrategias incrementa la precisión del proceso de auditoría y reduce la probabilidad de liberar vulnerabilidades sin detectar.

Tabla 1.14. Estrategias para minimizar errores de clasificación

Estrategia	Beneficio
Actualizar herramientas	Mejora la detección de vulnerabilidades recientes.
Combinar metodologías	Reduce la posibilidad de falsos negativos.
Validación manual	Confirma la existencia de vulnerabilidades críticas.
Automatización en DevSecOps	Ejecuta análisis continuos durante el desarrollo.
Capacitación del equipo	Facilita la interpretación adecuada de los resultados.

Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP) y su aplicación al desarrollo de software

¿Qué es la LFPDPPP?

La **Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP)** es una legislación mexicana cuyo propósito consiste en regular el tratamiento legítimo, controlado e informado de los datos personales que realizan personas físicas y morales del sector privado.

Su objetivo principal es garantizar que toda información personal sea utilizada de manera responsable, respetando los derechos fundamentales de los titulares y estableciendo obligaciones para quienes recopilan, almacenan, procesan o transfieren dichos datos.

Para un desarrollador de software, esta ley implica la necesidad de diseñar aplicaciones que incorporen mecanismos adecuados para proteger la confidencialidad, integridad y disponibilidad de la información personal, además de facilitar el ejercicio de los derechos que la legislación reconoce a los usuarios.

¿Qué son los datos personales?

La LFPDPPP considera como **datos personales** toda información concerniente a una persona física identificada o identificable.

Entre los ejemplos más comunes se encuentran:

- nombre completo;
- matrícula o número de identificación;
- correo electrónico;
- domicilio;
- teléfono;
- fotografía;
- dirección IP cuando puede asociarse a una persona;
- registros de acceso;
- identificadores de usuario;
- credenciales de autenticación.

Existen además los **datos personales sensibles**, cuya utilización indebida puede generar discriminación o afectar significativamente a su titular.

Estos incluyen información relacionada con:

- estado de salud;
- origen étnico;
- creencias religiosas;
- opiniones políticas;
- orientación sexual;
- datos biométricos.

Aunque **InterLockerUp** no procesa información sensible de este tipo, sí administra una cantidad considerable de datos personales que requieren protección conforme a la legislación vigente.

Obligaciones de la LFPDPPP relevantes para un desarrollador de software

El cumplimiento de la LFPDPPP implica diversas responsabilidades técnicas y organizacionales para quienes desarrollan aplicaciones.

Las más importantes se describen a continuación.

Aviso de privacidad

Toda aplicación que recopile datos personales debe informar claramente al usuario:

- qué información será recopilada;
- con qué finalidad será utilizada;
- quién será el responsable del tratamiento;
- cómo podrá ejercer sus derechos.

En **InterLockerUp**, el sistema debería presentar un aviso de privacidad durante el proceso de registro o primer acceso, explicando el tratamiento de datos como nombres, matrículas, correos electrónicos y registros de uso de lockers.

Principio de finalidad: Los datos personales únicamente deben utilizarse para las finalidades previamente informadas al titular.

Por ejemplo, la matrícula de un estudiante obtenida para administrar la asignación de lockers no debería utilizarse posteriormente para fines publicitarios o comerciales sin autorización expresa.

Principio de proporcionalidad: La aplicación debe solicitar únicamente la información estrictamente necesaria para proporcionar el servicio.

En el caso de **InterLockerUp**, solicitar datos como nombre, matrícula y correo institucional resulta razonable para administrar el acceso a los lockers. En cambio, solicitar información adicional que no tenga relación con la funcionalidad del sistema representaría un incumplimiento del principio de proporcionalidad.

Medidas de seguridad: Los responsables del tratamiento deben implementar controles técnicos y administrativos que protejan la información contra:

- pérdida;
- alteración;
- acceso no autorizado;
- destrucción;
- robo;
- divulgación.

Desde el punto de vista del desarrollo seguro, estas medidas incluyen:

- cifrado de comunicaciones mediante HTTPS;
- almacenamiento seguro de contraseñas utilizando algoritmos de hash;
- autenticación mediante JWT;
- control de acceso basado en roles (RBAC);
- registros de auditoría;
- copias de seguridad;
- monitoreo de incidentes.

Muchas de estas medidas ya forman parte de la arquitectura propuesta para **InterLockerUp**.

Confidencialidad: Toda persona con acceso a datos personales debe garantizar su confidencialidad.

En términos prácticos, esto significa que:

- los desarrolladores no deben exponer bases de datos;
- las credenciales no deben almacenarse dentro del código fuente;
- las variables sensibles deben mantenerse en archivos de configuración protegidos;
- únicamente los usuarios autorizados deben acceder a la información correspondiente.

Derechos ARCO

La LFPDPPP reconoce cuatro derechos fundamentales para los titulares de los datos personales.

Estos derechos se conocen como **ARCO**:

- **Acceso:** conocer qué datos posee la organización.
- **Rectificación:** corregir información incorrecta.
- **Cancelación:** solicitar la eliminación de los datos cuando proceda.
- **Oposición:** negarse al tratamiento de la información en determinadas circunstancias.

Una aplicación que administre información personal debe contemplar procedimientos que permitan atender estas solicitudes.

Tabla 1.15. Obligaciones de la LFPDPPP aplicadas a InterLockerUp

Obligación	Aplicación en InterLockerUp
Aviso de privacidad	Informar el tratamiento de nombres, matrículas y correos institucionales.
Finalidad	Utilizar la información únicamente para administrar lockers y usuarios.
Proporcionalidad	Solicitar solo los datos indispensables para el funcionamiento del sistema.

Medidas de seguridad	HTTPS, JWT, RBAC, hash de contraseñas y registros de auditoría.
Confidencialidad	Restringir el acceso a la información mediante permisos adecuados.
Derechos ARCO	Permitir que los usuarios soliciten acceso, corrección o eliminación de sus datos cuando corresponda.

Datos personales tratados por InterLockerUp

Durante su funcionamiento, **InterLockerUp** procesa distintos tipos de información que deben protegerse conforme a la LFPDPPP.

Entre ellos destacan:

- nombre del estudiante;
- matrícula;
- correo institucional;
- contraseña cifrada;
- identificador del locker;
- historial de asignaciones;
- registros de apertura;
- fecha y hora de acceso;
- incidencias reportadas;
- evidencias fotográficas (cuando existan);
- información de administradores.

La protección adecuada de estos datos resulta indispensable para preservar la privacidad de los usuarios y evitar accesos no autorizados.

Relación entre la LFPDPPP y el desarrollo seguro

El cumplimiento de la LFPDPPP se encuentra estrechamente relacionado con las prácticas modernas de desarrollo seguro estudiadas en este capítulo.

Por ejemplo:

- **SAST** ayuda a detectar código que pueda exponer información personal.
- **DAST** identifica configuraciones inseguras que permitan la divulgación de datos.

- **SCA** verifica que las dependencias utilizadas no incorporen vulnerabilidades conocidas.
- **STRIDE** permite identificar amenazas relacionadas con la divulgación de información (**Information Disclosure**).
- **DevSecOps** automatiza controles orientados a proteger los datos durante todo el ciclo de desarrollo.

En consecuencia, el cumplimiento de la legislación no depende únicamente de políticas administrativas, sino también de la correcta implementación de controles técnicos dentro de la aplicación. En concordancia con lo anterior, se anexa el Aviso de Privacidad de InterLockerUp, el cual detalla el tratamiento y protección de los datos personales recopilados.

SA.2 — Modelado de amenazas STRIDE aplicado al caso de estudio

Introducción

El modelado de amenazas permite identificar riesgos de seguridad en la arquitectura de un sistema antes de que estos sean explotados. Para ello, se analizan los actores, procesos, almacenes de datos, flujos de información y límites de confianza que intervienen en el funcionamiento de la aplicación.

En este apartado se aplicará la metodología STRIDE al proyecto InterLockerUp, una plataforma para la gestión de lockers inteligentes que integra una aplicación web, una API REST, una base de datos PostgreSQL y dispositivos IoT basados en ESP32. El análisis incluirá la elaboración de un Diagrama de Flujo de Datos (DFD), la identificación de los Trust Boundaries, una tabla con amenazas específicas del proyecto, el análisis de las tres amenazas más críticas y sus respectivos controles de mitigación.

Con esto termina el apartado 2.1. El siguiente será 2.2 Descripción breve del sistema.

Descripción del sistema

InterLockerUp es un sistema desarrollado para la administración de lockers inteligentes dentro de una institución educativa. La plataforma permite que los estudiantes soliciten y utilicen lockers mediante una aplicación web, mientras que los administradores gestionan usuarios, préstamos y disponibilidad de los casilleros.

La arquitectura del sistema está compuesta por un **frontend desarrollado en React**, un **backend implementado en Node.js**, una **base de datos PostgreSQL** y dispositivos **ESP32** encargados del control físico de las cerraduras. Para la autenticación se utilizan **JSON Web Tokens (JWT)** y el acceso a los lockers se realiza mediante **códigos QR temporales**, los cuales son validados antes de autorizar la apertura.

La **Tabla 2.1** resume los principales componentes que serán considerados durante el modelado de amenazas.

Tabla 2.1. Componentes principales de InterLockerUp

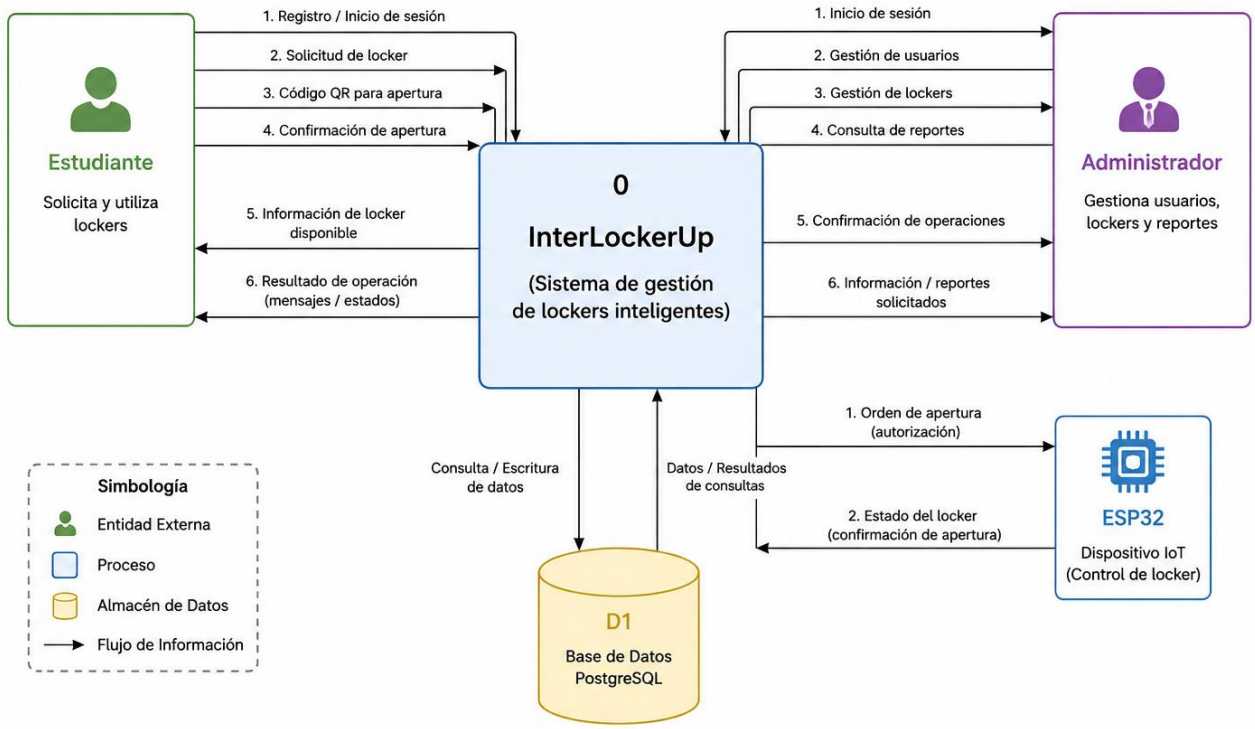
Componente	Función
Estudiante	Solicita y utiliza lockers.
Administrador	Gestiona usuarios, lockers y préstamos.
Frontend (React)	Interfaz web del sistema.
Backend (Node.js)	Procesa la lógica del negocio y las solicitudes.
API REST	Comunicación entre el frontend, backend y dispositivos IoT.
PostgreSQL	Almacena la información del sistema.
JWT	Autentica y autoriza las sesiones de los usuarios.
Código QR	Permite el acceso autorizado al locker.
ESP32	Controla la apertura física del locker.
Relay y cerradura electrónica	Ejecutan la apertura del locker.

Diagrama de Flujo de Datos (DFD)

El **Diagrama de Flujo de Datos (DFD)** representa gráficamente la forma en que la información circula dentro de InterLockerUp. Este diagrama permite identificar los actores que interactúan con el sistema, los procesos principales, los almacenes de datos y los flujos de información, proporcionando la base para la identificación de amenazas mediante la metodología STRIDE.

Para facilitar el análisis, se presentan dos niveles de detalle: un **DFD Nivel 0**, que muestra la interacción general del sistema con las entidades externas, y un **DFD Nivel 1**, donde se descomponen los procesos internos más relevantes.

Figura 2.1. Diagrama de Flujo de Datos (DFD) Nivel 0 de InterLockerUp

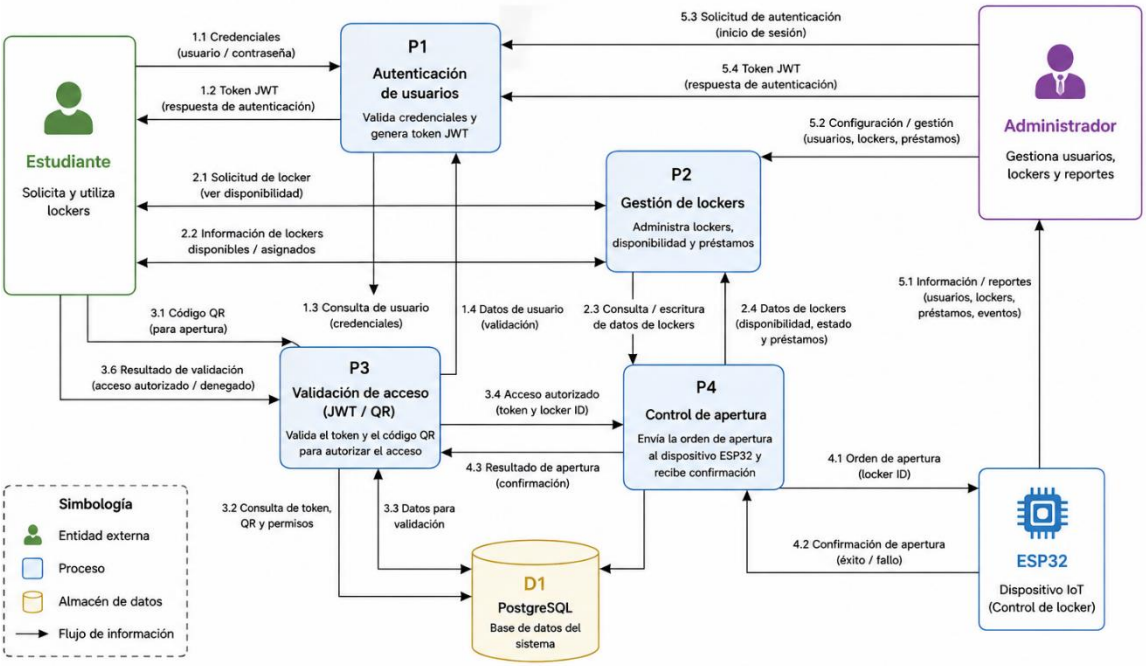


El DFD Nivel 0 muestra una vista general del sistema, donde los actores externos (**Estudiante**, **Administrador** y **ESP32**) interactúan con el proceso principal **InterLockerUp**. Este proceso consulta y almacena información en la base de datos PostgreSQL y envía las respuestas necesarias para la autenticación, la gestión de lockers y la autorización de apertura.

Diagrama de Flujo de Datos (DFD) Nivel 1

El **DFD Nivel 1** descompone el proceso principal de InterLockerUp en los procesos internos que participan en la autenticación, gestión de lockers y control de acceso. Este nivel de detalle permite identificar con mayor precisión los puntos donde podrían presentarse amenazas de seguridad y constituye la base para la aplicación de la metodología STRIDE.

Figura 2.2. Diagrama de Flujo de Datos (DFD) Nivel 1 de InterLockerUp



En este nivel se representan los procesos principales del sistema, los cuales interactúan con la base de datos PostgreSQL y con el dispositivo ESP32 para realizar la autenticación de usuarios, la administración de lockers y la autorización de apertura. Los flujos de información muestran el recorrido de las credenciales, los tokens JWT, los códigos QR y las respuestas enviadas a los usuarios y dispositivos, permitiendo identificar los puntos donde será aplicado el análisis de amenazas mediante STRIDE.

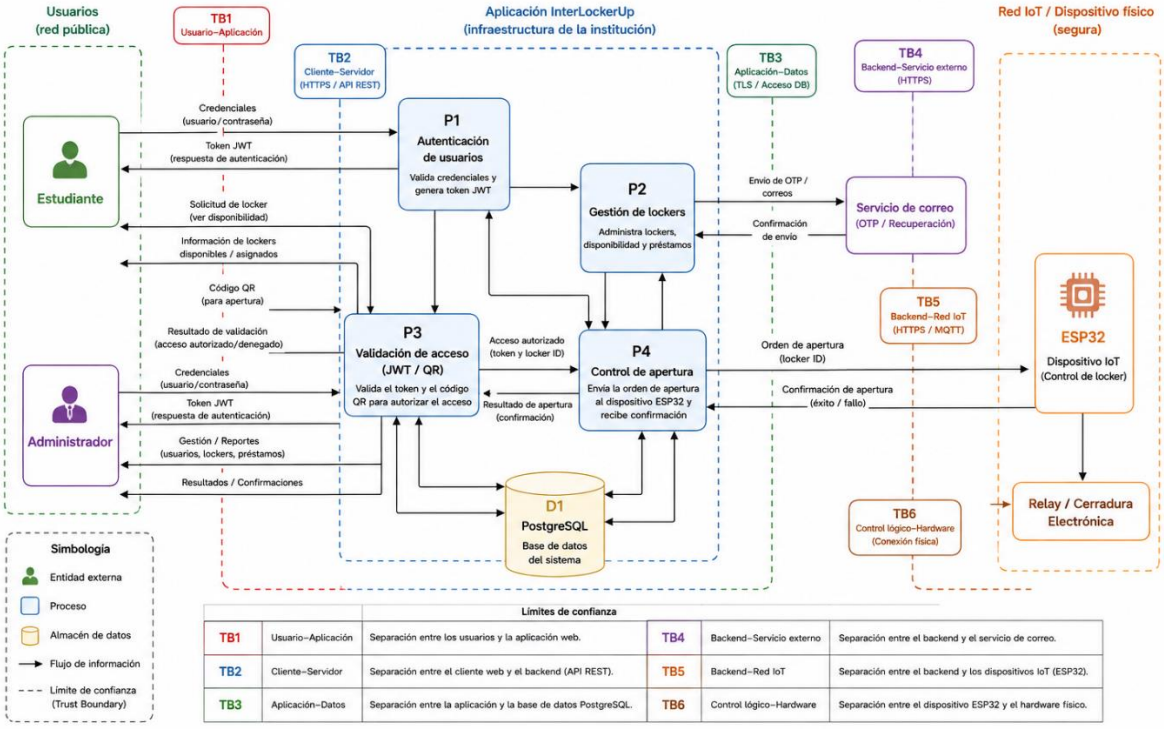
Límites de confianza (Trust Boundaries)

Los **Trust Boundaries** representan los puntos donde la información pasa de un entorno con un nivel de confianza a otro. Estos límites permiten identificar los flujos que requieren mayores controles de autenticación, autorización, cifrado y validación.

En InterLockerUp se identifican los siguientes límites de confianza:

ID	Límite de confianza	Componentes separados	Riesgo principal
TB1	Usuario–Aplicación	Estudiante/Administrador ↔ Frontend	Robo de credenciales y suplantación
TB2	Cliente–Servidor	Frontend ↔ Backend/API REST	Manipulación de solicitudes y tokens
TB3	Aplicación–Datos	Backend ↔ PostgreSQL	Alteración o divulgación de información
TB4	Backend–Servicio externo	Backend ↔ Servicio de correo	Intercepción o abuso de OTP
TB5	Backend–Red IoT	Backend/API ↔ ESP32	Suplantación del dispositivo y repetición de mensajes
TB6	Control lógico–Hardware	ESP32 ↔ Relay/Cerradura	Activación física no autorizada

Figura 2.3. Límites de confianza identificados en el DFD de InterLockerUp.



Límites de confianza					
TB1	Usuario–Aplicación	Separación entre los usuarios y la aplicación web.	TB4	Backend–Servicio externo	Separación entre el backend y el servicio de correo.
TB2	Cliente–Servidor	Separación entre el cliente web y el backend (API REST).	TB5	Backend–Red IoT	Separación entre el backend y los dispositivos IoT (ESP32).
TB3	Aplicación–Datos	Separación entre la aplicación y la base de datos PostgreSQL.	TB6	Control lógico–Hardware	Separación entre el dispositivo ESP32 y el hardware físico.

Tabla STRIDE

Con base en el DFD y en los límites de confianza identificados, se realizó el análisis de amenazas utilizando la metodología **STRIDE**. Las amenazas fueron definidas específicamente para la arquitectura de **InterLockerUp**, considerando sus componentes, flujos de información y dispositivos IoT.

Tabla 2.2. Análisis STRIDE aplicado a InterLockerUp

ID	Categoría STRIDE	Componente	Amenaza específica	Consecuencia
S1	**Spoofing**	Login	Suplantación de identidad mediante robo de credenciales.	Acceso no autorizado al sistema.
S2	**Spoofing**	ESP32	Suplantación del dispositivo ESP32 para recibir órdenes falsas.	Apertura no autorizada de lockers.
T1	**Tampering**	API REST	Modificación de solicitudes HTTP entre frontend y backend.	Alteración de operaciones del sistema.
T2	**Tampering**	PostgreSQL	Modificación de registros de lockers o préstamos.	Pérdida de integridad de la información.
R1	**Repudiation**	Registros de auditoría	Un usuario niega haber realizado una apertura.	Dificultad para realizar auditorías.
R2	**Repudiation**	Panel administrativo	Eliminación o alteración de registros de actividad.	Pérdida de trazabilidad.
I1	**Information Disclosure**	PostgreSQL	Exposición de datos personales de estudiantes.	Violación de la confidencialidad.
I2	**Information Disclosure**	JWT	Robo o filtración de tokens de autenticación.	Secuestro de sesiones.

D1	**Denial of Service**	API REST	Saturación de solicitudes al servicio.	Interrupción del sistema.
D2	**Denial of Service**	ESP32	Envío masivo de solicitudes de apertura.	Indisponibilidad del control de lockers.
E1	**Elevation of Privilege**	Panel administrativo	Escalamiento de privilegios de un usuario.	Acceso a funciones administrativas.
E2	**Elevation of Privilege**	Backend	Manipulación de permisos mediante vulnerabilidades de autorización.	Control total sobre la aplicación.

La tabla anterior identifica **12 amenazas específicas** relacionadas con la arquitectura de InterLockerUp, cubriendo las seis categorías del modelo STRIDE. Las amenazas fueron asociadas directamente a componentes presentes en el DFD, lo que facilita la identificación de los activos afectados y la propuesta de controles de mitigación en los apartados siguientes.

Análisis de las amenazas críticas

De las amenazas identificadas mediante STRIDE, se seleccionaron las tres de mayor impacto para InterLockerUp debido a que comprometen la autenticación, el acceso físico a los lockers y la disponibilidad del sistema.

Amenaza crítica 1 – Robo de credenciales (Spoofing)

Componente afectado: Inicio de sesión.

Descripción: Un atacante obtiene las credenciales de un usuario mediante phishing, filtraciones de datos o contraseñas débiles y las utiliza para autenticarse como un usuario legítimo.

Mecanismo de explotación: El atacante ingresa las credenciales robadas en el formulario de inicio de sesión. Si la autenticación es exitosa, obtiene un JWT válido y puede acceder a las funciones permitidas para ese usuario.

Consecuencia: Acceso no autorizado a la plataforma, consulta de información y posible apertura de lockers utilizando la identidad de la víctima.

Amenaza crítica 2 – Manipulación de la API REST (Tampering)

Componente afectado: API REST.

Descripción: Un atacante modifica las solicitudes enviadas entre el frontend y el backend para alterar parámetros relacionados con lockers, usuarios o permisos.

Mecanismo de explotación: Mediante herramientas de interceptación de tráfico, el atacante modifica el contenido de una petición HTTP antes de que llegue al servidor, intentando ejecutar acciones no autorizadas.

Consecuencia: Alteración de información del sistema, asignaciones incorrectas de lockers o ejecución de operaciones para las que el usuario no tiene autorización.

Amenaza crítica 3 – Denegación de servicio sobre la API (Denial of Service)

Componente afectado: API REST.

Descripción: Consiste en enviar un gran número de solicitudes al servidor con el objetivo de consumir sus recursos y afectar la disponibilidad del sistema.

Mecanismo de explotación: El atacante automatiza el envío masivo de peticiones hacia los servicios de autenticación o gestión de lockers, saturando la capacidad de procesamiento del backend.

Consecuencia: Los usuarios legítimos no pueden iniciar sesión, consultar lockers o realizar aperturas mientras el servicio permanece indisponible.

Análisis

Las amenazas seleccionadas representan los riesgos más importantes para InterLockerUp porque afectan directamente la autenticación de usuarios, la integridad de las operaciones y la disponibilidad del servicio. Su explotación podría comprometer tanto la información almacenada como el funcionamiento de los lockers inteligentes, por lo que requieren controles de seguridad específicos para reducir su probabilidad de ocurrencia.

Controles de mitigación

Con base en las amenazas críticas identificadas, se proponen los siguientes controles de seguridad para reducir la probabilidad de explotación y minimizar su impacto sobre la operación de InterLockerUp.

Tabla 2.3. Controles de mitigación para las amenazas críticas

Amenaza	Control de mitigación propuesto	Justificación
Robo de credenciales (Spoofing)	Implementar autenticación multifactor (MFA), políticas de contraseñas robustas, bloqueo temporal tras múltiples intentos fallidos y almacenamiento de contraseñas con algoritmos de hash seguros (bcrypt).	Reduce el riesgo de acceso no autorizado incluso cuando las credenciales han sido comprometidas.
Manipulación de la API REST (Tampering)	Validar todas las entradas del usuario, verificar la autorización en cada endpoint, utilizar HTTPS para proteger la comunicación y aplicar validación del lado del servidor.	Evita la modificación de solicitudes y protege la integridad de la información intercambiada entre el cliente y el servidor.
Denegación de servicio sobre la API (DoS)	Configurar **Rate Limiting**, limitar el número de solicitudes por dirección IP, establecer tiempos de espera (timeouts) y monitorear el tráfico mediante registros y alertas.	Disminuye el impacto de ataques de saturación y mantiene la disponibilidad del servicio para los usuarios legítimos.

Análisis

Los controles propuestos están orientados a mitigar los riesgos con mayor impacto identificados durante el modelado de amenazas. Su implementación fortalece la autenticación de usuarios, protege la integridad de las comunicaciones entre los componentes del sistema y mejora la disponibilidad de los servicios críticos de InterLockerUp.

SA.3 — SAST aplicado al caso de estudio (SonarQube + Semgrep)

En este apartado se realizó un análisis estático (SAST) sobre el backend del proyecto InterLockerUp utilizando SonarQube. El objetivo fue identificar vulnerabilidades, errores de programación y problemas de calidad del código para proponer acciones de mejora antes del despliegue de la aplicación. Posteriormente, los resultados serán complementados con el análisis realizado mediante Semgrep.

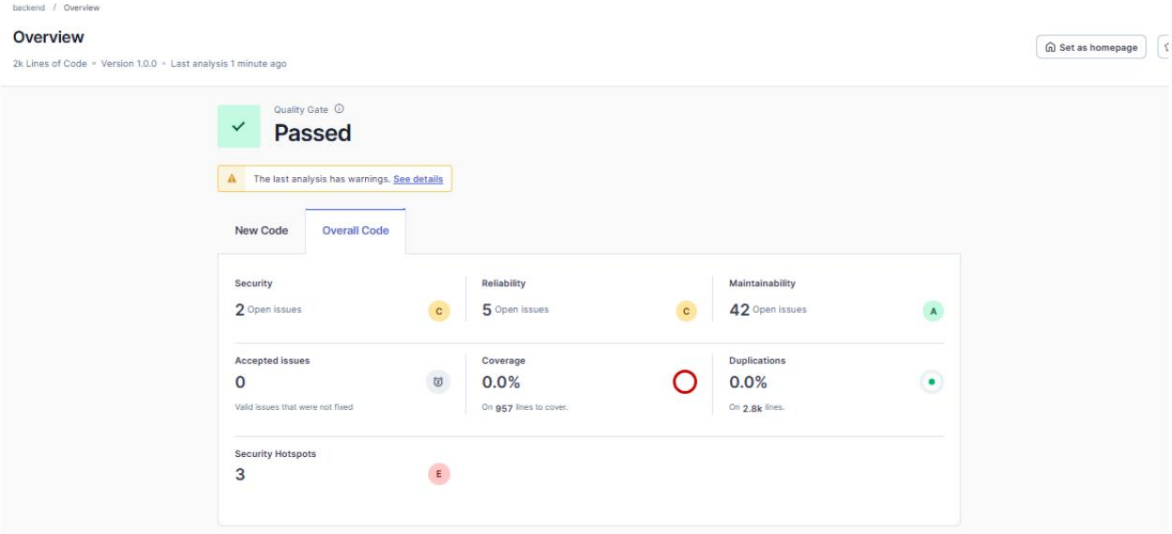
Resultados del análisis con SonarQube

Aquí reutilizamos la información del PDF que se hizo anteriormente en una práctica pasada: https://drive.google.com/file/d/1wl5cABCDE86URYpLrodJzlGG13fHYDnz/view?usp=drive_link

Tabla 3.1 Resumen del análisis

Métrica	Resultado
Security	2 vulnerabilidades (C)
Bugs	5 (C)
Code Smells	42 (A)
Security Hotspots	3 (E)
Código duplicado	0.0 %
Cobertura	0.0 %

Figura 3.1. Dashboard inicial de SonarQube para el backend de InterLockerUp.



La Figura 3.1 muestra el resumen generado por SonarQube después del análisis del backend de InterLockerUp. Se identificaron **2 vulnerabilidades de seguridad**, **5 bugs**, **42 code smells**, **3 security hotspots**, **0.0 % de código duplicado** y una **cobertura de pruebas del 0.0 %**, información que sirvió como base para documentar los hallazgos y definir las acciones de corrección.

Hallazgos de seguridad

Hallazgo 1. Exposición de información del framework

Campo	**Información**
Archivo	src/index.js
Tipo	Vulnerabilidad de seguridad
Severidad	Media
CWE	CWE-200 (Exposure of Sensitive Information to an Unauthorized Actor)
Descripción	La aplicación exponía información del framework mediante la cabecera HTTP X-Powered-By, lo que podría facilitar la identificación de tecnologías por parte de un atacante.

Corrección	Se deshabilitó la cabecera X-Powered-By.

Hallazgo 2. Riesgo de saturación por límite de carga

Campo	Información
Archivo	src/routes/incidentsRoutes.js
Línea	Línea 21
Tipo	Vulnerabilidad de seguridad
Severidad	Alta
CWE	CWE-400 (Uncontrolled Resource Consumption)
Descripción	El middleware multer aceptaba archivos sin límite de tamaño, permitiendo cargas excesivas que podían afectar el rendimiento del servidor.
Corrección	Se configuró un límite máximo de 5 MB para los archivos cargados.

Cobertura de pruebas

A pesar de que en el análisis inicial de calidad ejecutado con SonarQube el tablero general reflejó un indicador de cobertura del 0.0% debido a que el escáner estático básico no incorpora de forma automática los reportes locales en formato LCOV durante su escaneo estándar, el proyecto cuenta formalmente con una suite de pruebas unitarias e integración desarrollada con Jest y Supertest

La ejecución local de la suite de pruebas sobre el backend de InterLockerUp arrojó un total de 103 pruebas aprobadas con éxito distribuidas en 15 conjuntos (Test Suites), alcanzando un porcentaje de cobertura real del 88.94% en líneas y 88.61% en sentencias.

Resumen y Desglose de Cobertura de Pruebas Unitarias

Archivo del Controlador	% Stmts	% Branch	% Funcs	% Lines
accessController.js	95.55	100	100	95.55
adminController.js	96.66	90	100	100
auditsController.js	100	100	100	100
authController.js	86	71.42	100	85.71
dashboardController.js	100	100	100	100
historyController.js	100	100	100	100
homeController.js	96.55	66.66	100	100

incidentsController.js	97.72	50	100	97.67
lockerController.js	53.71	38.09	55.55	54.16
lockerRequestController.js	100	92.3	100	100
notificationController.js	100	100	100	100
perfilController.js	100	100	100	100
qrController.js	100	100	100	100
registerUserController.js	100	100	100	100
usersController.js	97.56	60	100	97.56
Promedio Total / Global	88.61	79.48	91.11	88.94

Test Suites: 15 pasadas, 15 en total.

Pruebas Totales: 103 aprobadas de 103.

Tiempo de ejecución: 5.341 s.

El diseño metodológico, los escenarios de prueba evaluados (casos de éxito, errores y límites) y las evidencias de ejecución correspondientes a cada controlador se encuentran íntegramente documentadas en el siguiente documento:
https://docs.google.com/document/d/11pUWRDc8vxX8wrgenFaA8GV712ZSdeQEIE4QY3rCSDw/edit?usp=sharing

Análisis con Semgrep

¿Qué es Semgrep y para qué sirve?

Semgrep es una herramienta de análisis estático de código fuente (SAST) de código abierto y alta velocidad que permite buscar patrones de vulnerabilidades, errores lógicos y violaciones de políticas de seguridad directamente en el código base sin necesidad de ejecutar la aplicación. Su propósito principal es automatizar la detección temprana de fallos de seguridad (como los del OWASP Top 10) facilitando su corrección antes de que el software sea desplegado a producción.

Proceso de Configuración y Ejecución del Análisis

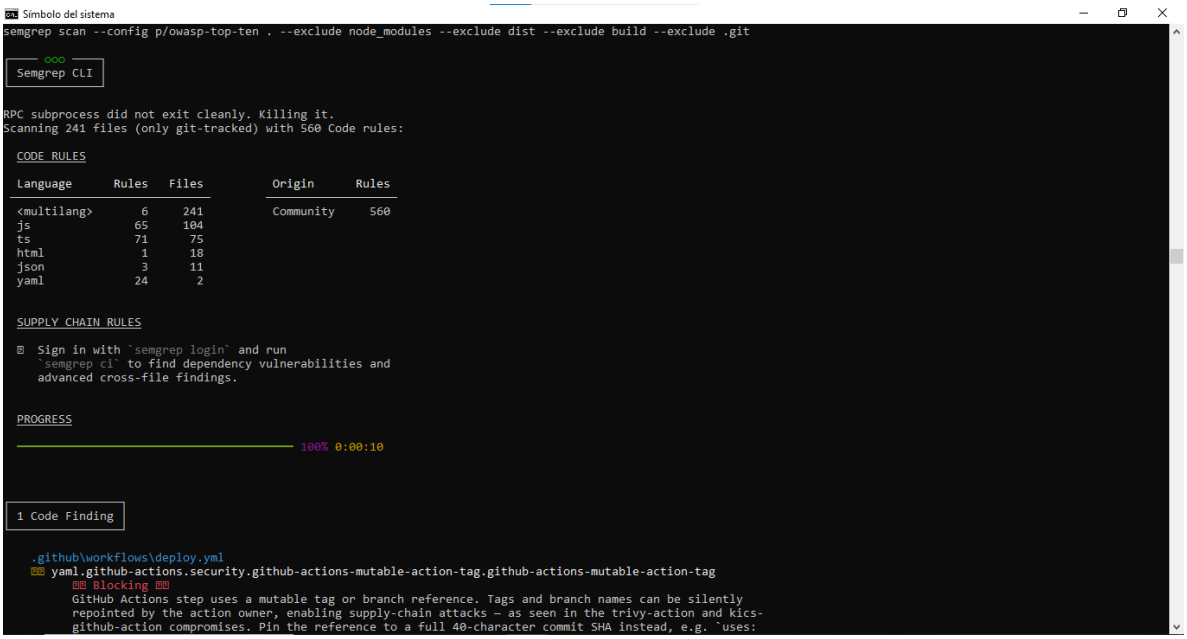
Para realizar el escaneo oficial bajo el conjunto de reglas del OWASP Top 10, fue necesario realizar un proceso de autenticación previa ejecutando el comando de inicio de sesión (semgrep login), lo que permitió conectar la CLI con la plataforma:

semgrep login

Asimismo, durante la ejecución inicial se detectó la necesidad de excluir directorios dependientes y pesados del sistema (como node_modules, dist, build, archivos de control de versiones .git y carpetas de cobertura) para optimizar el rendimiento del motor de análisis mediante la exclusión explícita con el parámetro --exclude. El comando final ejecutado para realizar el escaneo del código fuente fue el siguiente:

semgrep scan --config p/owasp-top-ten . --exclude node_modules --exclude dist --exclude build --exclude .git

Figura 3.2. Evidencia de la terminal ejecutando el escaneo con el conjunto de reglas p/owasp-top-ten, revisando 241 archivos y detectando 1 hallazgo bloqueante en las configuraciones de despliegue.



Resultados del Análisis con Semgrep

El análisis estático procesó 241 archivos rastreados por Git aplicando 105 reglas orientadas al estándar del OWASP Top 10, arrojando un total de 1 hallazgo bloqueante relacionado con la seguridad en la cadena de suministro y la configuración de integración continua.

Métrica / Regla	**Resultado**
Herramienta	Semgrep CLI
Reglas ejecutadas	105 reglas de código
Archivos analizados	241 archivos objetivo
Número total de hallazgos	1 hallazgo bloqueante

Hallazgo 1. Uso de etiquetas mutables en flujos de GitHub Actions

Campo	Detalle Técnico
Regla Aplicada	yaml.github-actions.security.github-actions-mutable-action-tag.github-actions-mutable-action-tag
Nivel de Severidad	Bloqueante (Blocking)
Archivo Afectado	.github\workflows\deploy.yml (Línea 12)
Descripción del Hallazgo	El flujo de trabajo utiliza una referencia mutable o etiqueta de rama (por ejemplo, uses: appleboy/ssh-action@master). Las etiquetas y nombres de ramas pueden ser modificados silenciosamente por el propietario de la acción, habilitando posibles ataques a la cadena de suministro (supply-chain attacks).
Propuesta de Solución	Fijar la referencia utilizando un hash de confirmación (commit SHA) completo de 40 caracteres en lugar de una etiqueta variable, asegurando la versión exacta y confiable del script ejecutado en el despliegue.

SA.4 — DAST aplicado al caso de estudio (OWASP ZAP)

¿Qué es OWASP ZAP y para qué sirve?

OWASP ZAP (Zed Attack Proxy) es una de las herramientas de código abierto más populares del mundo para realizar pruebas de seguridad dinámicas en aplicaciones web (DAST - Dynamic Application Security Testing). Funciona como un proxy de interceptación y un escáner automatizado que permite simular ataques informáticos reales contra una aplicación en ejecución. Su propósito principal es identificar vulnerabilidades de seguridad, fallas de configuración, cabeceras ausentes y riesgos expuestos antes de que el software sea desplegado en un entorno de producción.

Instalación

Para llevar a cabo la evaluación del caso de estudio, la herramienta fue instalada localmente en el equipo de desarrollo en Sistema operativo Windows.

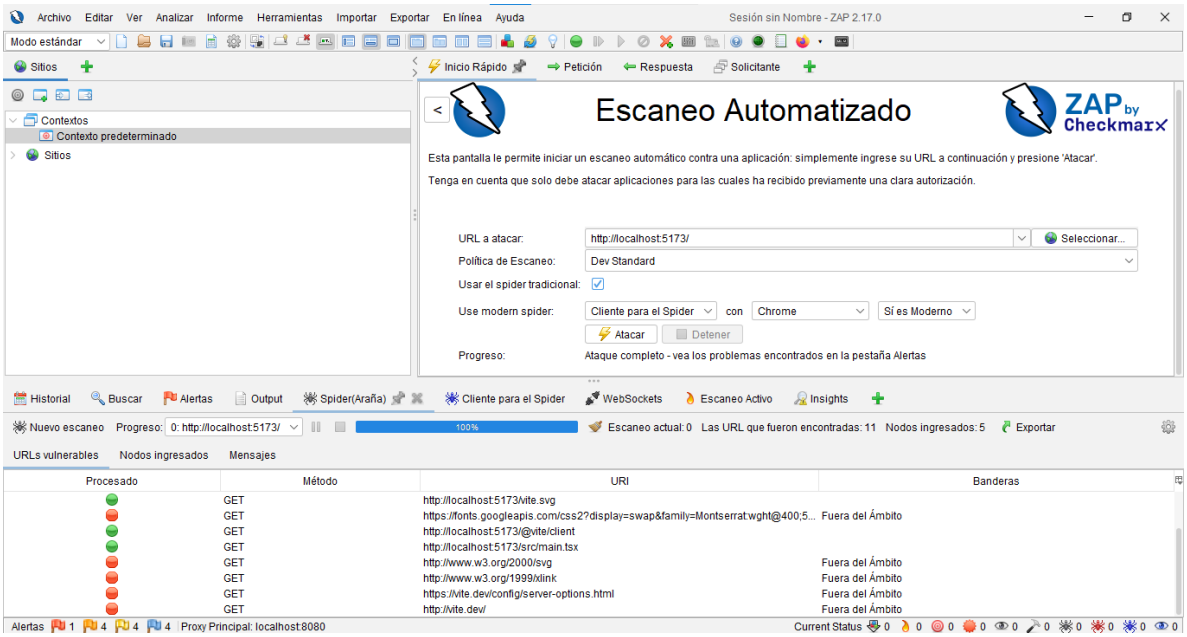
Se descargó el instalador oficial de la versión 2.17.0 desde el sitio web oficial de OWASP ZAP (zaproxy.org) y se ejecutó el asistente de configuración estándar para disponer de la aplicación de escritorio.

Escaneo DAST

Para llevar a cabo el escaneo DAST con OWASP ZAP, primero se encendieron los servicios locales del proyecto verificando que la interfaz web respondiera correctamente en la URL http://localhost:5173. Posteriormente, se abrió la aplicación OWASP ZAP, se ingresó dicha dirección local en el campo de inicio rápido (Quick Start) y se ejecutó el proceso de ataque automatizado, permitiendo que la herramienta analizara el árbol de directorios y los endpoints expuestos hasta completar el 100% del análisis.

Imagen 4.1: Escaneo automatizado completado al 100% en OWASP ZAP. La imagen muestra la interfaz de OWASP ZAP que muestra el proceso de escaneo finalizado exitosamente al 100%. En ella se visualiza el árbol de directorios y las URLs analizadas del entorno local (http://localhost:5173), lo

que confirma que la herramienta completó de forma correcta la exploración dinámica de la aplicación.



Reporte ZAP y Clasificación de Riesgos

Una vez finalizado el análisis dinámico, OWASP ZAP generó de forma automática un informe detallado con los hallazgos de seguridad encontrados en el caso de estudio, clasificándolos por su nivel de riesgo (Alto, Medio, Bajo e Informativo) y nivel de confianza. El reporte completo en formato PDF fue exportado y puede ser consultado en el siguiente link: https://drive.google.com/file/d/13XJibWCPU9fObTQJK7ieeZ_74FEohBcE/view?usp=sharing

Imagen 4.2: Resumen general de alertas clasificadas por nivel de riesgo obtenidas en el reporte de OWASP ZAP.

Summaries

Alert Counts by Risk and Confidence

This table shows the number of alerts for each level of risk and confidence included in the report.

(The percentages in brackets represent the count as a percentage of the total number of alerts included in the report, rounded to one decimal place.)

		Confidence				
		Confirmado por Usuario	Alta	Media	Baja	Total
Risk	Alto	0 (0,0 %)	0 (0,0 %)	0 (0,0 %)	1 (7,7 %)	1 (7,7 %)
	Medio	0 (0,0 %)	2 (15,4 %)	2 (15,4 %)	0 (0,0 %)	4 (30,8 %)
	Bajo	0 (0,0 %)	1 (7,7 %)	2 (15,4 %)	1 (7,7 %)	4 (30,8 %)
	Informativo	0 (0,0 %)	0 (0,0 %)	3 (23,1 %)	1 (7,7 %)	4 (30,8 %)
	Total	0 (0,0 %)	3 (23,1 %)	7 (53,8 %)	3 (23,1 %)	13 (100%)

A continuación, se presenta una tabla resumen que muestra los tipos de alertas detectadas, su nivel de riesgo y la cantidad de incidencias arrojadas durante la ejecución:

Tipo de Alerta	Nivel de Riesgo	Conteo / Porcentaje
Ruta Transversal	Alto	17 (130,8 %)
Cabecera Content Security Policy (CSP) no configurada	Medio	3 (23,1 %)
Configuración Incorrecta Cross-Domain	Medio	1 (7,7 %)
Falta atributo de integridad de recursos secundarios	Medio	3 (23,1 %)
Falta de cabecera Anti-Clickjacking	Medio	3 (23,1 %)
Divulgación de Marcas de Tiempo - Unix	Bajo	10 (76,9 %)
Falta encabezado X-Content-Type-Options	Bajo	5 (38,5 %)
Revelación de IP privada	Bajo	1 (7,7 %)
Strict-Transport-Security Header No Establecido	Bajo	3 (23,1 %)

Aplicación Web Moderna	Informativo	3 (23,1 %)
Divulgación de Información - Información sensible en URL	Informativo	1 (7,7 %)
Divulgación de información - Comentarios sospechosos	Informativo	24 (184,6 %)
Reexaminar las Directivas de Control de Caché	Informativo	1 (7,7 %)
Total	—	**13**

Análisis de Alertas

1. Alerta High

Campo	Detalle
Nombre de la Alerta	Ruta Transversal (Path Traversal)
Nivel de Riesgo	Alto (*High*)
URL / Endpoint Afectado	http://localhost:5173/node_modules/.vite/deps/@radix-ui_react-dropdown-menu.js?v=%40radix-ui_react-dropdown-menu.js
Descripción de la Vulnerabilidad	La técnica de ataque Path Traversal permite a un atacante acceder a los archivos, directorios y comandos que residen fuera del directorio raíz de documentos web. Un atacante puede controlar una URL de tal forma que el sitio web ejecutará o mostrará información de archivos arbitrarios ubicados en cualquier parte del servidor web.
Propuesta de solución	Para evitar este ataque, se debe prohibir el uso de símbolos para navegar entre carpetas (como barras o puntos suspensivos), aceptar únicamente nombres y rutas aprobadas mediante una lista estricta, y revisar la entrada de forma segura antes de usarla para impedir que los atacantes engañen al sistema con trucos de codificación.

Explicación del mecanismo de ataque de la alerta más crítica

La alerta más crítica identificada durante el análisis de seguridad fue Ruta Transversal (Path Traversal), clasificada con un nivel de riesgo Alto (High). Este tipo de ataque consiste en manipular las rutas de acceso a archivos mediante secuencias especiales como ../, con el objetivo de salir del directorio permitido por la aplicación y acceder a archivos o carpetas que deberían permanecer protegidos. Aunque muchos servidores implementan mecanismos para bloquear este comportamiento, los atacantes pueden

utilizar diferentes técnicas de evasión, como codificaciones alternativas (..%u2216, ..%c0%af), barras invertidas en sistemas Windows (..\) o doble codificación URL (..%255c), con la finalidad de evitar los filtros de seguridad.

Si la aplicación no valida correctamente la entrada proporcionada por el usuario, un atacante podría manipular una URL o un parámetro para acceder a archivos confidenciales del servidor, obtener información sensible, visualizar código fuente o incluso ejecutar acciones no autorizadas. En este análisis, la alerta fue detectada en el endpoint http://localhost:5173/node_modules/.vite/deps/@radix-ui_react-dropdown-menu.js?v=@radix-ui_react-dropdown-menu.js, donde la herramienta identificó un posible riesgo asociado al acceso a rutas de archivos. Este tipo de vulnerabilidad es considerado crítico porque puede comprometer la confidencialidad de la información almacenada en el servidor y facilitar ataques posteriores si no se implementan controles adecuados para validar y restringir el acceso a los recursos del sistema.

2. Cabecera de Seguridad HTTP Ausente Identificada

Campo	**Detalle**
Cabecera Ausente	Content-Security-Policy (CSP)
Endpoint / Contexto	http://localhost:5173/
Referencias (CWE / WASC)	CWE-693 (Fallo en el mecanismo de protección) / WASC-15
Descripción e Impacto	La respuesta de la aplicación no incluye la cabecera CSP. Su ausencia omite una capa fundamental de seguridad encargada de mitigar ataques de Cross-Site Scripting (XSS) y de inyección de datos, permitiendo que los navegadores ejecuten o carguen sin restricciones fuentes de contenido externas no aprobadas.
Posible Solución / Implementación	Asegúrese de que su servidor web, servidor de aplicaciones o framework (como Node.js/Express con la librería helmet o mediante middleware manual) esté configurado para establecer la cabecera Content-Security-Policy en todas las respuestas HTTP salientes (por ejemplo: default-src 'self'; script-src 'self').

SA.5 — SCA: análisis de dependencias del caso de estudio

Introducción y Metodología

En el marco de la auditoría de seguridad del proyecto InterLockerUp, se ha llevado a cabo un Análisis de Composición de Software (SCA, por sus siglas en inglés). El objetivo principal de esta fase es identificar, clasificar y mitigar los riesgos asociados con el uso de dependencias de terceros y componentes de código abierto. Dado que el backend del sistema está desarrollado en el entorno de ejecución Node.js, la seguridad de la cadena de suministro de software depende directamente del estado de los paquetes gestionados a través de NPM (Node Package Manager). La falta de actualización de estos componentes representa uno de los riesgos más críticos en la actualidad, categorizado sistemáticamente en el OWASP Top 10 (A06:2021 – Vulnerable and Outdated Components).

Herramienta SCA Ejecutada

Para llevar a cabo este análisis con rigor profesional, se ejecutó la herramienta nativa de auditoria de Node.js directamente sobre el repositorio del backend. Esta elección garantiza la compatibilidad absoluta con el árbol de dependencias actual del proyecto.

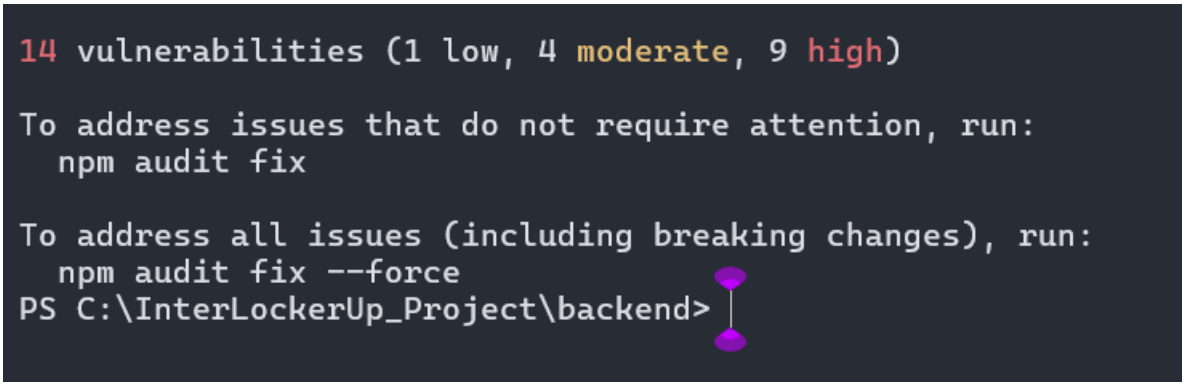
- **Herramienta:** npm audit.
- **Alcance:** Árbol completo de dependencias de producción y desarrollo definidas en el archivo package.json y resueltas en package-lock.json.
- **Comando ejecutado:** npm audit.

Reporte generado: Diagnóstico de Vulnerabilidades

Tras la ejecución de la herramienta, el motor de auditoría procedió a comparar el árbol de dependencias local con la base de datos de vulnerabilidades globales (GitHub Advisory Database). El escaneo arrojó un total de 14 vulnerabilidades documentadas (CVEs/GHSAs :14), distribuidas de la siguiente manera según su nivel de severidad e impacto potencial:

- **Nivel Crítico (Critical):** 0.
- **Nivel Alto (High):** 9 vulnerabilidades.

- **Nivel Medio (Moderate):** 4 vulnerabilidades.
- **Nivel Bajo (Low):** 1 vulnerabilidad.



Documentación Detallada de Dependencias Vulnerables

A continuación, se documenta exhaustivamente cada una de las bibliotecas de terceros afectadas descubiertas durante el escaneo. Se detalla su versión comprometida, los identificadores de vulnerabilidad pública (GHSA) y el riesgo técnico especifico que suponen para la integridad del servidor de InterLockerUp.

Dependencia	Versión Actual	CVE/GHSA	Severidad	Descripción del Riesgo y Vector de Ataque
adm-zip	<0.6.0	GHSA-xcpc-8h2w-3j85	Alta	Riesgo de asignación masiva de memoria (hasta 4GB) al procesar archivos ZIP manipulados maliciosamente. Puede causar la caída total del proceso del servidor por agotamiento de recursos.
axios	1.0.0 - 1.17.0	GHSA-42h9-826w-cgv3 GHSA-xj6q-8x83-jv6g	Alta	Contiene múltiples vectores de ataque, incluyendo recursividad excesiva en parámetros JSON (DoS), contaminación de prototipos en

				campos de autentificación y evasión de filtros de proxies, comprometiendo peticiones HTTP salientes.
body-parser	2.0.0 - 2.2.2	GHSA-v422-hmwv-36x6	Alta	Vulnerabilidad de denegación de servicio (DoS) cuando un valor de límite inválido deshabilita silenciosamente la restricción de tamño del payload HTTP, permitiendo inyecciones masivas de datos.
brace-expansion	1.17.17 - 5.0.8	GHSA-f886-m6hf-6m8v GHSA-3jxr-9vmj-r5cp	Alta	Agotamiento de memoria y cuelgue de procesos (DoS) debido a expansiones de longitud ilimitada y la generación de matrices intermedias no controladas al parsear expresiones.
js-yaml	<=3.14.2	GHSA-h67p-54hq-rp68	Alta	Consumo cuadrático de CPU provocado mediante alias repetidos y cadenas de claves de fusión complejas en archivos YAML, facilitando ataques de Denegación de Servicio (DoS).
multer	1.0.0 - 2.1.1	GHSA-72gw-mp4g-v24j	Alta	**Critíco para InterLockerUp:** Al usar esta libreria para la carga de evidencias de incidentes, el

		GHSA-3p4h-7m6x-2hcm		sistema es vulnerable a ataques DoS mediante el envío de nombres de campos profundamente anidados y la limpieza incompleta de cargas abortadas
nodemailer	<=9.0.0	GHSA-vvjj-xcjg-gr5g GHSA-p6gq-j5cr-w38f	Alta	**Crítico para InterLockerUp:** Riesgo de inyección de comandos SMTP vía caracteres CRLF y lectura de archivos arbitrarios (SSRF). Un atacante podría interceptar o inyectar código en los correos de confirmación de asignación de lockers de los alumnos.
patch-to - regexp	8.0.0 - 8-3-0	GHSA-j3q9-mxjg-w52f	Alta	Denegación de servicio por evaluación de expresiones regulares (ReDoS) explotable, mediante el abuso de comodines múltiples y grupos opcionales secuenciales en el enrutamiento.
picomatch	<=2.3.1	GHSA-3v7f-55p6-f55p	Alta	Vulnerabilidad de tipo ReDos a través de cuantificadores extglob excesivos e inyección de métodos anómalos en clases de caracteres POSIX.
qs	6.11.1 - 6.15.1	GHSA-q8mj-m7cp-5q26	Media	Riesgo de DoS activable remotamente

				cuando se formatea una matriz con entradas nulas o indefinidas, forzando la terminación inesperada del analizador de consultas.
uuid	<11.1.1	GHSA-w5hq-g745-h8pq	Media	Ausencia de verificación de límites de búfer subyacente al momento de generar tokens o identificadores universales (UUID v3/v5).

Proceso de Actualización y Justificación Técnica de Remediación

En respuesta a los múltiples hallazgos críticos detectados que atentaban contra la estabilidad del backend, el equipo de desarrollo procedió a ejecutar el comando de reparación automatizada y controlada provisto por la herramienta SCA:

npm audit fix

Dependencias Actualizadas Exitosamente:

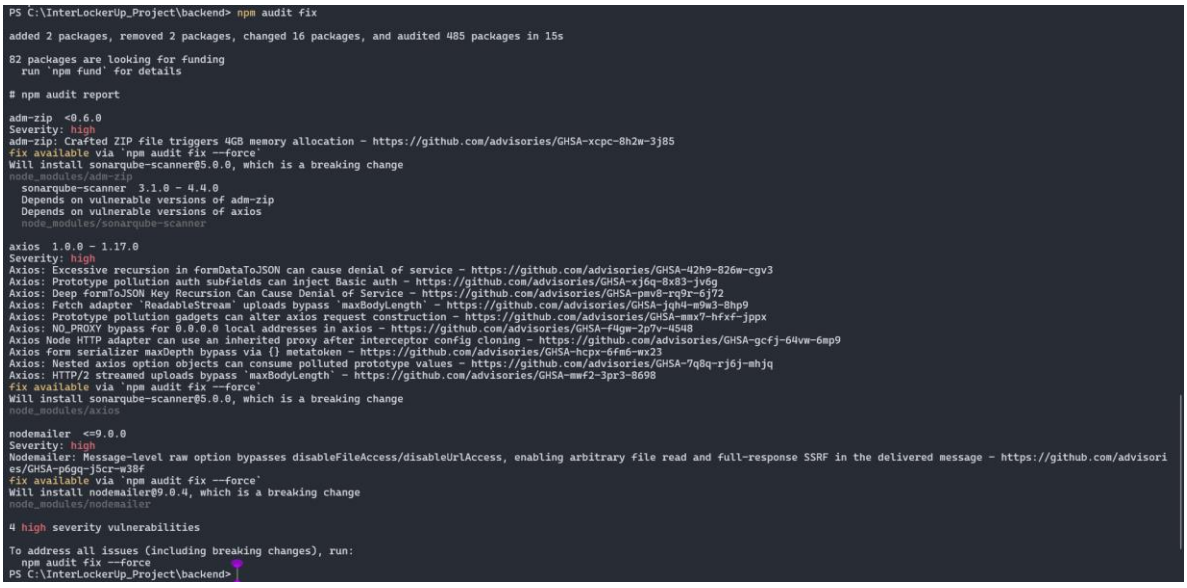
Mediante la ejecución de este comando conservador, se logró actualizar e instalar las versiones seguras de paquetes que interactúan directamente con el usuario final, tales como multer (gestor de evidencias gráficas), nodemailer (notificaciones por correo al alumno) y axios (peticiones HTTP internas). Estas dependencias fueron parcheadas a sus versiones inmunes mitigando por completo las amenazas de denegación de servicios (DoS) e inyección (CRLF/SSRF) sin requerir reestructuración del código fuente.

Justificación técnica de dependencias no forzadas:

El reporte inicial indicó que librerías subyacentes de desarrollo profundo (como adm-zip, que es una subdependencia de sonarqube-scanner)

requerían actualizaciones destructivas (“breaking changes”) para ser remediadas.

Se tomó la decisión fundamentada y técnica de NO ejecutar el comando invasivo npm audit fix –force. La justificación radica en que forzar la actualización de paquetes mayores rompería drásticamente la compatibilidad de los flujos de escaneo estático configurados previamente en la fase de análisis SAST de este mismo informe. La arquitectura priorizó salvaguardar la estabilidad operativa y análitica del sistema, asumiendo el riesgo residual bajo la primicia de que estas librerías afectadas se ejecutan exclusivamente en un entorno aisaldo de pruebas y no se exponen al tráfico web del entorno de producción.



Conclusión

El análisis de composición de software permitió sanear agresivamente la superficie de ataque derivada de la cadena de suministro de código abierto del proyecto. Al abordar rigurosamente las versiones vulnerables de librerías vitales de enrutamiento y procesamiento (como multer y body-parser), el backend de InterLockerUp ha blindado sus interfaces de programación (APIs) contra los ataques de agotamiento de recursos (DoS) y ejecución de secuencias no autorizadas. El proyecto cumple con las normativas de desarrollo seguro establecidas en el estándar OWASP, garantizando un ecosistema resiliente antes de pasar a su despliegue definitivo.

SA.6 — Configuración segura y hardening del caso de estudio

Introducción

La auditoría de seguridad tiene como propósito fundamental validar la robustez de las configuraciones de seguridad y las prácticas de endurecimiento (hardening) aplicadas al ecosistema de InterLockerUp. En un entorno digital donde las amenazas evolucionan constantemente, asegurar que cada componente del sistema desde la red hasta la base de datos esté configurado bajo estándares de máxima seguridad es vital para garantizar la integridad de los datos de los usuarios y la continuidad operativa del proyecto.

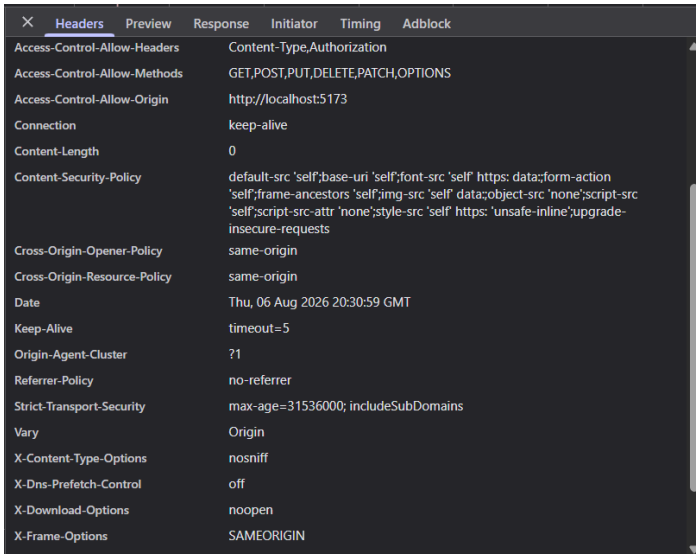
El proceso de Hardening consiste en asegurar el sistema reduciendo su superficie de vulnerabilidad. En el proyecto InterLockerUp, se evaluaron y aplicaron configuraciones avanzadas a nivel de red, aplicación y base de datos para cumplir con los estándares de Defensa en Profundidad.

Cabeceras HTTP de seguridad revisadas e implementadas

Para proteger la comunicación entre el cliente (frontend) y el servidor (backend), se integró el middleware helmet en el entorno de ejecución Node.js. La implementación de estas cabeceras es crucial en un entorno de producción, ya que actúan como la primera línea de defensa del navegador contra vectores de ataque comunes que eluden la lógica de aplicación tradicional. Sin estas políticas, las aplicaciones quedan expuestas a la ejecución de scripts no autorizados, el secuestro de sesiones mediante clickjacking y la interceptación de datos en tránsito. A continuación, se detallan las políticas aplicadas.

Cabecera HTTP	Estado	CWE Mitigado	Descripción y Justificación Técnica
Content-Security-Policy (CSP)	Implementada	CWE-79	Previene ataques XSS y de inyección de código limitando estrictamente las fuentes de contenido ejecutables en el navegador.

Strict-Transport-Security (HSTS)	Implementada	CWE-319	Fuerza la comunicación sobre canales cifrados HTTPS, previniendo ataques de tipo Man-in-the-Middle (MitM) y SSL Striping.
X-Frame-Options	Implementada	CWE-1021	Bloquea que la aplicación sea renderizada dentro de las etiquetas <¡frame> por dominios de terceros, mitigando ataques de Clickjacking.
X-Content-Type-Options	Implementada	CWE-430	Aplica el valor nosiff, forzando al cliente respetar el Content-Type declarado y preveniendo la ejecución anómala de archivos camuflados.
X-Powered-By	Eliminada	CWE-200	Se deshabilitó explícitamente para ocultar la tecnología subyacente (Express/Node.js) y evitar el escaneo pasivo de los atacantes (FingerPrinting).



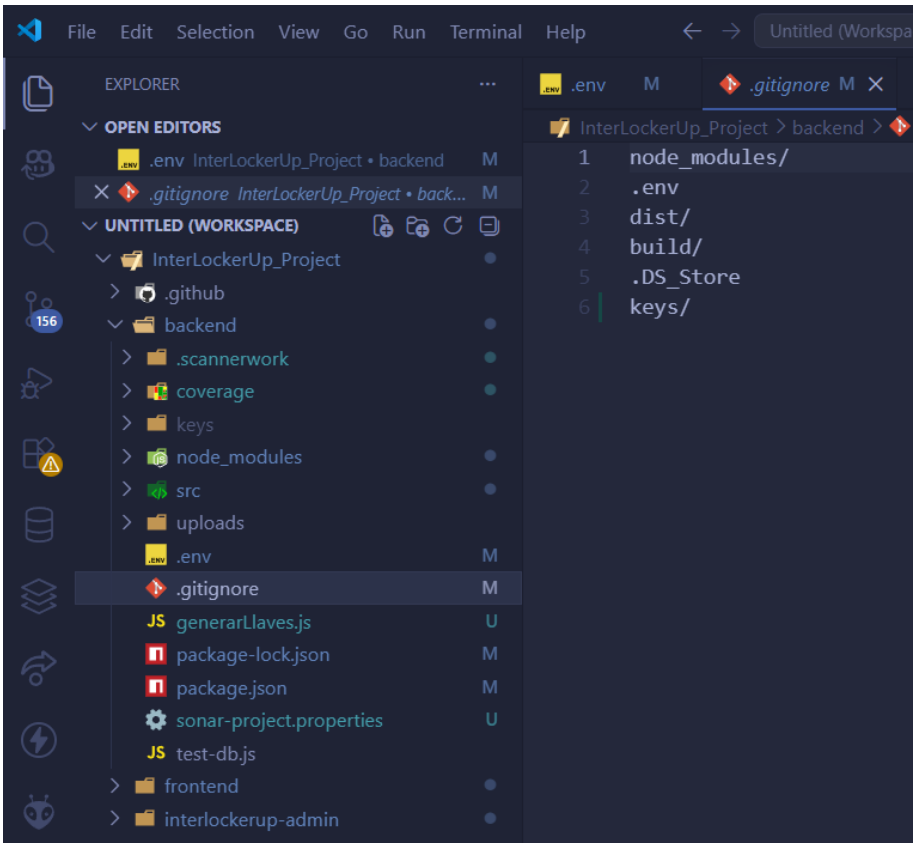
Verificación de cabeceras HTTP de seguridad en el cliente.

Estas implementaciones reducen significativamente la superficie de ataque al mitigar riesgos como XSS, Clickjacking y ataques de inyección, lo cual es fundamental para el cumplimiento de estándares de seguridad modernos.

Gestión de secretos revisada

Método de aislamiento: Inyección dinámica meidante variables de entorno (archivo .env excluido del repositorio).

Se auditó el código fuente garantizando que ningún secreto permanezca "harcodeado" (escrito en texto plano). Credenciales de la base de datos PostgreSQL, la clave secreta de firmado de los tokens JWT y las llaves del protocolo SMTP fueron movidas a un archivo .env. Mediante la librería y la adición estricta de la regla .env en el archivo .gitignore, se asegura que datos críticos jamás se filtren en el control de versiones.



Aislamiento de secretos y variables de entorno.

Logging de eventos de seguridad básico implementado

Se habilitó un mecanismo centralizado mediante un middleware personalizado en la capa superior del enrutamiento de Express para interceptar y auditar la actividad de la red:

- **Intentos de acceso anómalos:** Captura y registro en consola de códigos de estado 401 (Unauthorized) y 403 (Forbidden), evidenciando intentos de login fallidos o peticiones con tokens expirados, asociandolos a la IP origen del atacante.
- **Errores de Servidor:** Monitoreo de códigos 5xx, permitiendo identificar intentos de inyección o escaneos DAST agresivos que provoquen fallos lógicos en los controladores.

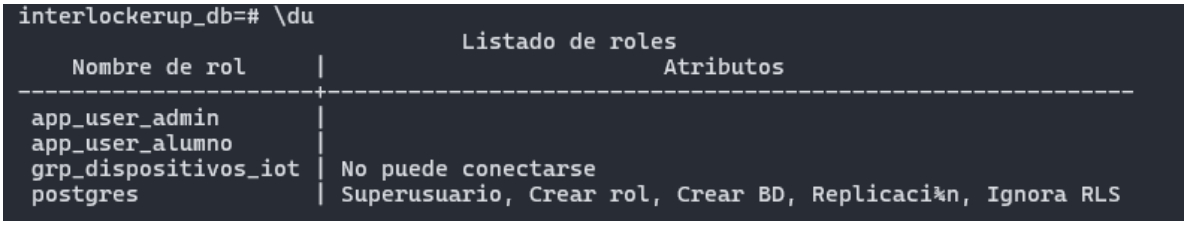


Monitoreo activo y registro de eventos de seguridad.

Configuración de defaults seguros verificada

De acuerdo al modelo de Confianza Cero y aplicando al Principio de Mínimo Privilegio, se certificó el endurecimiento en la base de datos PostgreSQL de InterLockerUp:

- **Ausencia de cuentas por defecto:** El superusuario administrador original (postgres) no se emplea en ninguna cadena de conexión al backend.
- **Privilegios fraccionados mediante RBAC:** Las operaciones son medidas a través de usuarios de servicio fuertemente limitados. El usuario *app_user_alumno* carece de permisos de comando DELETE, previniendo que un atacante que vulnere al backend pueda ejecutar un vaciado (Drop/Truncate) de las tablas transaccionales.



```
interlockerup_db=# \du
                                   Listado de roles
    Nombre de rol     |                         Atributos
----------------------+------------------------------------------------------------
 app_user_admin       |
 app_user_alumno      |
 grp_dispositivos_iot | No puede conectarse
 postgres             | Superusuario, Crear rol, Crear BD, Replicaci%n, Ignora RLS
```

Aplicación del Principio de Mínimo Privilegio (RBAC) en PostgreSQL.

Checklist de defensa en profundidad documentado

El sistema no depende de una sola capa de mitigación. Se validó la siguiente arquitectura de seguridad escalonada:

Capa de seguridad	Estado	Mecanismos Implementado en InterLockerUp
Validación de Entrada	Cumplido	Rechazo de payloads maliciosos y uso estricto de consultas SQL parametrizadas con pg-pool para evitar SQLi.
Autentificación	Cumplido	Integración de validación OAuth 2.0 (Google Login) o verificación de contraseñas mediante hash irreversible usando bycrypt.
Gestión de Sesión	Cumplido	Protección mediante JWT asimétricos validados en el authMiddleware.js para cada ruta protegida.
Cifrado	Cumplido	Cifrado asimétrico RSA-2048 para cargas críticas, y autenticación de la Base de datos bajo scram-sha-256.
Loggin y Auditoría	Cumplido	Trazabilidad estructurada de accesos HTTP fallidos para mitigar ataques de fuerza bruta.



Arquitectura de Defensa en Profundidad de InterLockerUp.

Conclusión

La implementación de estas medidas de seguridad reduce significativamente la superficie de ataque de InterLockerUp, reforzando el compromiso con la protección de datos y la continuidad operativa. Es fundamental destacar que la seguridad es un proceso continuo que requiere auditorías periódicas para adaptarse a nuevas amenazas y mantener la integridad del sistema.

Falta incluir resumen ejecutivo legible por audiencia no técnica del documento al inicio y agregar a la tabla de trazabilidad casos que salgan de SCA

Tabla de trazabilidad básica

A continuación, se presenta la tabla de trazabilidad unificada para el proyecto de InterLockerUp, la cual consolida de manera estructurada los principales hallazgos de seguridad detectados a lo largo de las distintas fases de evaluación.

Este instrumento permite relacionar directamente cada vulnerabilidad identificada con la herramienta que la descubrió, su clasificación dentro del estándar OWASP Top 10, su identificador oficial CWE (Common Weakness Enumeration) y su respectivo nivel de severidad, facilitando así la priorización de las acciones de mitigación y el seguimiento técnico dentro del ciclo de desarrollo seguro.

Hallazgo / Vulnerabilidad	**Herramienta de Detección**	**Categoría OWASP Top 10**	**CWE Asociado**	**Nivel de Severidad**
Exposición de información del framework	SonarQube (SAST)	A05:2021 – Security Misconfiguration	CWE-200	Media
Riesgo de saturación por límite de carga	SonarQube (SAST)	A04:2021 – Insecure Design	CWE-400	Alta
Uso de etiquetas mutables en GitHub Actions	Semgrep (SAST)	A06:2021 – Vulnerable and Outdated Components	CWE-1356	Bloqueante
Ruta Transversal (Path Traversal)	OWASP ZAP (DAST)	A01:2021 – Broken Access Control	CWE-22	Alta
Ausencia de cabecera Content-Security-Policy (CSP)	OWASP ZAP (DAST)	A05:2021 – Security Misconfiguration	CWE-693	Medio
Riesgo de denegación de servicio (DoS) por librería obsoleta	Npm audit(SCA)	A06:2021 – Vulnerable and Outdated Components	CWE-400	Alta

Firma del documento

Fecha	20/08/2026	Versión	1.0

OBJETIVO

El presente documento fue elaborado y revisado por los integrantes del equipo de trabajo, quienes participan en el desarrollo del proyecto InterLockerUp. Mediante el siguiente registro se establece la identificación de los responsables, la fecha de emisión y la versión correspondiente del documento, dejando constancia de su participación y validación.

Nombre	Participación	Firma
Ramírez Ramírez Elian Eduardo	✓	
Alcantara Mata Mauricio	✓	
López Rivera Luz Celeste	✓	
Hernández Cortés Valeria	✓	